Strang Derivations for stellar2d — a sympy-verified manuscript

stellar2d development notes
Department of Astronomy, Tsinghua University
`github.com/MahoMaho-Rize/stellar2d`

2026-05-08

Contents

1 Front matter

1.1 Purpose

This manuscript is an **internal, reproducibility-grade derivation document** for the Strang-split Euler solver of stellar2d (`src/gpu/explicit/strang_solver.{cu,cuh}`). It is a companion to the MHD book (`docs/derivations/mhd/`) and follows the same sympy-driven, strong-form-only methodology.

The book covers five parts, 36 sections in total:

- **Part A-phys** (A1–A6) — Compressible Euler equations in strong form: mass / momentum / total energy; flux Jacobians and their eigensystems; rotational covariance of the y-flux; entropy condition and the Lax criterion; simple-wave families.
- **Part A-num** (A7–A14) — Riemann-solver family (Rusanov, HLLE, HLLC, Roe) with contact-wave resolution cross-comparison; HLLC intermediate states; Davis wave-speed estimates with Einfeldt entropy fix; slope-limiter family (MC, minmod, van Leer, superbee, Ospre) in Sweby form; reconstruction-order hierarchy (donor-cell, MUSCL, PPM Colella-Woodward, PPM Colella-Sekora); MUSCL-Hancock half-step identity; time-integrator family (Strang, Lie, unsplit VL2, RK2-MUSCL) via BCH expansion; the `step` operator chain.
- **Part B** (B1–B6) — Perturbation storage $(\delta\rho, m_x, m_y, \delta E)$; isentropic HSE with the closed-form density profile; face-centred HSE reconstruction as a necessary condition for well-balancing; ghost-cell boundary conditions (periodic-x, reflective-y, outflow-y).
- **Part C** (C1–C4) — Gravity source-term work consistency; strong- form CFL bound; LM-HLLC pressure-jump blending and its $M \to 0$ dispersion analysis; smooth-flow entropy invariant.
- **Part D** (D1–D7) — Canonical initial conditions with analytic golden values: entropy wave; acoustic linwave; Sod shock tube; Woodward-Colella two-shock blast; bubble IC; HSE zero-perturbation lock; reflection-symmetric IC.

- **Part E** (E1–E5) — Post-hoc benchmark derivations: entropy-wave convergence order; acoustic-wave LM-HLLC order pathology; LM-HLLC effective viscosity; Strang split source-term commutator; long-time HSE drift bound.

1.2 Reproducibility protocol

Every algebraic identity in this document is **mechanically verified by sympy**. Each section corresponds to a script `docs/derivations/strang/scripts/<section>.py` that ends with `assert_zero(LHS - RHS, label)` calls. If sympy cannot symbolically simplify an expression that is physically correct, the script falls back to numerical random sampling at N ≥ 50 admissible points with absolute tolerance ≤ 10^{-10}, and the markdown section flags the fallback explicitly.

1.3 Strong-form rule

All derivations default to **strong-form**, pointwise identities of the shape $A(x,t) \equiv B(x,t)$ rather than weak-form identities against a test-function family. Weak-form fallback is allowed only for three situations: (1) non-linear operator identities with no closed-form expansion (e.g., full-non-linear BCH), (2) benchmarks with no closed-form solution (e.g., the Woodward-Colella blast after wave interactions), and (3) the finite-volume cell average itself, which is inherently integrated. Every weak-form step is labelled `[WEAK]` in the markdown, carries a sympy numerical-consistency check, and includes a plain-English justification.

1.4 Script independence

Scripts are intentionally **independent**. Each one imports only `_common.py` and re-derives everything it needs from first principles. No cross-script caching of intermediate symbolic results. This makes any section independently re-runnable — a prerequisite for trusting the manuscript as a source of truth for the solver implementation.

1.5 Conventions

- Units are arbitrary but consistent; the Strang solver in the codebase runs with $\gamma = 5/3$ and $G = g$ read from the configuration. The derivation is G-agnostic and γ-generic.
- γ denotes the ratio of specific heats; $\gamma - 1$ is commonly factored as `gm1` in the kernel.
- The perturbation-form variables $(\delta\rho, m_x, m_y, \delta E)$ are the on-disk storage; primitive variables (ρ, u, v, P) are reconstructed from the stored fields plus the HSE background $(\bar{\rho}(y), \bar{p}(y))$ every time the solver needs them.
- sympy variable names mirror the mathematical symbols wherever readable (`rho` = ρ, `u` = u, `p` = p, `c_sound` = c); see `scripts/_common.py` for the full inventory.
- Golden values for Part-D ICs live in `output/d*_goldens.json`, which is **not** committed to the repository. `bash run_all.sh` is a build-time prerequisite of `ctest` and regenerates every JSON.

1.6 How to regenerate this manuscript

```
cd docs/derivations/strang
bash run_all.sh             # refreshes output/*.latex.tex and output/d*_goldens.json
bash build_manuscript.sh    # assembles sections/*.md -> manuscript.{md,pdf}
```

If a sympy assertion fails during `run_all.sh`, the build halts and the offending section is flagged. No partial manuscript is emitted.

2 A1. Compressible Euler equations (strong-form conservation)

sympy script: `scripts/a01_euler_equations.py` **generated LaTeX:** `output/a01_euler_equations.latex.tex` **verified:** - energy-flux factorisation x: $(E+p)u = \rho(h+\frac{1}{2}|\mathbf{v}|^2)u$ - energy-flux factorisation y: $(E+p)v = \rho(h+\frac{1}{2}|\mathbf{v}|^2)v$ - x-momentum material-derivative form: $\rho D_t u = -\partial_x p$ - y-momentum material-derivative form: $\rho D_t v = -\partial_y p$ - internal-energy material derivative: $\rho D_t e_{\text{int}} = -p\,\nabla\cdot\mathbf{v}$

code checkpoints: - `src/gpu/explicit/strang_device.cuh::d_euler_flux_x` - `src/gpu/explicit/strang_device.cuh::d_euler_flux_y` - `src/gpu/explicit/strang_device.cuh::d_cons2prim`

2.1 Starting assumptions

Label	Assumption
A1a	Compressible neutral gas; no viscosity, no thermal conduction, no radiation, no species diffusion.
A1b	Ideal EOS, $p = (\gamma-1)\,\rho\,e_{\text{int}}$, with γ constant.
A1c	Solutions are smooth in (x, y, t); identities in this section are pointwise strong-form equalities. Discontinuous solutions are admitted only via the entropy condition derived in §A5 and the Rankine-Hugoniot relations exploited by §A7.
A1d	Gravity, if present, enters as a source term in §C1 — it is **not** part of the flux **F** here.

The two-dimensional state vector used by the kernel is $\mathbf{U} = (\rho,\ \rho u,\ \rho v,\ E)^\top$, with total energy per unit volume $E = p/(\gamma-1) + \frac{1}{2}\rho(u^2+v^2)$.

2.2 Mass conservation

$$\partial_t \rho + \nabla\cdot(\rho\mathbf{v}) = 0. \quad \text{(A1-mass)}$$

This is taken as a postulate, not a derived identity. Every identity below is verified by sympy **while treating the mass residual as a free symbolic object** — the identity can then be closed under the strong-form assumption (A1-mass).

2.3 Momentum conservation (divergence form)

$$\partial_t(\rho u) + \partial_x(\rho u^2 + p) + \partial_y(\rho u v) = 0, \quad \text{(A1-mom-x)}$$

$$\partial_t(\rho v) + \partial_x(\rho u v) \ \ + \partial_y(\rho v^2 + p) = 0. \quad \text{(A1-mom-y)}$$

These two equations define rows 1 and 2 of the flux vector $\mathbf{F}_x, \mathbf{F}_y$ used by every Godunov kernel in the codebase. The Riemann-solver derivation in §A7 onwards operates on precisely these flux components.

2.4 Total-energy conservation

$$E \;=\; \frac{p}{\gamma-1} \;+\; \tfrac{1}{2}\rho\,(u^2+v^2). \quad \text{(A1-E)}$$

$$\partial_t E + \partial_x[(E+p)\,u] + \partial_y[(E+p)\,v] = 0. \quad \text{(A1-energy)}$$

The combination $(E+p)$ rather than E alone in the flux is the source of the *enthalpy* form that every Riemann solver exploits (§A7) and that the HLLC contact-wave algebra of §A8 relies on.

2.5 Enthalpy form of the energy flux

Define the specific enthalpy

$$h \;\equiv\; e_{\text{int}} + \frac{p}{\rho} = \frac{\gamma\,p}{(\gamma-1)\,\rho}.$$

Then $(E+p)\,v_i = \rho\,(h + \tfrac{1}{2}|\mathbf{v}|^2)\,v_i$ identically.

sympy verification (strong form). For both directions $i \in \{x, y\}$,

$$(E+p)\,v_i \;-\; \rho(h+\tfrac{1}{2}|\mathbf{v}|^2)v_i \;\xrightarrow{\text{sp.simplify}}\; 0$$

at the symbolic level, without any ODE integration.

2.6 Momentum in material-derivative form

Expanding the conservative momentum equation with $\rho u = \rho \cdot u$ and using (A1-mass), one obtains

$$\rho\,D_t u = -\partial_x p, \qquad \rho\,D_t v = -\partial_y p, \qquad D_t \equiv \partial_t + u\,\partial_x + v\,\partial_y. \quad \text{(A1-material-mom)}$$

sympy verification (strong form). Letting $R_{\text{mom},x}$ and R_{mass} denote the left-hand side residuals of (A1-mom-x) and (A1-mass),

$$R_{\text{mom},x} \;-\; u\,R_{\text{mass}} \;-\; (\rho\,D_t u + \partial_x p) \;=\; 0,$$

verified in sympy without invoking either residual equal to zero. This is important: (A1-material-mom) is not merely a consequence of (A1-mass) + (A1-mom-x); it is an *algebraic identity* between the two LHS residuals, and it remains valid even off-solution.

An analogous identity holds for the y-component.

2.7 Internal-energy equation (first law on a trajectory)

$$\rho\,D_t e_{\text{int}} = -\,p\,\nabla\!\cdot\!\mathbf{v}, \qquad e_{\text{int}} = \frac{p}{(\gamma-1)\,\rho}. \quad \text{(A1-material-e)}$$

sympy verification (strong form). Denote the energy residual R_E. The algebraic identity

$$R_E \;-\; u\,R_{\text{mom},x} \;-\; v\,R_{\text{mom},y} \;-\; (e_{\text{int}} - \tfrac{1}{2}|\mathbf{v}|^2)R_{\text{mass}} \;-\; (\rho\,D_t e_{\text{int}} + p\,\nabla\!\cdot\!\mathbf{v}) \;=\; 0$$

holds by `sp.simplify` alone. This is the reduction that underlies the Riemann-solver energy balance: the interior of every Godunov cell obeys (A1-material-e) as long as (A1-mass), (A1-mom-x), (A1-mom-y), and (A1-energy) are simultaneously satisfied.

The product $p\nabla\cdot\mathbf{v}$ is the reversible compression/expansion work of thermodynamics. In §A5 (entropy condition) and in §C4 (entropy invariant) this identity is invoked to show that $s=\log(p/\rho^\gamma)$ is a Lagrangian invariant on smooth flow.

2.8 Compact conservative system

$$\boxed{\partial_t\mathbf{U}+\partial_x\mathbf{F}_x(\mathbf{U})+\partial_y\mathbf{F}_y(\mathbf{U})=\mathbf{0},\qquad \mathbf{U}=(\rho,\ \rho u,\ \rho v,\ E)^\top.}$$

$$\mathbf{F}_x=\begin{pmatrix}\rho u\\ \rho u^2+p\\ \rho uv\\ (E+p)\,u\end{pmatrix},\qquad \mathbf{F}_y=\begin{pmatrix}\rho v\\ \rho uv\\ \rho v^2+p\\ (E+p)\,v\end{pmatrix}.$$

This is the strong-form system that every discretisation in the rest of this book approximates. The kernel-level counterparts live in `strang_device.cuh::d_euler_flux_x` and `d_euler_flux_y`; they must return these four components verbatim, with the $(E+p)$ factoring derived from §A1 above.

2.9 Verification checkpoints

The kernel is required to satisfy three strong-form invariants derivable from §A1, checked by one-cell test cases in `tests/test_strang_muscl.cu` and `tests/test_strang_hllc.cu`:

1. **Zero-velocity flux.** For any state with $u=v=0$, the only non-zero flux component is $F_x[1]=F_y[2]=p$. All other entries vanish identically. Regression signal: bitwise zero.
2. **Energy flux factorisation.** For any state with $\rho>0$ and $p>0$, the kernel's computed energy flux equals $\rho(h+\frac{1}{2}|\mathbf{v}|^2)\,v_i$ to ULP-precision.
3. **Material-derivative equivalence.** Apply the kernel's flux Jacobian to a smooth test state and compare the resulting semi-discrete D_tu against a directly computed $-\partial_xp/\rho$: agreement to $\mathcal{O}(\Delta x^2)$, i.e., MUSCL-consistent truncation error.

Failure of any of (1)–(3) flags inconsistency between the kernel implementation and the continuum equations of this section; such a failure must be resolved before anything in §A2 or later is relied upon.

3 A2. Conservative ↔ primitive bijection

> **sympy script:** `scripts/a02_conservative_primitive.py` **generated LaTeX:** `output/a02_conservative_primitive.latex.tex` **verified:** - round-trip $\mathbf{W}\to\mathbf{U}\to\mathbf{W}$ and $\mathbf{U}\to\mathbf{W}\to\mathbf{U}$ on all 4 components - both Jacobian determinants (forward and reverse) - 16 chain-rule entries of $\partial\mathbf{U}/\partial\mathbf{W}$ times $\partial\mathbf{W}/\partial\mathbf{U}$ - positivity envelope: $\rho>0, P>0$ in $\mathbf{W}$-space maps to admissible $\mathbf{U}$-space
>
> **code checkpoints:** - `src/gpu/explicit/strang_device.cuh :: d_cons2prim`

The strang solver stores $\mathbf{U}=(\rho,m_x,m_y,E)^\top$ on disk but needs the primitive form $\mathbf{W}=(\rho,u,v,p)^\top$ every time a Riemann solver, a reconstruction, or a physically-motivated floor is

applied. This section proves strong-form that the two forms are related by a smooth bijection on the admissible state space and derives the consequence for the kernel's numerical floor.

3.1 Forward map (primitive -> conservative)

$$\mathbf{U}(\mathbf{W}) \;=\; \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \dfrac{p}{\gamma-1} + \tfrac{1}{2}\rho(u^2+v^2) \end{pmatrix}. \quad \text{(A2-prim2cons)}$$

3.2 Inverse map (conservative -> primitive)

$$\mathbf{W}(\mathbf{U}) \;=\; \begin{pmatrix} \rho \\ m_x/\rho \\ m_y/\rho \\ (\gamma-1)\left(E - \dfrac{m_x^2+m_y^2}{2\rho}\right) \end{pmatrix}. \quad \text{(A2-cons2prim)}$$

3.3 Round-trip identities

Primitive -> conservative -> primitive. For every admissible **W** (i.e., $\rho > 0$, $p > 0$),

$$\mathbf{W}\big(\mathbf{U}(\mathbf{W})\big) \;\equiv\; \mathbf{W}.$$

sympy verifies this at the *symbolic* level on each of the four components.

Conservative -> primitive -> conservative. For every admissible **U** (i.e., $\rho > 0$, $E > (m_x^2+m_y^2)/(2\rho)$),

$$\mathbf{U}\big(\mathbf{W}(\mathbf{U})\big) \;\equiv\; \mathbf{U}.$$

Again each of the four components is verified to reduce identically to its input.

3.4 Jacobian determinants

Expanding the 4×4 Jacobian $\partial\mathbf{U}/\partial\mathbf{W}$ along the E-row (the only row with a $1/(\gamma-1)$ factor) and using block triangularity,

$$\det\!\left(\frac{\partial\mathbf{U}}{\partial\mathbf{W}}\right) \;=\; \frac{\rho^2}{\gamma-1}. \quad \text{(A2-det-forward)}$$

Correspondingly,

$$\det\!\left(\frac{\partial\mathbf{W}}{\partial\mathbf{U}}\right) \;=\; \frac{\gamma-1}{\rho^2}. \quad \text{(A2-det-inverse)}$$

Neither determinant vanishes in the admissible region $\rho > 0$, so the inverse function theorem applies locally everywhere in that region; the $\mathbf{U} \leftrightarrow \mathbf{W}$ map is a diffeomorphism on the admissible open set.

Chain-rule verification. Evaluated at the forward image $\mathbf{U}(\mathbf{W})$ of any primitive state,

$$\frac{\partial \mathbf{U}}{\partial \mathbf{W}} \cdot \left.\frac{\partial \mathbf{W}}{\partial \mathbf{U}}\right|_{\mathbf{U}(\mathbf{W})} = I_{4\times 4}. \quad \text{(A2-chain-identity)}$$

sympy verifies this entry-by-entry — 16 scalar identities, all reducing to the expected Kronecker delta.

3.5 Positivity envelope

The pressure is recovered from the conservative state by

$$p = (\gamma - 1)\left(E - \frac{m_x^2 + m_y^2}{2\rho}\right).$$

Admissibility ($p > 0$) is therefore equivalent to the strict kinetic-energy inequality

$$E > \frac{m_x^2 + m_y^2}{2\rho}. \quad \text{(A2-positivity)}$$

The kernel's `d_cons2prim` applies the clamp `P = fmax(P, 1e-30)` whenever round-off of a nearly-vacuum state has driven the right-hand side below zero. This clamp is strictly outside the admissible region of the continuum theory; its purpose is to prevent catastrophic cancellation from propagating into $\sqrt{\gamma p/\rho}$ (the sound-speed used by the Riemann solver). No solution in the admissible region triggers the clamp.

3.6 Verification checkpoints

Three invariants the kernel must satisfy, checked by `tests/test_strang_unit.cu`:

1. **Round-trip identity at ULP precision.** For any admissible primitive state generated randomly (100 samples with $\rho \in [0.1, 10]$, $p \in [0.1, 10]$, $|u|, |v| \in [0, 3c_0]$), `cons_to_prim(prim_to_cons(W))` must agree with W to within $10\,\varepsilon_{\text{mach}}\,\|W\|_\infty$.
2. **Pressure-floor activation.** Construct a near-vacuum state with E chosen so that the analytic pressure is -10^{-20} (below floor but above round-off). Verify that `d_cons2prim` returns $p = 10^{-30}$ rather than propagating a negative value. This is the **only** legitimate trigger path for the floor.
3. **Sound-speed positivity.** For any admissible state, the sound speed $c = \sqrt{\gamma p/\rho}$ returned by the kernel must be strictly positive. Failure indicates that the pressure floor was not applied or that a negative pressure was passed through without clamping — both are bugs the Riemann solver would silently turn into NaN.

Failures of (1) and (3) are correctness bugs; failure of (2) would be catastrophic on a stratified-HSE simulation's low-pressure atmosphere region.

4 A3. x-direction flux Jacobian eigensystem

sympy script: `scripts/a03_flux_jacobian_x.py` **generated LaTeX:** `output/a03_flux_jacobian_x.latex.tex` **verified:** - characteristic polynomial - $A_x R_k = \lambda_k R_k$ for all 4 eigenpairs × 4 components - left-right orthogonality $LR = I$ (16 entries) - characteristic decomposition $R\,\mathrm{diag}(\lambda)\,L = A_x$ (16 entries)

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (uses sound speed + wave speeds) - indirectly: every Riemann solver that invokes $S_L, S_R, S_\star$

The flux Jacobian $A_x \equiv \partial \mathbf{F}_x / \partial \mathbf{U}$ governs the linearised propagation of infinitesimal perturbations. Its eigensystem is the algebraic foundation for four downstream facts:

- The **sound speed** $c = \sqrt{\gamma p/\rho}$ appears as the magnitude-of-largest-eigenvalue; §C2 (CFL) bounds Δt in terms of it.
- The **Davis wave-speed estimates** (§A9) use $u \pm c$ directly from $\{\lambda_k\}$.
- The **HLLC star region** (§A8) is parameterised by $p^\star, u^\star$ whose derivation invokes Rankine-Hugoniot jumps across $\lambda_0 = u - c$, $\lambda_{1,2} = u$, and $\lambda_3 = u + c$.
- The **acoustic linwave golden value** (§D2) is the right eigenvector R_3 scaled by amplitude A.

4.1 Matrix form

Writing the primitive components in terms of the conservative ones, $u = m_x/\rho$, $v = m_y/\rho$, and $p = (\gamma - 1)(E - (m_x^2 + m_y^2)/(2\rho))$, the 4×4 matrix $A_x(\mathbf{U})$ is the strong-form Jacobian

$$A_x \;=\; \frac{\partial \mathbf{F}_x}{\partial \mathbf{U}}. \quad \text{(A3-Ax-def)}$$

Its explicit entries are lengthy; sympy computes them automatically and uses them throughout the section. The key results are the eigensystem, not the matrix entries themselves.

4.2 Characteristic polynomial and eigenvalues

$$\det(A_x - \lambda I) \;=\; (u-\lambda)^2\big[(u-\lambda)^2 - c^2\big], \qquad c \;\equiv\; \sqrt{\gamma p/\rho}. \quad \text{(A3-charpoly)}$$

Factoring gives the four eigenvalues

$$\{\lambda_k\}_{k=0}^{3} \;=\; \{\, u - c,\; u,\; u,\; u + c \,\}. \quad \text{(A3-eigvals)}$$

Physical interpretation. The acoustic modes $u \mp c$ are the two genuinely-nonlinear characteristic families; simple-wave solutions in these families admit rarefactions or shocks (§A6). The doubly-degenerate eigenvalue u is linearly degenerate; its eigenspace is two-dimensional and hosts the **entropy wave** (density contrast at constant velocity and pressure) and the **shear wave** (tangential-velocity contrast at constant density and pressure).

4.3 Right eigenvectors

In conservative form, with specific total enthalpy

$$h \;=\; \frac{\gamma\, p}{(\gamma - 1)\,\rho} + \tfrac{1}{2}(u^2 + v^2), \quad \text{(A3-enthalpy)}$$

the right eigenvectors (columns of R) are

$$R_0 = \begin{pmatrix} 1 \\ u-c \\ v \\ h-u\,c \end{pmatrix}, \qquad R_1 = \begin{pmatrix} 1 \\ u \\ v \\ \frac{1}{2}(u^2+v^2) \end{pmatrix}, \qquad R_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ v \end{pmatrix}, \qquad R_3 = \begin{pmatrix} 1 \\ u+c \\ v \\ h+u\,c \end{pmatrix}. \quad \text{(A3-right-eigvecs)}$$

Degeneracy basis choice. R_1 and R_2 together span the two-dimensional eigenspace of $\lambda = u$. The choice above is the conventional one: R_1 (entropy) has unit density component and zero tangential-velocity component; R_2 (shear) has zero density component and unit tangential-velocity component. Any invertible linear combination of these two vectors is also a valid basis, but the chosen one matches the decomposition used by Toro §3.1.2 and by every Godunov-family reference in the literature.

Strong-form verification. For each $k \in \{0, 1, 2, 3\}$,

$$A_x R_k \;-\; \lambda_k R_k \quad \overset{\text{sp.simplify}}{\longrightarrow} \quad \mathbf{0}, \qquad (\text{after the substitution } \; c^2 \to \gamma p/\rho).$$

This is 16 scalar identities (4 eigenpairs × 4 components), all verified.

4.4 Left eigenvectors and orthogonality

$L = R^{-1}$ is computed symbolically by sympy. The normalisation is such that

$$LR \;=\; I_{4\times 4}, \qquad \text{(A3-orthogonality)}$$

verified entry-by-entry (16 scalar identities).

The k-th row L_k of L is the left eigenvector associated with λ_k. In the projection interpretation,

$$\Delta\mathbf{U} \;=\; \sum_{k=0}^{3} (L_k \cdot \Delta\mathbf{U})\, R_k,$$

so $L_k \cdot \Delta\mathbf{U}$ is the **characteristic amplitude** in the k-th wave family. This is the identity that §A7 (Riemann solver comparison) and §D2 (acoustic linwave) invoke when constructing ICs that excite exactly one wave family.

4.5 Characteristic decomposition

$$A_x \;=\; R\,\mathrm{diag}(u-c,\; u,\; u,\; u+c)\,L, \qquad L = R^{-1}. \quad \text{(A3-diagonalisation)}$$

sympy verifies this entry-by-entry (16 scalar identities), after substituting $c = \sqrt{\gamma p/\rho}$ and simplifying the resulting radical expressions. The diagonalisation confirms A_x is strictly hyperbolic on the admissible region (real eigenvalues, complete eigenvector basis).

4.6 Verification checkpoints

The kernel does not explicitly assemble A_x — it uses the Riemann solver directly. Consequently the §A3 invariants are verified indirectly through downstream sections:

1. **Acoustic linwave convergence.** An IC built from R_3 (right-going acoustic) with amplitude $A = 10^{-6}$ must evolve for one wavelength λ without any unphysical mode coupling. Test: `test_strang_linwave_convergence.cu` with `use_lm_fix = false` (§E2); ratios of $\{\delta\rho, \delta u, \delta p\}$ at each time must match the eigenvector components of R_3 to $O(\Delta x^2)$.
2. **Entropy wave non-mixing.** An IC built from R_1 (entropy only: non-zero $\delta\rho$, zero $\delta u, \delta v, \delta p$) must propagate without exciting any acoustic content. Test: `test_strang_convergence.cu` on an entropy IC in the co-moving frame — HLLC degenerates to pure upwind (§D1), and acoustic modes remain exactly zero to ULP.
3. **Sound speed positivity.** The kernel's $c = \sqrt{\gamma p/\rho}$ must be strictly positive; negative c indicates that §A2's positivity-envelope clamp failed upstream. Tested inside `test_strang_hllc.cu` on a Sod-tube IC.

Failures of (1) or (2) indicate an eigenvector bug (either in the Riemann solver's star-region algebra of §A8, or in the slope limiter of §A10 producing spurious characteristic mixing). Failure of (3) is a positivity-preservation bug, caught earlier in §A2.

5 A4. Rotational covariance of the y-direction flux

sympy script: `scripts/a04_rotational_covariance.py` **generated LaTeX:** `output/a04_rotational_covariance.latex.tex` **verified:** - involution $R^2 = I$ (16 entries), covariance $\mathbf{F}_y = R\,\mathbf{F}_x(R\,\mathbf{U})$ (4 components), Jacobian covariance $A_y = R\,A_x(R\,\mathbf{U})\,R$ (16 entries), y-direction characteristic polynomial

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_hllc_update_y` (argument permutation) - `src/gpu/explicit/strang_device.cuh :: d_euler_flux_y`

The 2D Euler flux is invariant under relabelling of the two coordinate axes. This means the y-sweep Riemann solver does not need to be written as a separate device function — the x-sweep solver, invoked with its momentum arguments permuted, produces the same numerical flux. The strang kernel exploits this in `k_hllc_update_y`, which passes the (m_x, m_y) components of its left/right states in swapped order to the shared `d_lmhllc` routine.

5.1 The permutation matrix

$$R = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad R^2 = I. \quad \text{(A4-R-def)}$$

R swaps the second and third components of the conservative state vector $\mathbf{U} = (\rho, m_x, m_y, E)^\top$, mapping $(\rho, m_x, m_y, E) \mapsto (\rho, m_y, m_x, E)$. Density and total energy are unaffected. Because this is a permutation, R is its own inverse; sympy verifies all 16 entries of $R^2 = I$ individually.

5.2 Flux covariance

$$\mathbf{F}_y(\mathbf{U}) = R\,\mathbf{F}_x(R\,\mathbf{U}). \quad \text{(A4-flux-covariance)}$$

Strong-form verification. Expressing $\mathbf{U}$ in primitive form via $\mathbf{U}(\rho, u, v, p)$ (§A1), computing both sides and simplifying with sympy yields zero on all four conservative- flux components.

Interpretation at the kernel. The component order in the argument list of `d_lmhllc` is (ρ, u_n, u_t, P), where u_n is the velocity along the face normal and u_t the velocity parallel to the face. For an x-face this is $(u_n, u_t) = (u, v)$; for a y-face it is $(u_n, u_t) = (v, u)$. The A4 identity says that the same `d_lmhllc` device function produces the correct numerical flux in both cases — what changes is only the argument binding at the call site. This is why `k_hllc_update_y` writes

```
d_lmhllc(d_wR[k0*4+0], d_wR[k0*4+2], d_wR[k0*4+1], d_wR[k0*4+3],
         d_wL[kT*4+0], d_wL[kT*4+2], d_wL[kT*4+1], d_wL[kT*4+3],
         ... );
```

with indices 1 and 2 swapped at both the left (`wR[k0]`) and the right (`wL[kT]`) state. No new Riemann solver code is generated; A4 guarantees correctness of this permutation.

5.3 Jacobian covariance

By the chain rule (and $\partial(R\mathbf{U})/\partial\mathbf{U} = R$),

$$A_y(\mathbf{U}) \;=\; \frac{\partial \mathbf{F}_y(\mathbf{U})}{\partial \mathbf{U}} \;=\; \frac{\partial}{\partial \mathbf{U}}\Big(R\,\mathbf{F}_x(R\,\mathbf{U})\Big) \;=\; R\,\frac{\partial \mathbf{F}_x}{\partial \mathbf{U}}\bigg|_{R\mathbf{U}} \cdot R.$$

That is,

$$A_y(\mathbf{U}) \;=\; R\,A_x\big(R\,\mathbf{U}\big)\,R. \quad \text{(A4-Jacobian-covariance)}$$

Strong-form verification. sympy computes the left-hand side by direct Jacobian construction from the primitive-form substitution of $\mathbf{F}_y$ into A_y; the right-hand side by the sequence swap $(m_x, m_y) \to (m_y, m_x)$ inside A_x, followed by two R multiplications. The 16 entries of the difference all vanish under `sp.simplify`.

5.4 Eigenvalues of A_y

Substituting the primitive form into the expression for A_y and taking its characteristic polynomial,

$$\det\big(A_y - \lambda I\big) \;=\; (v-\lambda)^2\big[(v-\lambda)^2 - c^2\big], \qquad c = \sqrt{\gamma p/\rho},$$

giving eigenvalues

$$\{\lambda_k^{(y)}\}_{k=0}^{3} \;=\; \{\,v-c,\; v,\; v,\; v+c\,\}. \quad \text{(A4-eigvals-y)}$$

This mirrors §A3 with $u \leftrightarrow v$. Rotational covariance means every y-direction eigensystem result in the rest of the book is obtained from the x-direction result of §A3 by the single substitution $u \leftrightarrow v$.

5.5 Verification checkpoints

The kernel expresses A4 as a permutation of arguments at the call site, not as an explicit matrix multiplication. Consequently the correctness of the A4 identity is verified through a kernel-level equivalence test: for any admissible state $\mathbf{U}$ and a pair of neighbouring states $(\mathbf{U}_L, \mathbf{U}_R)$ that differ only in their tangential component,

1. **y-sweep swap-invariance.** Call `d_lmhllc` with $(u, v) = (u_0, v_0)$ and then again with the permuted argument binding representing $(u, v) = (v_0, u_0)$. The returned flux components, after the reciprocal permutation on the output side, must be bitwise identical. This is the operational form of $\mathbf{F}_y(\mathbf{U}) = R\mathbf{F}_x(R\mathbf{U})$ at the kernel level.
2. **Isotropic smoke IC.** Initialise a 2D Euler solution rotated by 90°; evolve for N steps under both x-sweep and y-sweep alone and verify that the evolved states are related by R to machine precision.

Test location: `tests/test_strang_hllc.cu`, §A4 smoke block.

Failure of (1) indicates an index-mismatch bug in the y-sweep kernel call (the kind of bug that an off-by-one in `d_wL[k0*4+1]` versus `d_wL[k0*4+2]` would introduce). Failure of (2) indicates a deeper asymmetry in the solver — most likely in the ghost-cell BC (§B5), not in the Riemann solver itself, but A4 is the identification the test relies on.

6 A5. Entropy condition (thermodynamic and mathematical)

sympy script: `scripts/a05_entropy_condition.py` **generated LaTeX:** `output/a05_entropy_condition.latex.tex` **verified:** - $D_t s_{\text{alg}} = 0$, $D_t s_{\text{therm}} = 0$, $\partial_t \eta + \partial_i (u_i \eta) = 0$ - 1 numerical-consistency check (Hessian PSD on 80 random admissible states) - 1 documented **[WEAK]** inequality (Lax entropy condition across a shock)

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: write_vtk` (emits s_{alg} as diagnostic) - §C4 (kernel-level entropy invariance on smooth tests)

The Euler system is not uniquely solvable in the presence of discontinuities — a single initial value problem can admit multiple weak solutions. The **entropy condition** singles out the unique physically admissible solution. This section derives the smooth- flow version in strong form and documents the discontinuous version as a labelled weak form.

6.1 Algebraic entropy invariant

Define

$$s_{\text{alg}} \equiv \frac{p}{\rho^\gamma}. \quad \text{(A5-s-alg)}$$

This is the entropy **up to a monotone transformation**. The thermodynamic specific entropy is

$$s_{\text{therm}} = \frac{1}{\gamma - 1} \ln\!\left(\frac{p}{\rho^\gamma}\right) + \text{const}, \quad \text{(A5-s-therm)}$$

a monotone function of s_{alg}. The kernel emits s_{alg} as the VTK diagnostic rather than s_{therm} to avoid the ln near $\rho \to 0$ boundary.

6.2 Smooth-flow invariance — strong form

On smooth flow (no shocks), the entropy is a Lagrangian invariant:

$$D_t s_{\text{alg}} \;=\; 0, \qquad D_t \;\equiv\; \partial_t + u\,\partial_x + v\,\partial_y. \quad \text{(A5-Dt-s)}$$

Strong-form derivation. Starting from mass conservation (§A1-mass) and the internal-energy material-derivative equation (§A1-material-e), one can derive the auxiliary pressure rule

$$D_t p \;=\; -\gamma\, p\, \nabla\cdot\mathbf{v} \quad \text{(A5-Dt-p)}$$

which combined with $D_t\rho = -\rho\,\nabla\cdot\mathbf{v}$ (from mass) gives

$$D_t\!\left(\frac{p}{\rho^\gamma}\right) \;=\; \frac{D_t p}{\rho^\gamma} - \gamma\,\frac{p}{\rho^{\gamma+1}}\,D_t\rho \;=\; \frac{-\gamma p\,\nabla\cdot\mathbf{v} + \gamma\, p\,\nabla\cdot\mathbf{v}}{\rho^\gamma} \;=\; 0.$$

sympy verification. The script computes $D_t s_{\text{alg}}$ symbolically and substitutes the two smooth-flow rules (A5-Dt-p) and the mass rule $D_t\rho \;=\; -\rho\,\nabla\cdot\mathbf{v}$; the result reduces to 0 under `sp.simplify`. The thermodynamic form s_{therm} inherits invariance by chain rule.

6.3 Mathematical entropy (Harten 1983)

Define the convex scalar entropy

$$\eta(\mathbf{U}) \;=\; -\rho\,\ln\!\left(\frac{p}{\rho^\gamma}\right), \qquad q_i \;=\; u_i\,\eta. \quad \text{(A5-eta)}$$

On smooth flow,

$$\partial_t\eta + \partial_x q_x + \partial_y q_y \;=\; 0. \quad \text{(A5-eta-conservation)}$$

Strong-form verification. sympy substitutes the mass and pressure-smooth-flow rules into $\partial_t\eta + \nabla\cdot(\mathbf{v}\eta)$; the result is 0 under `sp.simplify`.

Convexity of $\eta(\mathbf{U})$ — [WEAK] numerical check. The Hessian $H_{ij} = \partial^2\eta/\partial U_i \partial U_j$ on the admissible region $\{\rho > 0,\ p > 0\}$ is positive definite. sympy does not admit a symbolic proof of positive- definiteness (the analytical eigenvalues of a 4×4 symbolic matrix with logarithmic entries are intractable), so this is the first Rule-4 numerical fallback of the book:

> *Numerical consistency: 80 random admissible states sampled with $\rho \in [0.1, 10]$, $|u|, |v| \in [0, 2]$, $p \in [0.1, 10]$, $\gamma \in \{1.4, 5/3, 2\}$. At each state, H is evaluated and its 4 eigenvalues are computed via `numpy.linalg.eigvalsh`. All eigenvalues are verified positive (within 10^{-10} of zero). The ensemble passes uniformly.*

Convexity is a cited result (Harten 1983 §3); the numerical check here confirms the claim holds on the stellar2d admissible region for the γ values used in the code.

6.4 Lax entropy condition across a shock — [WEAK]

For a k-shock with speed σ separating left and right states, the **Lax entropy condition** is

$$\sigma\,[\eta]_{\mathrm{L}}^{\mathrm{R}} \;-\; [q_n]_{\mathrm{L}}^{\mathrm{R}} \;\geq\; 0, \qquad \text{(A5-lax-inequality, [WEAK])}$$

where $[f]_L^R = f(\mathbf{U}_R) - f(\mathbf{U}_L)$ and q_n is the entropy flux along the shock normal.

Why this is weak form. At a shock, η and q_n are distributional — $\partial_t \eta$ contains a delta function supported on the shock trajectory. The inequality above is obtained by integrating the strong-form equation against a smooth non- negative test function $\varphi(x,t)$ over a neighbourhood of the shock, invoking the Rankine-Hugoniot jump conditions on the flux, and exploiting $\varphi \geq 0$ to flip the equality to an inequality. This cannot be written as a pointwise algebraic identity and is outside the sympy strong-form protocol.

Rule-4 justification. Reason: shocks are distributional; $D_t\eta$ at the shock does not admit a strong-form pointwise expression. Numerical consistency check for a specific shock-tube IC is performed in §D3 (Sod) and §D4 (Woodward-Colella) where the jump $[\eta]$ and $[q_n]$ across the solver's reconstructed shock are computed and the inequality verified numerically on the converged solution.

6.5 Verification checkpoints

The kernel does not enforce §A5 explicitly; the Riemann solver (§A7) embeds the entropy condition through its choice of wave speeds (§A9, Davis bounds) and its contact-speed formula (§A8). §A5's invariants are verified at the discrete level by:

1. **Smooth-flow entropy drift.** On a periodic, shock-free IC (e.g., D1 entropy wave or D2 acoustic linwave at $A = 10^{-6}$), the solver's $\max_t |s_{\text{alg}}(x,t) - s_{\text{alg}}(x,0)|$ measured at every cell must remain below the modified-equation bound $O(\Delta x^2 + \Delta t^2)$. Test: to be added as `tests/test_strang_entropy_invariance.cu` during the Part-E wire-up.
2. **Shock entropy production.** For Sod's tube (§D3), the total entropy $\int \eta\, dV$ must be a non-increasing function of time. (The flux q_n vanishes on periodic boundaries or cancels on reflective boundaries.) Test: `tests/test_strang_sod.cu` §E5 entropy block.
3. **Harten convexity round-trip.** Starting from a smooth state, perturbed by $\varepsilon\xi$ with ξ random and small, the change in η must be non-negative to $O(\varepsilon^2)$ for every random ξ — the discrete analogue of the Hessian positivity. Test: added to `test_strang_unit.cu` as a property- based check.

Failures on (1) indicate either a bug in the Riemann solver (entropy fix missing in a transonic rarefaction) or a problem in the slope limiter (over-damping triggering excess dissipation — still entropy-conserving but characteristically wrong). Failures on (2) indicate a positivity bug or a wave-speed estimate that doesn't bound the shock, either of which is a deep Riemann-solver problem.

7 A6. Simple-wave families (rarefactions, contacts, shocks)

sympy script: `scripts/a06_smooth_wave_families.py` **generated LaTeX:** `output/a06_smooth_wave_families.latex.tex` **verified:** - 13 symbolic strong-form identities (Riemann invariants for all three wave families, genuine nonlinearity of 1- and 3-family, linear degeneracy of 2-family) - 4 numerical-fallback strong-form

checks (mass / momentum / energy Rankine-Hugoniot jumps, Prandtl mass-flux equality) at 80 random admissible shock states

code checkpoints: - §A8 (HLLC intermediate states use the 1-shock / 3-shock Hugoniot relations derived here) - §D3 (Sod shock-tube reference profile uses the rarefaction integration curves $J_k^{(i)} = \text{const}$)

The Godunov finite-volume kernel does not resolve individual waves; it resolves Riemann problems at every cell face. But every non-trivial piece of the Riemann solver — the HLLC contact speed (§A8), the Davis wave-speed bounds (§A9), the Sod analytic reference (§D3) — is built from the three wave families of this section. This section derives the characteristic structure in strong form, using both the genuine-nonlinearity framework (Lax 1957) and the explicit Rankine-Hugoniot locus (Toro 2009 §4.2).

The tangential velocity v plays no algebraic role in the x-sweep apart from the shear wave (§A6.3 below). We therefore present the derivation in the 1D projection (ρ, u, p); the shear wave is handled separately.

7.1 Right eigenvectors in primitive form

Using the primitive state $\mathbf{W} = (\rho, u, v, p)^\top$ and the sound speed $c = \sqrt{\gamma p/\rho}$,

$$R_1^{(\text{prim})} = \begin{pmatrix} 1 \\ -c/\rho \\ 0 \\ c^2 \end{pmatrix}, \qquad R_{2a}^{(\text{prim})} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \qquad R_{2b}^{(\text{prim})} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \qquad R_3^{(\text{prim})} = \begin{pmatrix} 1 \\ +c/\rho \\ 0 \\ c^2 \end{pmatrix}. \quad \text{(A6-R-prim)}$$

These are the primitive-space counterparts of the conservative eigenvectors of §A3; the mapping between them is $R_k^{(\text{cons})} = \partial\mathbf{U}/\partial\mathbf{W} \cdot R_k^{(\text{prim})}$. The two-fold degeneracy of the $\lambda = u$ eigenspace is split into an **entropy** direction R_{2a} (density contrast at constant u, v, p) and a **shear** direction R_{2b} (tangential- velocity contrast at constant ρ, u, p).

7.2 Riemann invariants

1-family ($\lambda = u - c$, acoustic left):

$$J_1^{(1)} \;=\; u + \frac{2c}{\gamma - 1}, \qquad J_1^{(2)} \;=\; \frac{p}{\rho^\gamma}. \quad \text{(A6-RI-1)}$$

3-family ($\lambda = u + c$, acoustic right):

$$J_3^{(1)} \;=\; u - \frac{2c}{\gamma - 1}, \qquad J_3^{(2)} \;=\; \frac{p}{\rho^\gamma}. \quad \text{(A6-RI-3)}$$

2-family ($\lambda = u$, entropy + shear):

- Entropy wave: u and p constant through; ρ may jump.
- Shear wave: ρ, u, p all constant; only tangential v jumps.

Strong-form verification. For each invariant $J_k^{(i)}$ and eigenvector R_k,

$$\nabla_{\mathbf{W}} J_k^{(i)} \;\cdot\; R_k \;\overset{\text{sp.simplify}}{\longrightarrow}\; 0.$$

9 scalar identities, all verified.

7.3 Genuine nonlinearity and linear degeneracy

The Lax classification requires computing $\nabla_{\mathbf{W}}\lambda_k \cdot R_k$ for each family:

$$\nabla_{\mathbf{W}}\lambda_1 \cdot R_1 \;=\; -\frac{(\gamma+1)\,c}{2\rho} \;\neq\; 0 \qquad \text{(1-family, genuinely nonlinear)}, \quad \text{(A6-gn-1)}$$

$$\nabla_{\mathbf{W}}\lambda_3 \cdot R_3 \;=\; +\frac{(\gamma+1)\,c}{2\rho} \;\neq\; 0 \qquad \text{(3-family, genuinely nonlinear)}, \quad \text{(A6-gn-3)}$$

$$\nabla_{\mathbf{W}}\lambda_2 \cdot R_{2a} \;=\; 0, \quad \nabla_{\mathbf{W}}\lambda_2 \cdot R_{2b} \;=\; 0 \qquad \text{(2-family, linearly degenerate)}. \quad \text{(A6-ld)}$$

Consequence for the Riemann problem. The acoustic 1- and 3-families admit genuine non-linear solutions: either rarefactions (smooth, self-similar fans) or shocks (jump discontinuities). The entropy/shear 2-family admits only contact discontinuities — a jump where the velocity and pressure are continuous. This is the structural fact §A7 and §A8 exploit when constructing the HLLC flux.

7.4 Rarefaction integration curves

Through a genuinely-nonlinear rarefaction fan, the invariants $J_k^{(1)}, J_k^{(2)}$ are simultaneously constant. Writing the fan parametrised by $\xi = (x - x_0)/t$ in the 1-family,

$$J_1^{(1)} = u + \frac{2c}{\gamma - 1} = \text{const}, \qquad J_1^{(2)} = \frac{p}{\rho^\gamma} = \text{const}, \qquad \xi = u - c.$$

Combining the three relations gives explicit expressions for $\rho(\xi), u(\xi), p(\xi)$ through the fan; these are the formulas §D3 (Sod) uses to sample the analytic rarefaction profile.

7.5 Rankine-Hugoniot jump conditions

For a 1D shock of speed σ separating pre-shock state (ρ_L, u_L, p_L) from post-shock state (ρ_R, u_R, p_R),

$$\sigma\,[\rho] \;=\; [\rho u], \qquad \sigma\,[\rho u] \;=\; [\rho u^2 + p], \qquad \sigma\,[E] \;=\; [(E+p)\,u], \quad \text{(A6-RH)}$$

with $[f] = f_R - f_L$ and $E = p/(\gamma-1) + \frac{1}{2}\rho u^2$.

Hugoniot locus for a 1-shock (Toro 2009 §4.2.1). Given $p^\star > p_L$ (compressive shock), the post-shock state is

$$\rho_L^\star \;=\; \rho_L\,\frac{p^\star/p_L + (\gamma-1)/(\gamma+1)}{(\gamma-1)/(\gamma+1)\,p^\star/p_L + 1}, \quad \text{(A6-hug-rho)}$$

$$u^\star \;=\; u_L - (p^\star - p_L)\sqrt{\frac{A}{p^\star + B}}, \qquad A = \frac{2}{(\gamma+1)\rho_L}, \quad B = \frac{\gamma-1}{\gamma+1}\,p_L, \quad \text{(A6-hug-u)}$$

$$\sigma \;=\; u_L - c_L\sqrt{\frac{\gamma+1}{2\gamma}\,\frac{p^\star}{p_L} + \frac{\gamma-1}{2\gamma}}. \quad \text{(A6-hug-sigma)}$$

Strong-form verification with numerical fallback. Substituting $\rho_L^\star, u^\star, \sigma$ back into the three RH jump equations yields expressions with nested square roots that exceed `sp.simplify`'s sqrt-denest capability.

> *Per Rule 1, when sp.simplify cannot reach 0 on a strong-form pointwise identity that is physically correct, the script falls back to numerical random sampling at* $N = 80$ *admissible shock states with* $\rho_L \in [0.1, 10]$*,* $|u_L| \in [0, 3]$*,* $p_L \in [0.1, 10]$*,* $p^\star/p_L \in [1.05, 20]$ *(compressive),* $\gamma \in \{1.4, 5/3, 2\}$*. Tolerance* 10^{-9}*. All four identities (mass jump, momentum jump, energy jump, Prandtl mass-flux equality* $\rho_L(u_L - \sigma) = \rho^\star(u^\star - \sigma)$*) pass. Note this is* ***not a weak-form step****: the identities are strong-form pointwise; the numerical fallback is a sympy-capability workaround, not a distributional relaxation.*

Achieved residual sizes (maxima across the 80-sample ensemble):

Identity	Max $\lvert$residual$\rvert$	Tolerance
Mass jump	3×10^{-14}	10^{-9}
Momentum jump	2×10^{-13}	10^{-9}
Energy jump	1×10^{-12}	10^{-9}
Prandtl flux	4×10^{-14}	10^{-9}

All four are several orders of magnitude below the tolerance.

7.6 Contact discontinuity

Across a 2-family discontinuity (contact wave), the strong-form conditions are

$$\sigma \;=\; u_L \;=\; u_R \;\equiv\; u^\star, \qquad p_L \;=\; p_R \;\equiv\; p^\star, \quad \text{(A6-contact)}$$

while ρ_L and ρ_R are unrelated (density contrast permitted) and v_L and v_R are unrelated (shear permitted). This is the identity §A8 uses to define the HLLC intermediate states: the two star-region cells share $u^\star$ and $p^\star$ but carry independent densities and tangential velocities.

7.7 Verification checkpoints

The kernel does not encode §A6 explicitly; the HLLC solver (§A8) uses the star-region fact $u_L^\star = u_R^\star$ and $p_L^\star = p_R^\star$ as its defining algebra. §A6's results are verified at the solver level through §D3 and §D4:

1. **Sod shock-tube reference.** §D3 dumps the analytic $\{\rho, u, p\}(x, t = 0.2)$ profile computed from the 1-family rarefaction fan + 2-family contact + 3-family shock Rankine-Hugoniot locus. Comparison to the simulation output gives the convergence slope expected in §E1.
2. **Tangential-velocity invariance.** Across the contact, §A6 permits shear but not pressure / normal-velocity jumps. Test `test_strang_hllc.cu` §A6-contact block: initialise a stationary contact with $\rho_L \neq \rho_R$, $v_L \neq v_R$, $u_L = u_R$, $p_L = p_R$; verify that the evolved u and p fields stay constant to ULP, while ρ and v are advected through the moving contact at speed u.

Failure on (1) indicates the Riemann solver's star-region algebra is wrong (§A8 bug). Failure on (2) indicates the HLLC treatment of the tangential velocity is incorrect — specifically, that the shear wave is being spuriously excited by acoustic-family mixing, which diagnoses a bug in the §A3 eigenvector basis or in the slope-limiter interaction with the shear component (§A10).

8 A7. Riemann solver family (Rusanov, HLLE, HLLC, Roe)

sympy script: `scripts/a07_riemann_solver_family.py` **generated LaTeX:** `output/a07_riemann_solver_family.latex.tex` **verified:** - consistency of Rusanov and HLLE on identity states (8 components) - mass-flux diffusion signature of Rusanov and HLLE on a stationary contact - HLLC $S_\star = 0$ at a stationary contact - $F_L = F_R$ at a stationary contact (used by Roe)

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (HLLC branch used by the kernel; Rusanov / HLLE / Roe derived here for comparison only)

This is the **first alternative-scheme comparison** section of the book (rule 4 of the README: book is the numerical reference, not just justification for the kernel's own choice). All four classical Godunov-family Riemann solvers are derived in the same template, then benchmarked against each other on the single most informative identity: the numerical flux at a stationary contact discontinuity.

8.1 Riemann solver template and consistency

Every Godunov-family Riemann solver is required by Harten–Lax–van Leer (1983) to satisfy the consistency condition

$$F_{\text{num}}(\mathbf{U}, \mathbf{U}) \;=\; \mathbf{F}_x(\mathbf{U}) \quad \text{(A7-consistency)}$$

for every admissible $\mathbf{U}$. This is the identity-state regression that rules out any solver whose flux disagrees with the continuum flux on a uniform state.

8.2 Rusanov (local Lax–Friedrichs)

The simplest scheme: blend $\mathbf{F}_L$ and $\mathbf{F}_R$ by an upper bound on the wave speed,

$$F^{\text{Rusanov}} \;=\; \tfrac{1}{2}(\mathbf{F}_L + \mathbf{F}_R) - \tfrac{1}{2}\alpha\,(\mathbf{U}_R - \mathbf{U}_L), \qquad \alpha = \max\big(|u_L| + c_L,\; |u_R| + c_R\big). \quad \text{(A7-Rusanov)}$$

Consistency. Substituting $\mathbf{U}_R = \mathbf{U}_L$ yields $F^{\text{Rusanov}} = \mathbf{F}_L$. All 4 components verified by sympy.

Strengths: single wave-speed estimate α, trivially positive-definite on any admissible state, robust under vacuum.

Weaknesses: maximally diffusive — every wave, including linearly-degenerate contact waves, is smeared by the full α bound.

8.3 HLLE (Harten–Lax–van Leer–Einfeldt, two-wave)

Replace α by separate lower/upper wave-speed estimates $S_L \le 0 \le S_R$ derived from the flux Jacobian eigenvalues:

$$F^{\mathrm{HLLE}} \;=\; \frac{S_R\,\mathbf{F}_L \;-\; S_L\,\mathbf{F}_R \;+\; S_L S_R\,(\mathbf{U}_R - \mathbf{U}_L)}{S_R - S_L}. \quad \text{(A7-HLLE)}$$

Consistency. At $\mathbf{U}_R = \mathbf{U}_L$ the jump term vanishes and $(S_R\mathbf{F}_L - S_L\mathbf{F}_L)/(S_R - S_L) = \mathbf{F}_L$. All 4 components verified by sympy.

Strengths: sharper than Rusanov (distinct S_L, S_R); still positive-definite (Einfeldt 1988).

Weaknesses: no internal structure between the two extreme waves, so the contact wave (linearly degenerate 2-family) is averaged into the single two-wave HLL state.

8.4 HLLC (Harten–Lax–van Leer–Contact, three-wave)

Resolve the contact wave explicitly by inserting a third wave of speed $S_\star$ between S_L and S_R. The full definition is

$$F^{\mathrm{HLLC}} \;=\; \begin{cases} \mathbf{F}_L & \text{if } 0 \le S_L, \\ \mathbf{F}_L + S_L(\mathbf{U}_L^\star - \mathbf{U}_L) & \text{if } S_L \le 0 \le S_\star, \\ \mathbf{F}_R + S_R(\mathbf{U}_R^\star - \mathbf{U}_R) & \text{if } S_\star \le 0 \le S_R, \\ \mathbf{F}_R & \text{if } S_R \le 0, \end{cases} \quad \text{(A7-HLLC)}$$

where $\mathbf{U}_L^\star, \mathbf{U}_R^\star$ are the star-region intermediate states and $S_\star$ is the contact speed. Their algebra is derived in §A8 in full strong form.

Strengths: contact resolution exact at stationary contacts (proved below); robust; positivity-preserving under Batten 1997 conditions.

Weaknesses: the low-Mach limit can create too much pressure dissipation — the kernel uses the Rieper (2011) LM-HLLC variant, derived in §C3.

8.5 Roe (flux-difference splitting)

Build a Roe-averaged Jacobian $A_{\mathrm{Roe}}(\mathbf{U}_L, \mathbf{U}_R)$ satisfying $A_{\mathrm{Roe}}(\mathbf{U}_R - \mathbf{U}_L) = \mathbf{F}_R - \mathbf{F}_L$ exactly (the Roe property); then

$$F^{\mathrm{Roe}} \;=\; \tfrac{1}{2}(\mathbf{F}_L + \mathbf{F}_R) - \tfrac{1}{2}\,|A_{\mathrm{Roe}}|\,(\mathbf{U}_R - \mathbf{U}_L). \quad \text{(A7-Roe)}$$

Strengths: exact on isolated single-wave Riemann problems (including contacts), sharpest contact resolution.

Weaknesses: can violate the Lax entropy condition at transonic rarefactions (sonic glitch); requires Harten entropy fix or Einfeldt wave-speed bound. The Roe matrix does not remain positive-definite across strong shocks.

8.6 Contact-wave scorecard (strong-form derivation)

Consider the stationary isolated contact discontinuity: $u_L = u_R = 0$, $p_L = p_R$, $\rho_L \neq \rho_R$, $v_L \neq v_R$. The exact Riemann solution has **no motion**; the exact flux is

$$\mathbf{F}^{\text{exact}} \;=\; (0,\; p_L,\; 0,\; 0)^\top,$$

since $\rho u = 0$, $\rho u^2 + p = p$, $\rho u v = 0$, and $(E + p)u = 0$ at $u = 0$.

Rusanov mass-flux diffusion (strong form). Substituting $u_L = u_R = 0$, $p_L = p_R$ into the Rusanov formula,

$$F_\rho^{\text{Rusanov}} \;-\; F_\rho^{\text{exact}} \;=\; -\tfrac{1}{2}\alpha\,(\rho_R - \rho_L), \qquad \alpha = \max(c_L, c_R). \quad \text{(A7-Rusanov-contact)}$$

Non-zero for $\rho_L \neq \rho_R$; the solver smears the density jump at speed $\alpha/2$.

HLLE mass-flux diffusion (strong form). Substituting the same IC,

$$F_\rho^{\text{HLLE}} \;-\; F_\rho^{\text{exact}} \;=\; \frac{S_L\,S_R\,(\rho_R - \rho_L)}{S_R - S_L}. \quad \text{(A7-HLLE-contact)}$$

Non-zero for $\rho_L \neq \rho_R$ (with $S_L < 0 < S_R$, so the denominator is positive); smaller than Rusanov but still diffusive.

HLLC exact resolution (strong form). The contact speed at the stationary contact is

$$S_\star \;=\; \frac{p_R - p_L + \rho_L u_L(S_L - u_L) - \rho_R u_R(S_R - u_R)}{\rho_L(S_L - u_L) - \rho_R(S_R - u_R)} \;=\; 0,$$

since both the numerator ($p_R - p_L = 0$, $u_L = u_R = 0$) and the sign structure give $S_\star = 0$ identically. The HLLC flux in the left-star branch ($S_L \le 0 \le S_\star$) has mass component $\rho_L^\star \cdot S_\star = \rho_L^\star \cdot 0 = 0$. Hence

$$F_\rho^{\text{HLLC}} \;-\; F_\rho^{\text{exact}} \;=\; 0. \quad \text{(A7-HLLC-contact)}$$

Exact resolution, no density smearing.

Roe exact resolution (strong form). At the stationary contact, $\mathbf{F}_L = \mathbf{F}_R = \mathbf{F}^{\text{exact}}$ (sympy verified on all 4 components). The Roe flux reduces to $\frac{1}{2}(\mathbf{F}_L + \mathbf{F}_R) - \frac{1}{2}|A_{\text{Roe}}|(\mathbf{U}_R - \mathbf{U}_L)$; on a pure-contact jump, $\mathbf{U}_R - \mathbf{U}_L$ lies exactly in the null space of A_{Roe} (this is the Roe property applied to the contact eigenvalue), so the dissipative second term vanishes. Hence

$$F_\rho^{\text{Roe}} \;-\; F_\rho^{\text{exact}} \;=\; 0. \quad \text{(A7-Roe-contact)}$$

Also exact resolution.

8.7 Summary scorecard

solver	identity state $F_{\text{num}}(U,U) = F_x(U)$	stationary contact mass flux diffusion	contact wave exact?
Rusanov	[ok]	$-\frac{1}{2}\alpha(\rho_R - \rho_L)$	no
HLLE	[ok]	$S_L S_R(\rho_R - \rho_L)/(S_R - S_L)$	no
HLLC	[ok] (§A8)	0	**yes**
Roe	[ok] (§A8 via Roe)	0	**yes**

The Strang kernel uses HLLC (via LM-HLLC in `d_lmhllc`) precisely because of the bottom-right cell of this table: without contact resolution, any simulation of stratified convection (which is dominated by density contrasts at near-zero velocity) would be crushed by numerical diffusion at every cell face. HLLE and Rusanov are derived here only for comparison.

8.8 Verification checkpoints

1. **Identity-state flux.** For any admissible state $\mathbf{U}$, `d_lmhllc(U, U)` must return $\mathbf{F}_x(\mathbf{U})$ to ULP precision. Test: `test_strang_hllc.cu` §A7-identity block.
2. **Stationary-contact mass flux.** Initialise $u_L = u_R = 0$, $p_L = p_R$, $\rho_L = 1$, $\rho_R = 5$ (canonical contact); `d_lmhllc` must return $F_\rho = 0$ to ULP. If the returned value is non-zero by more than $10\varepsilon_{\text{mach}}$, the solver has lost HLLC's contact-resolution property (likely through a bug in the star-state formula of §A8). Test: `test_strang_hllc.cu` §A7-contact block.

Failures of (1) indicate a kernel bug in the basic flux construction (fixed before §A8 is trusted). Failures of (2) indicate a bug in the $S_\star$ formula or in the star-state branch selection — a deeper algebraic issue that must be caught before any stratified-convection benchmark is run.

9 A8. HLLC intermediate states

sympy script: `scripts/a08_hllc_intermediate_states.py` **generated LaTeX:** `output/a08_hllc_intermediate_states.latex.tex` **verified:** - 17 strong-form pointwise identities — $p_L^\star = p_R^\star$ - mass / momentum-x / momentum-y / energy Rankine-Hugoniot across S_L and S_R (8 identities) - HLLC flux in the left-star and right-star branches reduces to $F(\mathbf{U}_L^\star)$ and $F(\mathbf{U}_R^\star)$ (8 components)

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (the $S_\star$ formula and $U_L^\star, U_R^\star$ constructions in the kernel are LM-scaled via f_M; the LM factor is derived separately in §C3)

The HLLC Riemann solver resolves three waves: the left and right acoustics at speeds S_L, S_R bounding the fan, and an interior contact at speed $S_\star$. Between the two acoustic waves, the flow is divided into two **star regions** with shared normal velocity $u^\star = S_\star$ and shared pressure $p^\star$, but distinct densities $\rho_L^\star, \rho_R^\star$ and tangential velocities $v_L^\star = v_L, v_R^\star = v_R$ (linearly-degenerate contact carries only density and tangential-velocity contrast, §A6).

This section derives all six star-region quantities — $S_\star, p^\star, \rho_L^\star, \rho_R^\star, \mathbf{U}_L^\star, \mathbf{U}_R^\star$ — in strong form, then verifies the Rankine- Hugoniot jump conditions on every wave (16 scalar identities, one per conservative component per wave side).

9.1 Contact speed $S_\star$

From the Rankine-Hugoniot mass-jump equations across S_L and S_R, plus the pressure-balance condition $p_L^\star = p_R^\star$ (which is imposed as the **definition** of the contact wave — §A6, linear degeneracy), algebraic elimination yields Toro's formula (2009 eq. 10.37):

$$S_\star = \frac{p_R - p_L + \rho_L u_L(S_L - u_L) - \rho_R u_R(S_R - u_R)}{\rho_L(S_L - u_L) - \rho_R(S_R - u_R)}. \quad \text{(A8-Sstar)}$$

Denominator note: $\rho_L(S_L - u_L) < 0$ and $\rho_R(S_R - u_R) > 0$ on admissible states (since $S_L < u_L$ and $S_R > u_R$); hence the denominator is strictly negative, never vanishing on an admissible state.

9.2 Star pressure $p^\star$

Two equivalent expressions (Toro 2009 eq. 10.38), one from each side of the contact:

$$p^\star \;=\; p_L + \rho_L(u_L - S_L)(u_L - S_\star) \;=\; p_R + \rho_R(u_R - S_R)(u_R - S_\star). \quad \text{(A8-pstar)}$$

Strong-form consistency. sympy verifies that substituting the $S_\star$ formula into both expressions yields the same value:

$$p_L^\star - p_R^\star \overset{\text{sp.simplify}}{\longrightarrow} 0.$$

9.3 Star densities

From the mass Rankine-Hugoniot across S_L and S_R (Toro 2009 eq. 10.36):

$$\rho_L^\star \;=\; \rho_L \,\frac{S_L - u_L}{S_L - S_\star}, \qquad \rho_R^\star \;=\; \rho_R \,\frac{S_R - u_R}{S_R - S_\star}. \quad \text{(A8-rhostar)}$$

Strong-form verification. sympy substitutes $\rho_L^\star$ and $\rho_R^\star$ into the mass jump condition $S_K[\rho] = [\rho u_n]$ across each wave and `sp.simplify`-reduces to 0.

9.4 Star momenta

Normal momentum in each star state is $\rho^\star$ times the common normal velocity $u^\star = S_\star$:

$$(\rho u)_K^\star \;=\; \rho_K^\star \, S_\star. \quad \text{(A8-momstar-n)}$$

Tangential momentum is unchanged across an acoustic wave (linear-degeneracy of the tangential component along 2-family):

$$(\rho v)_K^\star \;=\; \rho_K^\star \, v_K \qquad K \in \{L, R\}. \quad \text{(A8-momstar-t)}$$

Strong-form verification. Both momentum jump conditions verified on each acoustic wave (4 identities × 2 waves = 8 identities). The tangential-momentum identities use $v_K^\star = v_K$ as the unchanged-tangential-velocity condition.

9.5 Star energies

From the total-energy Rankine-Hugoniot (Toro 2009 eq. 10.39), after substituting the already-derived star densities and momenta:

$$E_K^\star \;=\; \rho_K^\star \left[\frac{E_K}{\rho_K} \;+\; (S_\star - u_K) \left(S_\star + \frac{p_K}{\rho_K \,(S_K - u_K)} \right) \right] \qquad K \in \{L, R\}. \quad \text{(A8-Estar)}$$

Strong-form verification. sympy verifies $S_K[E] = [(E + p)u_n]$ on each acoustic wave (2 identities, one per side).

9.6 HLLC numerical flux

The full piecewise definition, with branches selected by the sign of the local wave speeds:

$$F^{\mathrm{HLLC}} \;=\; \begin{cases} \mathbf{F}_L & 0 \le S_L, \\ \mathbf{F}_L + S_L(\mathbf{U}_L^\star - \mathbf{U}_L) & S_L \le 0 \le S_\star, \\ \mathbf{F}_R + S_R(\mathbf{U}_R^\star - \mathbf{U}_R) & S_\star \le 0 \le S_R, \\ \mathbf{F}_R & S_R \le 0. \end{cases} \qquad \text{(A8-HLLC)}$$

Strong-form consistency. In the left-star branch, the flux simplifies to

$$\mathbf{F}_L + S_L(\mathbf{U}_L^\star - \mathbf{U}_L) \;=\; \mathbf{F}(\mathbf{U}_L^\star) \;=\; \begin{pmatrix} \rho_L^\star S_\star \\ \rho_L^\star S_\star^2 + p^\star \\ \rho_L^\star S_\star v_L \\ (E_L^\star + p^\star) S_\star \end{pmatrix}.$$

sympy verifies all 4 components. Analogous result for the right- star branch. This means the HLLC flux is equivalently the **Euler flux evaluated at the star state** in each subsonic branch — a structurally stronger result than the general HLL template, and the reason HLLC exactly resolves isolated contacts (§A7).

9.7 Verification checkpoints

The §A8 identities are implemented inside `d_lmhllc`. The regression tests should check:

1. **S_□computation.** For random admissible L/R states with Davis-bounded S_L, S_R (§A9), the kernel's $S_\star$ must match the analytic formula to ULP precision. Test: `test_strang_hllc.cu` §A8-Sstar block.
2. **Pressure consistency.** Verify that $p^\star$ computed from the L-side formula and from the R-side formula agree to ULP. Any disagreement larger than $10\varepsilon_{\mathrm{mach}}$ indicates a bug in either $S_\star$ or in the star-pressure algebra. Test: `test_strang_hllc.cu` §A8-pstar block.
3. **Star-state reconstruction.** Given L, R, $S_L, S_R, S_\star, p^\star$, verify that the kernel's reconstructed $\mathbf{U}_L^\star$ satisfies $\rho_L^\star \cdot S_\star$ = normal-momentum component and $\rho_L^\star \cdot v_L$ = tangential-momentum component to ULP. Test: `test_strang_hllc.cu` §A8-U-star block.
4. **Contact-wave resolution.** With $u_L = u_R = 0$, $p_L = p_R$, $\rho_L = 1$, $\rho_R = 5$, `d_lmhllc` must return $(F_\rho, F_{\rho u}, F_{\rho v}, F_E) = (0, p_L, 0, 0)$ to ULP. Strengthens the A7 scorecard test: if the kernel returns non-zero mass flux, the bug is localised to the $S_\star$ formula or to the left-star branch selection. Test: `test_strang_hllc.cu` §A8-contact block.

Failures of (1) or (2) indicate algebraic bugs in the $S_\star/p^\star$ computation — the most common form is a sign error on the denominator (see Toro §10.5.1 for the canonical form). Failures of (3) would affect the energy balance and show up as energy-conservation drift in long-time simulations. Failure of (4) is the smoking-gun regression for HLLC's contact resolution and must be fixed before any convection or stratified benchmark can be run.

10 A9. Wave-speed estimates S_L, S_R

sympy script: `scripts/a09_wave_speed_estimates.py` **generated LaTeX:** `output/a09_wave_speed_estimates.latex.tex` **verified:** - 1 Davis-bracket identity (80 random admissible (L,R) pairs × 8 eigenvalue-bracket inequalities = 640 scalar

checks, all residuals ≤ 0) - 4 Roe-property identities (80 random admissible (L,R) pairs, max residual 6×10^{-14} vs tol 10^{-9})

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (Davis speeds $S_L = \min(u_L - c_L, u_R - c_R)$, $S_R = \max(u_L + c_L, u_R + c_R)$ hard-coded at the top of the solver)

The HLLC algebra of §A8 requires **bounding wave speeds** S_L, S_R that bracket every characteristic of the exact Riemann fan. This section derives the three canonical choices, verifies each bounds the A3 eigensystem correctly, and documents the Roe-average construction used by Einfeldt's tighter bounds and by the Roe solver of §A7.

10.1 Davis wave speeds (used by the kernel)

$$S_L \;=\; \min(u_L - c_L,\; u_R - c_R), \qquad S_R \;=\; \max(u_L + c_L,\; u_R + c_R). \quad \text{(A9-Davis)}$$

Strong-form bracketing (Min/Max lattice). By construction,

$$S_L \;\leq\; u_K - c_K \;\leq\; u_K \;\leq\; u_K + c_K \;\leq\; S_R \qquad \forall K \in \{L, R\}.$$

This says **every** eigenvalue of the A3 spectrum on either side is contained in $[S_L, S_R]$. In particular, the entropy/shear eigenvalues $\lambda_1 = \lambda_2 = u_K$ are trivially bracketed by the acoustic bounds, and the acoustic eigenvalues $\lambda_{0,3} = u_K \mp c_K$ are bracketed by construction.

Verification (numerical, Rule 1 fallback for Min/Max lattice). sympy's `Min/Max` simplifier does not automatically reduce expressions like $a - \min(a, b) = \max(0, a - b)$ when a and b contain `sqrt(...)`. We therefore verify the bracketing identities numerically at 80 random admissible (L, R) pairs × 8 eigenvalues (4 per side) = 640 scalar checks. All residuals are exactly zero (the Min/Max operations are hardware-level comparisons, not floating-point arithmetic). This is not a weak-form step; the identities are lattice-algebraic, and the numerical fallback is a sympy-capability workaround only.

10.2 Einfeldt tighter bounds

Using the Roe-averaged state $(\tilde{\rho}, \tilde{u}, \tilde{v}, \tilde{c})$ (defined below), Einfeldt (1988) proposed

$$S_L \;=\; \min\!\big(u_L - c_L,\; \tilde{u} - \tilde{c}\big), \qquad S_R \;=\; \max\!\big(u_R + c_R,\; \tilde{u} + \tilde{c}\big). \quad \text{(A9-Einfeldt)}$$

These are strictly tighter than Davis when the Roe-averaged state lies inside the fan. The tighter bounds reduce numerical dissipation in HLLE and give the stellar2d-relevant "Einfeldt positivity" property that HLLE preserves $\rho > 0, p > 0$ even across near- vacuum states.

Trade-off. Einfeldt's bounds are more accurate but require computing the Roe average. The Davis bounds are trivially cheaper (no square-roots beyond the sound speed) and still safely bracket every eigenvalue. The stellar2d kernel uses Davis for simplicity; the Einfeldt alternative is derived here for comparison.

10.3 Roe-averaged primitive state

Define the signed-weighted averages (Roe 1981, extended to 2D in Glaister 1988):

$$\tilde{\rho} \;=\; \sqrt{\rho_L\, \rho_R}, \qquad \tilde{u} \;=\; \frac{\sqrt{\rho_L}\, u_L + \sqrt{\rho_R}\, u_R}{\sqrt{\rho_L} + \sqrt{\rho_R}},$$

$$\tilde{v} \;=\; \frac{\sqrt{\rho_L}\,v_L + \sqrt{\rho_R}\,v_R}{\sqrt{\rho_L} + \sqrt{\rho_R}}, \qquad \tilde{h} \;=\; \frac{\sqrt{\rho_L}\,h_L + \sqrt{\rho_R}\,h_R}{\sqrt{\rho_L} + \sqrt{\rho_R}},$$

$$\tilde{c}^2 \;=\; (\gamma-1)\Big(\tilde{h} - \tfrac{1}{2}(\tilde{u}^2 + \tilde{v}^2)\Big). \quad \text{(A9-Roe-avg)}$$

The specific enthalpies are $h_K = \gamma p_K/((\gamma-1)\rho_K) + \frac{1}{2}(u_K^2 + v_K^2)$.

10.4 The Roe property

The defining property of the Roe-averaged state is that the flux Jacobian A_x evaluated at the Roe average exactly reproduces the flux jump:

$$A_{\text{Roe}}(\mathbf{U}_L, \mathbf{U}_R)\,(\mathbf{U}_R - \mathbf{U}_L) \;=\; \mathbf{F}_x(\mathbf{U}_R) - \mathbf{F}_x(\mathbf{U}_L). \quad \text{(A9-Roe-property)}$$

This is the **algebraic identity** that (a) makes the Roe solver exact on isolated single-wave Riemann problems, (b) gives the Einfeldt bounds their contact-resolution sharpness, and (c) underpins the Roe entropy-fix discussion at transonic rarefactions.

Strong-form verification via numerical fallback. The Roe matrix $A_x(\tilde{\mathbf{U}})$ has square-root-averaged entries that sympy cannot simplify to the closed-form flux jump $\mathbf{F}_R - \mathbf{F}_L$. We fall back to numerical random sampling at 80 admissible (L, R) pairs:

> *80 random* $(\rho_L, \rho_R, u_L, u_R, v_L, v_R, p_L, p_R, \gamma)$ *samples with* $\rho_K \in [0.1, 10]$, $|u_K|, |v_K| \in [0, 2]$, $p_K \in [0.1, 10]$, $\gamma \in \{1.4, 5/3, 2\}$. *Four conservative-component residuals of* $A_{\text{Roe}}(\mathbf{U}_R - \mathbf{U}_L) - (\mathbf{F}_R - \mathbf{F}_L)$ *checked with tolerance* 10^{-9}. *Achieved max* $|\textit{residual}| = 6 \times 10^{-14}$.

This is the same class of sympy-capability fallback as §A6's Rankine-Hugoniot identities — strong-form, not weak-form. The numerical check merely confirms what sympy cannot denest symbolically.

10.5 Entropy fix and transonic rarefaction

Lax-entropy admissibility across a shock was introduced in §A5 and flagged **[WEAK]** (distributional). The Roe solver violates this at a transonic rarefaction (where a genuinely-nonlinear acoustic eigenvalue changes sign inside the fan): A_{Roe} treats the fan as a single jump at the averaged speed, producing a non-physical "expansion shock" that carries the fan's density/velocity data across a stationary discontinuity.

The fix is to replace $|\lambda_k|$ inside the Roe flux with Harten's (1983) mollified absolute value $H_\epsilon(\lambda_k)$ whenever λ_k changes sign across the fan. For HLLC + Davis speeds this is not needed: the fan bounds are wide enough that the transonic case is still resolved correctly. This is a consequence of Davis's conservative over-estimate — one of the reasons the stellar2d kernel uses Davis rather than the tighter Einfeldt bounds.

10.6 Verification checkpoints

The kernel's Davis wave-speed computation is simple enough that no subtle bugs are likely; the checks are smoke tests:

1. **Davis brackets the spectrum.** For random admissible (L, R) pairs (100 samples), the kernel's S_L, S_R must satisfy $S_L \le u_K \pm c_K \le S_R$ for both $K \in \{L, R\}$. Test: `test_strang_hllc.cu` §A9-Davis-bracket block.

2. **Davis bounds are Min/Max exact.** For any (L, R) pair, $S_L = \min(u_L - c_L, u_R - c_R)$ to bitwise precision (no intermediate floating-point rounding). Test: one-line assertion using `std::min` and `std::max` comparing to the kernel's implementation.
3. **Einfeldt is tighter when applicable.** For a random state where the Roe-averaged state falls inside the fan, the Einfeldt bounds $[\tilde{u} - \tilde{c}, \tilde{u} + \tilde{c}]$ must lie strictly inside the Davis bounds. This is a property-based test; it verifies understanding of the algebraic hierarchy, not the kernel itself (Einfeldt is not used in the kernel). Test: `test_strang_hllc.cu` §A9-Einfeldt- tighter block (optional scheme-characterisation check).

Failures of (1) or (2) indicate the kernel's Davis implementation diverges from the §A9 definition — a bug in one of the `fmin(u_L - c_L, u_R - c_R)` arithmetic ops.

11 A10. Slope-limiter family (MC / minmod / van Leer / superbee / Ospre)

sympy script: `scripts/a10_slope_limiter_family.py` **generated LaTeX:** `output/a10_slope_limiter_family.latex.tex` **verified:** - second-order consistency $\phi(1) = 1$ for all 5 limiters - spot values at $r \in \{-1/2, 3/2, 2, 3, 4\}$ - zero-at-extremum property for minmod/VL/MC/superbee - explicit Ospre non-TVD demonstration at $r = -1/2$ - 6 numerical-fallback identities (symmetry $\phi(1/r) = \phi(r)/r$ for all 5 limiters + MC kernel/Sweby equivalence) at ≥ 100 random samples each, all residuals $\leq 10^{-15}$

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_mc_limit`

This is the **second alternative-scheme comparison** section of the book. The Strang kernel uses the MC limiter exclusively, but a derivation book that does not place MC in the wider family of Sweby-form TVD limiters cannot answer the practical question "would van Leer be enough?" or "is superbee too aggressive for stratified convection?". All five canonical limiters are derived here, their TVD regions compared, and the kernel's two-argument form `d_mc_limit(a, b)` is proven equivalent to its Sweby-form counterpart.

11.1 Sweby form and TVD constraints

Let $r = \Delta_L / \Delta_R$ be the ratio of the two consecutive slope estimates (left difference over right difference) at a cell centre. A Sweby-form limiter $\phi(r)$ selects the effective slope $\phi(r)\,\Delta_R$ subject to the total-variation-diminishing (TVD) constraints of Sweby (1984):

$$0 \;\leq\; \phi(r) \;\leq\; \min(2r, 2) \quad (r \geq 0), \qquad \phi(r) = 0 \quad (r < 0). \quad \text{(A10-Sweby-region)}$$

The zero-at-negative-r condition means **local extrema get flat reconstructions** (no overshoot). Second-order consistency on smooth flow requires

$$\phi(1) \;=\; 1. \quad \text{(A10-second-order)}$$

This is the one constraint every limiter below obeys; differences lie in how they fill the admissible TVD region.

11.2 Limiter family

limiter	Sweby form $\phi(r)$	behaviour
minmod	$\max(0,\ \min(1,\ r))$	most diffusive; TVD-strict
van Leer	$(r + \lvert r\rvert)/(1 + \lvert r\rvert)$	smooth C^1; TVD-strict
MC	$\max(0,\ \min(2r,\ (1+r)/2,\ 2))$	kernel's choice; TVD-strict
superbee	$\max(0,\ \min(2r, 1),\ \min(r, 2))$	most aggressive; TVD-strict
Ospre	$\frac{3}{2} \cdot (r^2 + r)/(r^2 + r + 1)$	rational; extends into $r < 0$ (**not strictly TVD**)

Ospre non-TVD note. Unlike the other four, Ospre is a rational function that does not enforce $\phi(r) = 0$ for $r < 0$. At $r = -1/2$, $\phi_{\text{Ospre}}(-1/2) = -1/2$, violating the Sweby condition $\phi \geq 0$. Ospre trades strict TVD for third- order smooth-extremum resolution; it is listed here for comparison but is not a drop-in replacement for MC in stellar2d.

11.3 Spot values and aggressiveness ranking

At $r = 3/2$ (moderate positive gradient, a typical value in smooth-but-non-trivial flow):

$$\phi_{\text{minmod}}(3/2) = 1 \ < \ \phi_{\text{Ospre}}(3/2) = 45/38 \approx 1.184 \ < \ \phi_{\text{VL}}(3/2) = 6/5 \ < \ \phi_{\text{MC}}(3/2) = 5/4 \ < \ \phi_{\text{superbee}}(3/$$

Each ϕ value is verified algebraically.

At $r = 2$ (steep gradient), the $(1 + r)/2$ constraint of MC binds: $\phi_{\text{MC}}(2) = 3/2$. MC's hard cap $\phi \leq 2$ only binds for $r \geq 3$; verified at $r = 3$ and $r = 4$ explicitly. At $r = 2$, superbee hits the upper TVD envelope ($\phi = 2$), while minmod stays at $\phi = 1$ (TVD maximal diffusion), and van Leer sits at $\phi = 4/3$ (moderate).

11.4 Zero-at-extremum

At any local extremum, the two neighbouring slopes have opposite sign; hence $r < 0$. All four non-Ospre limiters return $\phi = 0$ there, giving a flat reconstruction at the extremum — no overshoot, no oscillation. Ospre's explicit failure at $r = -1/2$ is noted above.

11.5 Symmetry property

For all five limiters (including Ospre),

$$\phi(1/r) \ = \ \phi(r)/r \qquad (r > 0). \quad \text{(A10-symmetry)}$$

This ensures the reconstruction is **direction-symmetric**: reversing the ordering of the two neighbours gives the same effective slope magnitude. Verified by numerical random sampling at 100 positive values of r with atol 10^{-12} (sympy's Min/Max simplifier cannot reduce this identity symbolically; this is a sympy-capability workaround, not a weak-form step).

11.6 Kernel's two-argument form is Sweby-form MC

The kernel stores two consecutive slope differences a, b and returns the limited slope directly (not the ratio):

$$\text{d\_mc\_limit}(a,b) \;=\; \begin{cases} \operatorname{sign}(a)\,\min\!\left(\frac{|a+b|}{2},\; 2|a|,\; 2|b|\right) & \operatorname{sign}(a)\operatorname{sign}(b) > 0, \\ 0 & \text{otherwise.} \end{cases} \qquad \text{(A10-kernel-form)}$$

Equivalence. For the same-sign branch ($a > 0$, $b > 0$ without loss of generality), the kernel's $\min\!\left(\frac{a+b}{2}, 2a, 2b\right)$ equals $\phi_{\text{MC}}(b/a) \cdot a$ at every admissible (a, b) — verified at 100 random positive pairs, max residual 10^{-15}.

The opposite-sign branch returns 0 directly, which matches the zero-at-extremum property of $\phi_{\text{MC}}(r)$ at $r < 0$.

11.7 Verification checkpoints

The kernel's `d_mc_limit` is a simple two-line function. Tests:

1. **Kernel identity at canonical points.** At $a = b$ (smooth, $r = 1$), `d_mc_limit(a, a) == a` exactly. At $a = 0$ or $b = 0$, `d_mc_limit == 0`. At opposite signs, `d_mc_limit(+a, -b) == 0`. These are the three corner cases that define the limiter's behaviour. Test: `test_strang_muscl.cu` §A10-corner-cases.
2. **Kernel-vs-Sweby equivalence.** For 100 random same-sign pairs (a, b), `d_mc_limit(a, b)` must equal $a \cdot \phi_{\text{MC}}(b/a)$ to ULP precision. Test: `test_strang_muscl.cu` §A10-Sweby-equivalence.
3. **TVD region.** For 100 random $r > 0$ via the kernel's two-argument form at some (a, b) with $r = b/a$, the returned $\phi =$ kernel output$/a$ must satisfy $0 \le \phi \le \min(2r, 2)$. Test: `test_strang_muscl.cu` §A10-TVD-region.

Failure of (1) indicates a sign-handling bug; failure of (2) indicates an arithmetic error in the min-of-three; failure of (3) would mean the kernel violates TVD, which would show up as spurious oscillations in a shock-tube test (§D3).

12 A11. Reconstruction order (donor-cell / MUSCL / PPM)

> **sympy script:** `scripts/a11_reconstruction_order.py` **generated LaTeX:** `output/a11_reconstruction_order.latex.tex` **verified:** - 10 strong-form identities via Taylor expansion of cell averages — donor-cell h^1 and h^2 coefficients - MUSCL h^0 and h^1 coefficients vanish (establishing 2nd order) - PPM h^0, h^1, h^2, h^3 coefficients all vanish (establishing 4th-order face reconstruction)
>
> **code checkpoints:** - `src/gpu/explicit/strang_solver.cu :: k_muscl_hancock_x/y` (MUSCL) (PPM is derived here for comparison; the kernel does not implement it.)

This is the **third alternative-scheme comparison** section. Four reconstruction orders are derived in strong form by Taylor expansion of cell averages around a cell centre, and their leading truncation errors compared. The Strang kernel uses MUSCL (2nd order); PPM is derived here so that the book can answer "what do we gain by upgrading to PPM?".

12.1 Setup: cell averages vs point values

For a smooth $u(x)$ on a uniform grid of spacing h, the cell average at cell j is

$$\langle u\rangle_j \;=\; \frac{1}{h}\int_{x_j-h/2}^{x_j+h/2} u(x')\,dx' \;=\; u(x_j) + \frac{h^2}{24}\,u''(x_j) + \frac{h^4}{1920}\,u''''(x_j) + O(h^6).$$

The distinction between $\langle u\rangle_j$ and $u(x_j)$ is $O(h^2)$ — negligible for 1st-order schemes but crucial for PPM (which claims 4th-order face reconstruction).

The **true face value** is the point value at the face centre,

$$u(x_{j+1/2}) \;=\; u(x_j+h/2) \;=\; u(x_j)+\tfrac{h}{2}\,u'(x_j)+\tfrac{h^2}{8}\,u''(x_j)+\tfrac{h^3}{48}\,u'''(x_j)+\tfrac{h^4}{384}\,u''''(x_j)+O(h^5).$$

Every Godunov scheme of this book operates on cell averages $\{\langle u\rangle_{j-1}, \langle u\rangle_j, ...\}$ and reconstructs face values against the **point-value** target above.

12.2 Donor-cell (1st order)

$$u^{\text{donor}}_{j+1/2,L} \;=\; \langle u\rangle_j. \quad \text{(A11-donor)}$$

Error expansion:

$$\varepsilon^{\text{donor}} \;=\; u^{\text{donor}}_{j+1/2,L} - u(x_{j+1/2}) \;=\; -\tfrac{h}{2}\,u'(x_j) - \tfrac{h^2}{12}\,u''(x_j) + O(h^3).$$

Strong-form verification. sympy confirms the h^1 and h^2 coefficients of the error match the expected $-\frac{1}{2}u'$ and $-\frac{1}{12}u''$ (note the h^2 constant is $\frac{1}{12}$, not $\frac{1}{8}$, because cell average and point value differ by $\frac{h^2}{24}u''$; this is the "cell-average correction" that PPM exploits).

12.3 Unlimited MUSCL (2nd order)

The unlimited central-difference slope

$$\sigma_j \;=\; \frac{\langle u\rangle_{j+1} - \langle u\rangle_{j-1}}{2h},$$

combined with linear reconstruction

$$u^{\text{MUSCL}}_{j+1/2,L} \;=\; \langle u\rangle_j + \tfrac{h}{2}\,\sigma_j. \quad \text{(A11-MUSCL)}$$

Error expansion:

$$\varepsilon^{\text{MUSCL}} \;=\; -\tfrac{h^2}{12}\,u''(x_j) + O(h^3).$$

Strong-form verification. The h^0 and h^1 error coefficients vanish identically (sympy-verified), confirming 2nd- order accuracy. The leading h^2 coefficient is $-\frac{1}{12}u''(x_j)$ (sympy reports this directly). The numerical dissipation of the full Godunov scheme is further reduced by cancellation between reconstruction and time integration (§E1 modified-equation analysis).

12.4 PPM Colella-Woodward (4th order face reconstruction)

$$u_{j+1/2}^{\text{PPM-CW}} \;=\; \tfrac{7}{12}\big(\langle u\rangle_j + \langle u\rangle_{j+1}\big) - \tfrac{1}{12}\big(\langle u\rangle_{j-1} + \langle u\rangle_{j+2}\big). \quad \text{(A11-PPM-CW)}$$

Error expansion (at x_j-frame): $\varepsilon^{\text{PPM-CW}} = -\frac{h^4}{30}\,u''''(x_j) + O(h^6)$.

Strong-form verification. sympy confirms that the h^0, h^1, h^2, h^3 coefficients all cancel identically — i.e., the PPM formula is 4th-order-accurate at the face. (Colella–Woodward 1984 eq. 1.7 quotes the leading error in the $x_{j+1/2}$-frame as $\frac{3h^4}{640}u''''(x_{j+1/2})$; the two constants differ by the Taylor-frame shift; both are $O(h^4)$, which is what matters.)

12.5 PPM Colella-Sekora variant

Colella & Sekora (2008) keep the same unlimited 4-cell reconstruction (A11-PPM-CW) but replace the Colella–Woodward parabolic-overshoot limiter with an extremum detector. At smooth extrema the original CW limiter degrades to 1st order; CS detects genuine smooth extrema and preserves 3rd-order accuracy there. The unlimited reconstruction algebra is identical, so the §A11 truncation analysis applies without modification. The limiter difference manifests only on non-smooth data (shocks, contacts).

12.6 Reconstruction-order hierarchy

reconstruction	leading error	scheme overall order	stencil width
donor-cell	$O(h)$	1st	1
MUSCL (unlimited)	$O(h^2)$	2nd	3
MUSCL (limited)	$O(h^2)$ away from extrema, $O(h)$ at extrema	2nd (smooth) / 1st (shocks)	3
PPM-CW	$O(h^4)$ reconstruction -> $O(h^3)$ full scheme	3rd	4
PPM-CS	$O(h^4)$ reconstruction -> $O(h^3)$ everywhere	3rd	4

Kernel's choice. The Strang solver uses limited MUSCL with the MC limiter. The decision matrix: for a 2D compressible simulation at $N^2 \sim 512^2$ with many limiter activations at grid-scale turbulence, MUSCL's 2nd-order error is comparable to PPM's 3rd-order error after accounting for limiter clips, while costing ~40% less per step. Upgrading to PPM is listed as an optional scheme-characterisation experiment in §E (not in the current kernel scope).

12.7 Verification checkpoints

The kernel implements §A11's MUSCL reconstruction inside `k_muscl_hancock_x/y`. Tests:

1. **Smooth-IC 2nd-order convergence.** On a smooth sinusoidal IC, the entropy-wave L^1 error must decrease as h^2 as N is refined from 64^2 to 512^2. Slope must fall in $[1.8, 2.2]$. Test: `test_strang_convergence.cu` §A11-MUSCL- order (already exists; will be extended to read goldens from D1 in Part D).
2. **Donor-cell fallback regression.** With MUSCL's limiter clamped to 0 (donor-cell behaviour), the same test must give slope in $[0.8, 1.2]$. This confirms the limiter, not the

base scheme, is responsible for 2nd-order accuracy. Test: `test_strang_convergence.cu` §A11-donor-cell-fallback (to be added).

Failure of (1) indicates either a bug in the Hancock predictor (§A12) or in the limiter (§A10). Failure of (2) would indicate a deep structural bug in the spatial reconstruction itself, visible only at 1st-order setting.

13 A12. MUSCL-Hancock half-step predictor

sympy script: `scripts/a12_muscl_hancock_halfstep.py` **generated LaTeX:** `output/a12_muscl_hancock_halfstep.latex.tex` **verified:** - Hancock linear-advection equivalence (u_{hancock} equals $u_0 + (\Delta t/2)\, u_t$ after substituting the PDE constraint $u_t = -a u_x$) - 2nd-order time-truncation identity ($u_{\text{true}} - u_{\text{hancock}} = (\Delta t^2/8)\, a^2 u_{xx} + O(\Delta t^3)$) - cell-average conservation of the half-step

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_muscl_hancock_x` - `src/gpu/explicit/strang_solver.cu :: k_muscl_hancock_y`

The Hancock predictor is the second pillar of MUSCL: after reconstructing face states (§A11), it evolves those states forward by a **half time step** using the physical flux, producing time-centred values at $t_n + \Delta t/2$ to feed the Riemann solver. This makes MUSCL globally 2nd-order accurate in both space and time while keeping the stencil local.

13.1 Predictor formulas

For a cell j with reconstructed face states $\mathbf{U}^n_{j+1/2,L}$ (at its right face, left of the Riemann problem) and $\mathbf{U}^n_{j-1/2,R}$ (at its left face, right of the Riemann problem),

$$\mathbf{U}_L^{n+1/2} \;=\; \mathbf{U}^n_{j+1/2,L} - \frac{\Delta t}{2h}\left[\mathbf{F}(\mathbf{U}^n_{j+1/2,L}) - \mathbf{F}(\mathbf{U}^n_{j-1/2,R})\right],$$

$$\mathbf{U}_R^{n+1/2} \;=\; \mathbf{U}^n_{j-1/2,R} - \frac{\Delta t}{2h}\left[\mathbf{F}(\mathbf{U}^n_{j+1/2,L}) - \mathbf{F}(\mathbf{U}^n_{j-1/2,R})\right]. \quad \text{(A12-Hancock)}$$

Both face states are updated by the **same** flux-difference term; this is what enforces cell-average conservation through the half-step.

13.2 Linear advection consistency (strong form)

For the scalar linear advection $u_t + a u_x = 0$ with constant a, treating face value u_0 and its derivatives as independent symbols and substituting the PDE constraint,

$$u_{\text{hancock}} \;=\; u_0 - \frac{\Delta t}{2}\, a\, u_x \;\xrightarrow{\;u_t = -a u_x\;}\; u_0 + \frac{\Delta t}{2}\, u_t. \quad \text{(A12-linear-advect)}$$

The right-hand side is the exact midpoint-time value to linear order in Δt: $u(x_{j+1/2}, t_n + \Delta t/2) \;=\; u_0 + (\Delta t/2)\, u_t(x_{j+1/2}, t_n) + O(\Delta t^2)$. Hancock thus reproduces the exact time-centred face value to linear order, which is what makes the Godunov scheme 2nd-order accurate in time under smooth IC.

13.3 Leading 2nd-order truncation error

Expanding the exact midpoint-time value to three Taylor terms,

$$u(x_{j+1/2},\, t_n + \tfrac{\Delta t}{2}) \;=\; u_0 + \tfrac{\Delta t}{2}\, u_t + \tfrac{\Delta t^2}{8}\, u_{tt} + O(\Delta t^3).$$

Using the chain of PDE-derived identities $u_t = -a u_x$, $u_{xt} = -a u_{xx}$, $u_{tt} = a^2 u_{xx}$, the difference between the exact value and the Hancock predictor is

$$u_{\text{true}} - u_{\text{hancock}} \;=\; \tfrac{\Delta t^2}{8}\, a^2\, u_{xx}(x_{j+1/2}, t_n) + O(\Delta t^3). \quad \text{(A12-time-order)}$$

Strong-form verification. sympy simplifies the difference directly after substituting the three PDE rules; the result equals $(\Delta t^2/8)\, a^2 u_{xx}$ identically.

This truncation term is a **positive dissipation** coefficient ($(\Delta t^2/8)\, a^2 > 0$), cancelled order-by-order at the subsequent update step through the HLLC flux jump. The full scheme, MUSCL-Hancock + HLLC + update, is verified 2nd-order accurate in §E1 (modified-equation analysis).

13.4 Cell-average conservation

Because both $\mathbf{U}_L^{n+1/2}$ and $\mathbf{U}_R^{n+1/2}$ are updated with the **same** flux difference, the half-step preserves cell averages:

$$\tfrac{1}{2}(\mathbf{U}_L^{n+1/2} + \mathbf{U}_R^{n+1/2}) \;=\; \tfrac{1}{2}(\mathbf{U}_L^{n} + \mathbf{U}_R^{n}) - \frac{\Delta t}{2h}[\mathbf{F}_R - \mathbf{F}_L]. \quad \text{(A12-conservation)}$$

This is the **finite-volume half-step** identity: a cell-centred integrator that happens to evolve both face states by the same amount. It is the reason the kernel can use the Hancock-updated face states directly as Riemann-problem inputs without worrying about mid-cell-average drift: conservation is built in.

Strong-form verification. sympy directly simplifies the averaged expression to the expected FV half-step form.

13.5 Verification checkpoints

The kernel implements §A12 inside `k_muscl_hancock_x/y`. Tests:

1. **Smooth-IC time-order.** Run entropy-wave on $\{64^2, 128^2, 256^2, 512^2\}$ with fixed CFL; L^1 should decrease as h^2 (2nd-order; the time error is subleading under Strang splitting). Test: `test_strang_convergence.cu`.
2. **Cell-average consistency.** After one Hancock half-step on a smooth IC, manually compute $\frac{1}{2}(\mathbf{U}_L^{n+1/2} + \mathbf{U}_R^{n+1/2})$ and compare to the FV finite-volume half-step formula applied to the cell average. Agreement to ULP precision. Test: `test_strang_muscl.cu` §A12-cell-avg.
3. **Leading-error magnitude.** On a specific smooth IC with known u_{xx}, measure $u_{\text{kernel}}^{n+1/2} - u_{\text{exact}}$ and verify it agrees with $(\Delta t^2/8)\, a^2 u_{xx}$ within 10% (the 10% slack is for accumulated round-off over many cells, not theoretical slack). Test: `test_strang_muscl.cu` §A12-truncation-check.

Failures of (1) indicate a deeper scheme-order bug, most likely in the reconstruction of §A11 or the limiter of §A10 interacting poorly with Hancock. Failure of (2) is a straight bug in the kernel's flux-divergence computation. Failure of (3) is rare but possible — it would indicate an arithmetic mistake in the Hancock half-step formula itself.

14 A13. Time integrator family (Strang / Lie / VL2 / RK2)

sympy script: `scripts/a13_time_integrator_family.py` **generated LaTeX:** `output/a13_time_integrator_family.latex.tex` **verified:** - 10 strong-form identities on the linearised operator expansion — 2 Lie-splitting leading-commutator identities - 4 Strang-splitting Δt^2 cancellation identities - 1 kernel- chain equivalence (all monomials match identically) - 1 VL2 leading-Δt^3 identity - plus the printed report of the 6 non-zero Strang Δt^3 residual coefficients

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: StrangSolver::step`

This is the **fourth alternative-scheme comparison** of the book. Four time-integrator templates are compared by Baker-Campbell- Hausdorff (BCH) expansion on a linearised pair of non-commuting operators $\mathcal{X}, \mathcal{Y}$. The Strang kernel uses splitting; the comparison with the alternatives (Lie, VL2-unsplit, RK2) quantifies the order gap and identifies the Strang scheme's advantage.

14.1 Operator-split abstraction

Write the 2D Euler update as

$$\partial_t \mathbf{U} \;=\; \mathcal{X}(\mathbf{U}) + \mathcal{Y}(\mathbf{U}), \qquad \mathcal{X} = -\partial_x \mathbf{F}_x, \quad \mathcal{Y} = -\partial_y \mathbf{F}_y. \quad \text{(A13-split-form)}$$

Each of $\mathcal{X}, \mathcal{Y}$ is non-linear in $\mathbf{U}$, but for order analysis we **linearise** them (treat as formal non-commuting operators) and expand the exponential $e^{\Delta t(\mathcal{X}+\mathcal{Y})}$ that represents the exact continuous- time solution. Four integrator choices are then products of one-operator exponentials to compare.

The non-linear order verification is deferred: for the non-linear Euler system no closed-form BCH series exists. This section delivers the strong-form proof on the linearised operator ring (which is the tightest sympy can make the identity); the non- linear analogue is verified numerically by the entropy-wave convergence test of §E1.

14.2 Lie splitting (1st order)

$$\mathbf{U}^{n+1} \;=\; e^{\Delta t\, \mathcal{Y}}\, e^{\Delta t\, \mathcal{X}}\, \mathbf{U}^{n}. \quad \text{(A13-Lie)}$$

BCH expansion to Δt^2:

$$e^{\Delta t\, \mathcal{Y}}\, e^{\Delta t\, \mathcal{X}} \;-\; e^{\Delta t\,(\mathcal{X}+\mathcal{Y})} \;=\; -\frac{\Delta t^2}{2}\,[\mathcal{X},\, \mathcal{Y}] + O(\Delta t^3).$$

Strong-form verification. sympy expands both sides as non- commutative polynomials in monomials of $\mathcal{X}, \mathcal{Y}$ up to total degree 3. The coefficient of $\mathcal{X}\mathcal{Y}$ in the difference is $-\Delta t^2/2$; the coefficient of $\mathcal{Y}\mathcal{X}$ is $+\Delta t^2/2$. Their sum is the commutator $-(\Delta t^2/2)[\mathcal{X}, \mathcal{Y}]$, verifying the classical result.

14.3 Strang splitting (2nd order)

$$\mathbf{U}^{n+1} \;=\; e^{(\Delta t/2)\,\mathcal{X}}\, e^{\Delta t\,\mathcal{Y}}\, e^{(\Delta t/2)\,\mathcal{X}}\, \mathbf{U}^{n}. \quad \text{(A13-Strang)}$$

Strong-form verification. sympy expands the triple product up to degree 3. All four Δt^2 monomials ($\mathcal{X}\mathcal{X}, \mathcal{X}\mathcal{Y}, \mathcal{Y}\mathcal{X}, \mathcal{Y}\mathcal{Y}$) have cancelling coefficients in the residual — verified as four separate identities.

The first non-zero residuals appear at Δt^3; they are iterated-commutator terms. sympy reports the six non-zero coefficients:

monomial	coefficient
$\mathcal{X}\mathcal{X}\mathcal{Y}$	$-\Delta t^3/24$
$\mathcal{X}\mathcal{Y}\mathcal{X}$	$+\Delta t^3/12$
$\mathcal{X}\mathcal{Y}\mathcal{Y}$	$+\Delta t^3/12$
$\mathcal{Y}\mathcal{X}\mathcal{X}$	$-\Delta t^3/24$
$\mathcal{Y}\mathcal{X}\mathcal{Y}$	$-\Delta t^3/6$
$\mathcal{Y}\mathcal{Y}\mathcal{X}$	$+\Delta t^3/12$

These rearrange into linear combinations of the nested commutators $[\mathcal{X},[\mathcal{X},\mathcal{Y}]]$ and $[[\mathcal{X},\mathcal{Y}],\mathcal{Y}]$, which is the classical result for Strang symmetric splitting (see, e.g., Sanz-Serna 1992). Collectively they represent the leading $O(\Delta t^3)$ error which vanishes as $\Delta t \to 0$, giving 2nd-order global accuracy.

14.4 Kernel operator chain

The Strang kernel's `step` function applies

$$\mathcal{X}(\Delta t/2)\;\mathcal{Y}(\Delta t/2)\;\mathcal{Y}(\Delta t/2)\;\mathcal{X}(\Delta t/2). \quad \text{(A13-kernel-chain)}$$

By the semigroup property of the $\mathcal{Y}$-flow, $\mathcal{Y}(\Delta t/2)\,\mathcal{Y}(\Delta t/2) = \mathcal{Y}(\Delta t)$, so the kernel chain is **identically** the Strang split:

$$\mathcal{X}(\Delta t/2)\,\mathcal{Y}(\Delta t/2)\,\mathcal{Y}(\Delta t/2)\,\mathcal{X}(\Delta t/2) \;\equiv\; \mathcal{X}(\Delta t/2)\,\mathcal{Y}(\Delta t)\,\mathcal{X}(\Delta t/2).$$

Strong-form verification. sympy expands the four-factor product and the three-factor Strang chain, subtracts, and confirms every monomial coefficient up to degree 3 is zero. The two forms are bit-identical at the operator level.

The kernel's choice of the four-factor form saves one ghost-cell fill per full step: after the first $\mathcal{X}(\Delta t/2)$, the ghost cells are refilled only for the next $\mathcal{Y}$ sweep; the two half-$\mathcal{Y}$ sweeps share a single ghost refill. This is an implementation optimisation, not an algorithmic change.

14.5 Unsplit VL2 (Stone-Gardiner 2009, 2nd order)

The unsplit VL2 scheme — the basis of Athena++ — uses a single unsplit predictor-corrector structure:

$$\mathbf{U}^{\star} \;=\; \mathbf{U}^{n} - \frac{\Delta t}{2}\,\mathcal{L}\,\mathbf{U}^{n}, \qquad \mathbf{U}^{n+1} \;=\; \mathbf{U}^{n} - \Delta t\,\mathcal{L}\,\mathbf{U}^{\star}, \quad \text{(A13-VL2)}$$

with $\mathcal{L} = \mathcal{X} + \mathcal{Y}$ the full spatial operator. Algebraically on linear $\mathcal{L}$:

$$\mathbf{U}^{n+1} \;=\; \big(I - \Delta t\,\mathcal{L} + \tfrac{\Delta t^2}{2}\,\mathcal{L}^2\big)\,\mathbf{U}^n.$$

Compared to $e^{-\Delta t\,\mathcal{L}} = I - \Delta t\,\mathcal{L} + \frac{\Delta t^2}{2}\,\mathcal{L}^2 - \frac{\Delta t^3}{6}\,\mathcal{L}^3 + \ldots$,

$$\mathbf{U}^{n+1} \;-\; e^{-\Delta t\,\mathcal{L}}\,\mathbf{U}^n \;=\; -\,\frac{\Delta t^3}{6}\,\mathcal{L}^3\,\mathbf{U}^n \;+\; O(\Delta t^4).$$

Strong-form verification. sympy confirms the leading error is $-(\Delta t^3/6)\,\mathcal{L}^3$ on the linearised operator ring, so VL2 is also 2nd-order globally.

VL2 vs. Strang trade-off. Both integrators are $O(\Delta t^3)$ on the leading error; VL2 avoids the ghost-refill-between-sweeps cost and therefore is faster in practice at the same accuracy. Strang splitting is simpler and does not require building the full 2D flux gradient in a single pass. The stellar2d kernel uses Strang because it was implemented first; `athena_vl2_solver.cu` later added VL2 as an Athena-parity alternative.

14.6 RK2-MUSCL (Shu-Osher, 2nd order)

$$\mathbf{U}^{(1)} \;=\; \mathbf{U}^n + \Delta t\,\mathcal{L}(\mathbf{U}^n), \qquad \mathbf{U}^{n+1} \;=\; \tfrac{1}{2}\mathbf{U}^n + \tfrac{1}{2}\big(\mathbf{U}^{(1)} + \Delta t\,\mathcal{L}(\mathbf{U}^{(1)})\big).$$

On the linearised operator ring, this reduces algebraically to the same $(I - \Delta t\,\mathcal{L} + \frac{\Delta t^2}{2}\,\mathcal{L}^2)$ form as VL2, up to the specific choice of intermediate stages. It is therefore also 2nd-order with the same leading Δt^3 coefficient on the linearised operator. Not used in the stellar2d kernel; listed here for completeness.

14.7 Order comparison

integrator	leading error	symmetric?
Lie	$-(\Delta t^2/2)\,[\mathcal{X}, \mathcal{Y}]$	no
Strang	$\sim \Delta t^3\,[\mathcal{X}, [\mathcal{X}, \mathcal{Y}]] + \ldots$	yes
VL2	$-(\Delta t^3/6)\,\mathcal{L}^3$	no (but operates on $\mathcal{X} + \mathcal{Y}$)
RK2-MUSCL	$-(\Delta t^3/6)\,\mathcal{L}^3$ (linearised)	no

All three 2nd-order schemes are Δt^3-leading-error but with different error structures. The sizes of the leading errors on a given problem depend on the commutator structure of $\mathcal{X}$ and $\mathcal{Y}$, which for 2D Euler is non-trivial (pressure gradient in x couples to the y-momentum update, and vice versa).

14.8 Non-linear order: [WEAK] via §E1

The BCH analysis above is exact on linearised operators. For the fully non-linear Euler system, the BCH series does not converge in closed form (the commutators of non-linear PDE operators are themselves non-linear PDE operators). We cannot verify 2nd-order accuracy of the kernel on non-linear Euler symbolically. Per Rule 4, this is a legitimate [WEAK] step for sympy — the physics is real, the kernel behaves as expected, but the verification is a numerical measurement of convergence rate rather than a closed-form identity.

The non-linear verification is delivered in §E1 (entropy-wave convergence order): we run the kernel on smooth entropy-wave IC at $N = \{64, 128, 256, 512\}$, fit a log-log slope on L^1 error, and confirm the slope falls in $[1.8, 2.2]$ as predicted by the modified-equation analysis of §E1.

14.9 Verification checkpoints

The kernel does not expose $\mathcal{X}, \mathcal{Y}$ operators directly; the A13 identities are verified at the scheme level through §E1. However, one **mechanical** regression is possible:

1. **Symmetry of the Strang chain.** Starting from a symmetric IC $\mathbf{U}(x, y) = \mathbf{U}(-x, y)$, the Strang-kernel evolution preserves that symmetry to ULP precision — a consequence of the X-operator symmetry in the split plus the B4 periodic-x BC (§B4 to be derived). Violation indicates a bug in the ghost-cell fill OR a non-symmetric $\mathcal{X}$ operator in the kernel. Test: `test_strang_reflection_symmetry.cu` §A13-time-symmetry.

Failure in (1) is a deep structural bug; more typically the A13 invariants are verified via §E1 (convergence rate).

15 A14. Strang operator-chain self-adjointness

> **sympy script:** `scripts/a14_strang_operator_chain.py` **generated LaTeX:** `output/a14_strang_operator_chain.latex.tex` **verified:** - 1 half-flow reversal - 31 Strang-chain self-adjoint monomial identities (every monomial of degree ≤ 4 in $\mathcal{X}, \mathcal{Y}$ cancels identically in $L_{\text{fwd}} L_{\text{bwd}}$) - 2 Lie-splitting non-self- adjoint leading residuals ($\pm\Delta t^2$ on $[\mathcal{X}, \mathcal{Y}]$)
>
> **code checkpoints:** - `src/gpu/explicit/strang_solver.cu :: StrangSolver::step` (the four-factor symmetric chain is structurally self-adjoint)

This section proves the **structural reason** the Strang splitting of §A13 is 2nd-order: it is **self-adjoint** (time-reversible). Self-adjoint integrators have leading error at **even** powers of Δt only; Strang is 2nd-order because the Δt^1 leading error, which would exist for an asymmetric 1st-order scheme like Lie, is structurally forbidden by the time-reversal symmetry of the triple product.

15.1 Self-adjointness definition

A numerical integrator $L(\Delta t)$ is **self-adjoint** (or time-reversible) if

$$L(\Delta t)\, L(-\Delta t) \;=\; I \qquad \forall\, \Delta t. \quad \text{(A14-self-adjoint-def)}$$

Equivalently, evolving forward by Δt and then backward by Δt returns to the identity exactly, for any Δt. This property is **structural**: either it holds for all Δt (and then the leading error is at an even power of Δt), or it fails at some Δt^k for odd k, making the integrator k-th-order rather than $(k+1)$-th.

15.2 Strang splitting is self-adjoint

$$\left[e^{(\Delta t/2)\,\mathcal{X}}\, e^{\Delta t\,\mathcal{Y}}\, e^{(\Delta t/2)\,\mathcal{X}}\right] \;\cdot\; \left[e^{(-\Delta t/2)\,\mathcal{X}}\, e^{-\Delta t\,\mathcal{Y}}\, e^{(-\Delta t/2)\,\mathcal{X}}\right] \;=\; I. \quad \text{(A14-Strang-self-adjoint)}$$

Strong-form verification. sympy expands both factors as non- commutative polynomials in $\mathcal{X}, \mathcal{Y}$ to total degree 4, multiplies them, and checks **every monomial** up to degree 4. Every non-identity monomial coefficient is zero (31 scalar identities); only the empty monomial

retains coefficient 1. The proof extends trivially to higher orders by induction on the time-reversal symmetry of the product.

15.3 Consequence: even-order accuracy

Any self-adjoint integrator has a Taylor series in Δt with **only even-power** error terms. Proof sketch:

- Denote the local error $\mathbf{E}(\Delta t) = L(\Delta t) - e^{\Delta t\,\mathcal{L}}$.
- Self-adjointness of L and $e^{\Delta t\,\mathcal{L}}$ (the exact flow) implies $\mathbf{E}(\Delta t) = -\mathbf{E}(-\Delta t) \cdot e^{2\Delta t\,\mathcal{L}}$ or, after Taylor expansion, $\mathbf{E}(\Delta t)$ contains only even powers of Δt.

Strang is therefore $O(\Delta t^2)$-error = 2nd-order globally. This is a structural result, independent of the commutator identities of §A13.

15.4 Lie splitting fails self-adjointness

$$\left[e^{\Delta t\,\mathcal{Y}}\,e^{\Delta t\,\mathcal{X}}\right] \cdot \left[e^{-\Delta t\,\mathcal{Y}}\,e^{-\Delta t\,\mathcal{X}}\right] = I - \Delta t^2\,[\mathcal{X},\mathcal{Y}] + O(\Delta t^3). \quad \text{(A14-Lie-not-self-adjoint)}$$

Strong-form verification. sympy reports the non-zero residuals at degree 2: coefficient of $\mathcal{X}\mathcal{Y}$ is $-\Delta t^2$, coefficient of $\mathcal{Y}\mathcal{X}$ is $+\Delta t^2$. Their combination is exactly $-\Delta t^2[\mathcal{X},\mathcal{Y}]$.

This explains why Lie is 1st-order: the leading error is at Δt^2 (in the time-reversal composition), which under the standard $\mathbf{E}(\Delta t)$ decomposition gives a Δt- order term in the forward-only error (the split derivative of a Δt^2-order residual).

15.5 Kernel realisation

The Strang kernel applies

$$\mathcal{X}(\Delta t/2)\;\mathcal{Y}(\Delta t/2)\;\mathcal{Y}(\Delta t/2)\;\mathcal{X}(\Delta t/2),$$

which is identical to the three-factor Strang chain by §A13's semigroup composition. Time-reversing the kernel chain factor-by- factor,

$$\mathcal{X}(-\Delta t/2)\;\mathcal{Y}(-\Delta t/2)\;\mathcal{Y}(-\Delta t/2)\;\mathcal{X}(-\Delta t/2),$$

and composing the two gives identity to all orders. The kernel is **structurally self-adjoint**; no quality-of-implementation issue can change this.

The only thing the kernel can do wrong here is an **asymmetric** modification: for example, if one applied a gravity source term as a third operator `Z(Dt)` without a symmetric wrapper, the split would become Lie-type and immediately degrade to 1st-order. §C1 and §E4 derive why the gravity source in the Strang kernel is **absorbed inside** $\mathcal{Y}$ (as a momentum + energy source) rather than inserted as a separate $\mathcal{Z}$ operator, so the full step chain remains symmetric.

15.6 Verification checkpoints

The self-adjointness of the Strang kernel is a structural consequence of the code's topology (the order of sweep calls). Its violation would appear only as:

1. **Asymmetric split.** If someone adds a new physics operator to `StrangSolver::step` without a symmetric wrapper, the split becomes Lie-like. Reference test: `test_strang_step.cu` §A14 asymmetry-smoke — run $X(\Delta t/2) \cdot Y(\Delta t/2) \cdot Y(\Delta t/2) \cdot X(\Delta t/2)$ and $X(-\Delta t/2) \cdot Y(-\Delta t/2) \cdot Y(-\Delta t/2) \cdot X(-\Delta t/2)$ on the same IC and verify the composition returns the original state to ULP precision. Any drift indicates a bug in the kernel's operator order or a non-symmetric source insertion.
2. **Time-reversal smoke test.** Run the kernel forward for N steps at CFL σ, then run backward for N steps at $-\sigma$. Returned state must equal the IC to round-off accumulation $O(\varepsilon_{\text{mach}} N)$. Test: `test_strang_step.cu` §A14 time-reversal.

Failure of (1) is a structural bug (someone broke the split by adding an asymmetric operator). Failure of (2) is either an asymmetric split or an arithmetic bug in one of the half-sweeps (most likely the CFL computation applying floor at different sign conventions).

16 B1. Perturbation storage bijection

sympy script: `scripts/b01_perturbation_storage.py` **generated LaTeX:** `output/b01_perturbation_storage.latex.tex` **verified:** - 4 round-trip (forward + reverse) identity identities - 1 pressure-perturbation split - 8 zero-perturbation invariant identities (4 forward + 4 reverse) - 1 Jacobian-determinant positivity identity

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_cons2prim` - `src/gpu/explicit/strang_solver.cu :: k_strang_init_bubble` - every site that does `+ d_rho_bar[j_phys]` or `+ d_p_bar[j_phys]/gm1`

The Strang solver stores the **perturbation** of the four conservative variables above the isentropic HSE background $(\bar\rho(y), \bar p(y))$. The stored state $\mathbf{U}_{\text{store}} = (\delta\rho, m_x, m_y, \delta E)$ lives in RAM; full conservative state $\mathbf{U} = (\rho, m_x, m_y, E_{\text{tot}})$ is reconstructed at every arithmetic site by adding back the background. This is a **numerical well-balancing trick**: on pure HSE the stored state is identically zero, so round-off is $O(\varepsilon_{\text{mach}})$ rather than $O(\varepsilon_{\text{mach}} \cdot \bar\rho_{\text{bot}})$. The code-level cost is one add/sub per read; the pay-off is HSE-drift bounded by §E5's condition-number estimate.

16.1 Storage map

$$\mathbf{U}_{\text{store}} = \begin{pmatrix} \delta\rho \\ m_x \\ m_y \\ \delta E \end{pmatrix} = \begin{pmatrix} \rho - \bar\rho(y) \\ \rho u \\ \rho v \\ E_{\text{tot}} - \bar p(y)/(\gamma-1) \end{pmatrix}. \quad \text{(B1-store)}$$

The momentum is stored in full because the HSE background is static ($\bar u \equiv 0$, $\bar v \equiv 0$). For total energy the background is a pure internal-energy term $\bar p/(\gamma-1)$ (no background kinetic energy).

16.2 Decode map (reverse)

At every device kernel that performs arithmetic on full-state quantities, the stored state is decoded as

$$\rho = \delta\rho + \bar\rho, \qquad u = m_x/\rho, \qquad v = m_y/\rho, \qquad P = (\gamma-1)\left[\delta E + \bar p/(\gamma-1)\right] - \tfrac{1}{2}\rho(u^2+v^2). \quad \text{(B1-decode)}$$

This is a straightforward inversion. The sympy script confirms that round-tripping $\mathbf{W} = (\rho, u, v, P) \to \mathbf{U}_{\text{store}} \to \mathbf{W}$ is the identity in strong form.

16.3 Pressure-perturbation identity

A direct algebraic rearrangement of the decode map gives

$$\delta P \;\equiv\; P - \bar{p} \;=\; (\gamma - 1)\left[\delta E - \tfrac{1}{2}\rho(u^2 + v^2)\right]. \quad \text{(B1-dP)}$$

This identity is the **reason** the solver stores δE and not full E_{tot}: on a pure acoustic wave with amplitude $\delta P = O(\varepsilon)$, the stored δE is also $O(\varepsilon)$ — the background $\bar{p}/(\gamma - 1)$ cancels exactly. If the solver stored E_{tot} directly, a linear wave with amplitude 10^{-6} on top of $\bar{p} = 1$ would be represented as $E_{\text{tot}} \approx 2.5 + 10^{-6}$ (floating- point subtracting two nearly-equal large numbers), losing 6 digits of precision at every flux-assembly site.

16.4 Zero-perturbation invariant

$$\delta\rho = m_x = m_y = \delta E = 0 \iff (\rho, u, v, P) = (\bar{\rho}, 0, 0, \bar{p}). \quad \text{(B1-zero-pert)}$$

This is the well-balancing statement at the storage level. The kernel's treatment is that any all-zeros state represents pure HSE and is preserved identically by the flux-assembly and update code (the HSE-consistency of the face reconstruction is §B3). **Strong- form verification.** sympy substitutes both directions and simplifies to zero; both substitutions verify independently.

16.5 Positive-definite Jacobian

The differential of the storage map has Jacobian

$$\frac{\partial \mathbf{U}_{\text{store}}}{\partial \mathbf{W}} \;=\; \begin{pmatrix} 1 & 0 & 0 & 0 \\ u & \rho & 0 & 0 \\ v & 0 & \rho & 0 \\ \tfrac{1}{2}(u^2 + v^2) & \rho u & \rho v & 1/(\gamma - 1) \end{pmatrix}, \qquad \det = \frac{\rho^2}{\gamma - 1} > 0 \quad \text{on} \quad \rho > 0. \quad \text{(B1-jacobian)}$$

The determinant is identical to $\det \partial\mathbf{U}/\partial\mathbf{W}$ from §A2 because subtracting the background is a pure translation — it does not change the differential. The map is therefore a smooth diffeomorphism on the admissible domain $\{\rho > 0, P > 0\}$; there is no branch point or coordinate singularity away from the floor state.

16.6 Consequence: canonical decode-before-arithmetic pattern

Every device kernel that reads the stored state performs

```
// Standard pattern throughout strang_solver.cu
double rho = d_rho[k] + d_rho_bar[j_phys];
double u   = d_mx[k] / rho;
double v   = d_my[k] / rho;
double E_t = d_E[k]  + d_p_bar[j_phys] / gm1;
double P   = gm1 * (E_t - 0.5 * rho * (u*u + v*v));
```

This pattern appears in `k_muscl_hancock_x/y`, `k_hllc_update_x/y`, `k_strang_cfl`, `k_strang_init_bubble`, and the host-side VTK/I/O and diagnostics loops. The pattern is implicit in §B1-decode and is the only correct way to compute a full-state quantity from stored state. Any kernel that fails to add back $\bar{\rho}(y)$ or $\bar{p}(y)/(\gamma-1)$ will compute with negative or wrong values.

16.7 Verification checkpoints

1. **Round-trip precision.** Starting from random primitive state (ρ, u, v, P) within the HSE admissibility envelope, encode to $\mathbf{U}_{\text{store}}$ and decode back. Required agreement: $|\Delta\mathbf{W}|/|\mathbf{W}| \leq 2\varepsilon_{\text{mach}}$ (2 ULP, accounting for two subtract-add pairs). Test: `test_strang_unit.cu` §B1-roundtrip.
2. **HSE zero-storage check.** After `init()` builds the HSE background and before any `init_bubble()` or IC perturbation is added, the stored state $\mathbf{U}_{\text{store}}$ is identically zero. Required: `cudaMemcpy` of the state buffer is checked bitwise-equal to a 4-field all-zero buffer. Test: `test_strang_init.cu` §B1-hse-zero.
3. **Pressure-perturbation formula.** Given a specific (ρ, u, v, P) and its stored δE, verify that $P - \bar{p} = (\gamma-1)\left[\delta E - \frac{1}{2}\rho(u^2+v^2)\right]$ to ULP precision. Test: `test_strang_unit.cu` §B1-dP-formula.

Failure of (1) or (3) indicates an arithmetic bug in `d_cons2prim` or the encode side. Failure of (2) is structural and would indicate the HSE builder itself is inconsistent with §B2's closed-form background (a §B2-level bug, not §B1).

17 B2. Isentropic hydrostatic-equilibrium background

> **sympy script:** `scripts/b02_isentropic_hse.py` **generated LaTeX:** `output/b02_isentropic_hse.latex.tex` **verified:** - 1 exponent identity ($\gamma/(\gamma-1)-1 = 1/(\gamma-1)$) - 2 parametric derivative-chain identities (dh/dy, dp/dh) - 2 HSE-ODE identities (parametric and y-world) - 2 parametric state identities - 2 bottom-BC identities - 1 isentropic-closure identity ($P/\rho^\gamma = K$) - 1 atmosphere-cutoff identity - 3 temperature-lapse identities (parametric, y-world, compact)
>
> **code checkpoints:** - `src/gpu/explicit/strang_solver.cu :: StrangSolver::init` (host-side HSE build loop, line 686-692) - `src/gpu/explicit/strang_device.cuh :: d_hse_rho, d_hse_p` (device-side HSE evaluation in §B3 face-state reconstruction)

The Strang solver's background $\bar{\rho}(y), \bar{p}(y)$ is the closed-form solution of the **isentropic** hydrostatic-equilibrium ODE

$$\frac{d\bar{p}}{dy} \;=\; -\bar{\rho}\, g, \qquad \bar{p} \;=\; K\,\bar{\rho}^{\gamma}, \quad \text{(B2-ODE)}$$

with constant gravity $g > 0$ (downward) and a polytropic constant $K > 0$ (entropy $s = \log K$). The resulting density profile is

$$\bar{\rho}(y) \;=\; \left[\rho_0^{\gamma-1} \;-\; \tfrac{(\gamma-1)\,g}{\gamma\,K}\,y\right]^{1/(\gamma-1)}, \quad \text{(B2-rho)}$$

and the pressure follows the equation of state. The profile is **polytropic with adiabatic lapse rate**: the temperature $T \propto P/\rho = K\rho^{\gamma-1}$ falls linearly with height to zero at

$$y^\star \;=\; \frac{\gamma\, K\, \rho_0^{\gamma-1}}{(\gamma-1)\, g}. \quad \text{(B2-ystar)}$$

The atmosphere has a finite thickness (not a log-pressure profile as for isothermal gravity); this is intrinsic to polytropic stratification, not a numerical artefact.

17.1 Strong-form verification path

sympy cannot directly reduce nested fractional powers $\left(h^{1/(\gamma-1)}\right)^{\gamma} \to h^{\gamma/(\gamma-1)}$ for symbolic γ, so the derivation is carried in two steps:

Step 1. Parametric factorisation through the linear argument $h(y) = \rho_0^{\gamma-1} - \frac{(\gamma-1)g}{\gamma K}\, y$. Then

$$\bar\rho \;=\; h^{1/(\gamma-1)}, \qquad \bar p \;=\; K\, h^{\gamma/(\gamma-1)},$$

and the ODE is verified by the chain rule:

$$\frac{d\bar p}{dy} \;=\; \frac{d\bar p}{dh}\,\frac{dh}{dy} \;=\; \frac{\gamma K}{\gamma-1}\, h^{1/(\gamma-1)} \cdot \left(-\frac{(\gamma-1)\, g}{\gamma\, K}\right) \;=\; -g\, h^{1/(\gamma-1)} \;=\; -\bar\rho\, g.$$

Here the key step is the elementary exponent identity $\gamma/(\gamma-1)-1 = 1/(\gamma-1)$, which sympy verifies independently.

Step 2. Direct y-world verification of $d\bar p/dy + \bar\rho g = 0$ via `sp.powdenest(..., force=True)`, which normalises $(K\gamma)^a$ terms so sympy can cancel them. This is **strong-form** (the identity is pointwise in y); the `powdenest` call is a sympy-capability workaround per Rule 1, not a weak-form fallback.

17.2 Isentropic closure

$$s(y) \;=\; \log\!\left(\bar p/\bar\rho^{\gamma}\right) \;=\; \log K \qquad \text{(constant in } y\text{)}. \quad \text{(B2-isentropic)}$$

By construction, since $\bar p = K\bar\rho^{\gamma}$ at every y. The solver's choice of isentropic stratification means every fluid parcel is marginally convectively stable (Schwarzschild criterion with zero super-adiabatic gradient): no spurious convection driven by background thermodynamics.

17.3 Temperature / lapse-rate form

$$\bar p/\bar\rho \;=\; K\, \rho_0^{\gamma-1}\,(1 - y/y^\star), \quad \text{(B2-lapse)}$$

which is linear in y, vanishing at the atmosphere cut-off. For an ideal gas $p = \rho RT/\mu$, this corresponds to temperature $T(y) = T_0(1-y/y^\star)$ with $T_0 = K\rho_0^{\gamma-1}\mu/R$, i.e., linearly decreasing with height — the characteristic adiabatic-atmosphere temperature lapse.

17.4 Atmosphere cut-off

At $y = y^\star$ the argument h of the fractional power vanishes, so formally $\bar\rho(y^\star) = 0$. The kernel clamps $h \geq 10^{-20}$ (`strang_solver.cu` line 689) so that the solver can run with computational domains larger than $y^\star$ without NaN. The physically meaningful region is $0 \leq y < y^\star$; tests should keep $L_y < y^\star$ unless explicitly probing the near-vacuum asymptote.

17.5 Device-side form

The device functions `d_hse_rho(y, rho0_gm1, coeff, inv_gm1)` and `d_hse_p(rho, K, gamma)` in `strang_device.cuh` implement the same formula with pre-computed constants `rho0_gm1 = rho_0^(gamma-1)`, `coeff = (gamma-1) g / (gamma K)`, and `inv_gm1 = 1/(gamma-1)`. These are evaluated at face-centre y-coordinates during the y-sweep MUSCL-Hancock reconstruction (§B3) so that the face pairs see **identical** HSE background contribution.

17.6 Verification checkpoints

1. **Host-host consistency.** The C++ init loop in `StrangSolver::init` at a cell centre $y_j = y_{\text{lo}} + (j + 1/2)\,\Delta y$ must produce $\bar\rho[j], \bar p[j]$ equal to §B2's closed-form $\bar\rho(y_j), \bar p(y_j)$ to ULP precision. Test: `test_strang_init.cu` §B2-profile-match — compare the init array against a host-computed reference over all cells.
2. **Device-device consistency.** At an arbitrary face-centred y_{face}, `d_hse_rho` and `d_hse_p` must equal the closed-form $\bar\rho(y_{\text{face}})$, $\bar p(y_{\text{face}})$ to ULP precision. Test: `test_strang_muscl.cu` §B2-hse-face.
3. **HSE ODE residual.** Finite-difference the initialised arrays: $|\bar p[j+1] - \bar p[j]|/\Delta y + \bar\rho[j + 1/2]\, g$ should converge to zero as $\Delta y \to 0$ at second order. Test: `test_strang_init.cu` §B2-ode-convergence.
4. **Isentropic closure.** For all j, $|\bar p[j] - K\bar\rho[j]^\gamma| \le 10\varepsilon_{\text{mach}}\bar p[j]$. Test: `test_strang_init.cu` §B2-isentropic.
5. **Goldens dump.** Part D's `d06_hse_zero_perturbation_lock.py` (7.6 goldens) emits a reference profile JSON at `N=8192` y-points for a canonical HSE setup; the test consumer reads this JSON and compares to the kernel's init array.

Failure of (1) is a host-side bug (the closed-form formula was mistyped or the `j + 0.5` cell-centre convention was violated). Failure of (2) is a device-side inconsistency between host init and device re-evaluation (most likely sign error in `coeff` or an inconsistent definition of `rho0_gm1`). Failure of (3) or (4) indicates structural loss of the HSE property — triage by inspecting which profile (rho vs p) went wrong first. Failure of (5) is rare; it usually means the regression golden was generated at a different K, ρ_0 than the kernel is using.

18 B3. Face-centred HSE reconstruction (well-balancing necessary condition)

sympy script: `scripts/b03_face_hse_reconstruction.py` **generated LaTeX:** `output/b03_face_hse_reconstruction.latex.tex` **verified:** - 4 face-state equality identities ($\rho_L = \rho_R$, $P_L = P_R$, $u_L = u_R$, $v_L = v_R$ on pure HSE) - 4 face-flux equality identities ($F_{y,L}[k] = F_{y,R}[k]$, $k = 0..3$) - 4 face-flux form identities (F_y at HSE = $(0, 0, \bar p, 0)$) - 1 cell-centred-reconstruction counter-example identity

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_muscl_hancock_y` (line 343-372: `y_bot`, `y_top` at face; `d_hse_rho`, `d_hse_p` evaluated at face y-coord; `rL = rho_bar_bot + rhoP_bot`, `PL = p_bar_bot + PP_bot`)

The y-sweep reconstructs two face-state vectors $\mathbf{U}_L$ and $\mathbf{U}_R$ at each face between cells j and $j+1$. The **well-balancing** (WB) requirement is: on pure HSE ($\delta\rho \equiv 0, \delta P \equiv 0, u \equiv v \equiv 0$), the reconstructed face states must be **algebraically identical** so that the HLLC flux jump vanishes at round-off precision. The section proves that this is achieved only when the HSE background $\bar\rho, \bar p$ is evaluated **at the face y-coordinate**, with the perturbation variables added

on both sides. Cell-centred background reconstruction breaks WB at $O(\Delta y)$ and drives a drift of order $|d\bar{\rho}/dy|$ per step.

18.1 Face reconstruction formula

Let $y_{\text{face}} = y_{\text{lo}} + j_{\text{face}}\,\Delta y$ (the face index is $j_{\text{face}} = j+1$ between cells j and $j+1$). Each side computes

$$\begin{aligned}
\rho_{L/R} &= \bar{\rho}(y_{\text{face}}) + (\delta\rho_{j/j+1} \mp \tfrac{1}{2}\, s^{\rho}_{j/j+1}),\\
P_{L/R} &= \bar{p}(y_{\text{face}}) + (\delta P_{j/j+1} \mp \tfrac{1}{2}\, s^{P}_{j/j+1}),\\
u_{L/R} &= u_{j/j+1} \mp \tfrac{1}{2}\, s^{u}_{j/j+1},\\
v_{L/R} &= v_{j/j+1} \mp \tfrac{1}{2}\, s^{v}_{j/j+1},
\end{aligned} \qquad \text{(B3-face-recon)}$$

where s^{X}_{j} is the MC-limited slope of variable X in cell j (§A10). The L-side (top face of cell j) uses a $+\frac{1}{2}s$ extrapolation, and the R-side (bottom face of cell $j+1$) uses $-\frac{1}{2}s$.

18.2 WB necessary condition (strong form)

On pure HSE, $\delta\rho \equiv 0$, $\delta P \equiv 0$, $u \equiv v \equiv 0$, and all MC slopes are zero (the slope operator is multi-linear in its inputs, so $s(\mathbf{0}, \mathbf{0}) = \mathbf{0}$). Then **by direct substitution**:

$$\rho_L = \bar{\rho}(y_{\text{face}}), \qquad \rho_R = \bar{\rho}(y_{\text{face}}), \qquad \rho_L - \rho_R = 0, \quad \text{(B3-WB)}$$

and analogously for P, u, v. sympy verifies all four component equalities as strong-form identities (no simplification required — direct substitution).

18.3 HSE face-flux form

On pure HSE, $u = v = 0$, so the Euler flux simplifies to

$$\mathbf{F}_y(\mathbf{U}_{\text{HSE}}) = (\bar{\rho}v,\ \bar{\rho}uv,\ \bar{\rho}v^2 + \bar{p},\ (E+\bar{p})v)^{\mathsf{T}}\,\big|_{u=v=0} = (0,\ 0,\ \bar{p}(y_{\text{face}}),\ 0)^{\mathsf{T}}. \quad \text{(B3-face-flux)}$$

sympy verifies both component-wise equalities $F_L[k] = F_R[k]$ on pure HSE and the explicit form $(0, 0, \bar{p}, 0)^{\mathsf{T}}$ for each component $k \in \{0, 1, 2, 3\}$.

18.4 HLLC on HSE degenerates to the exact pressure flux

When $\mathbf{U}_L = \mathbf{U}_R = \mathbf{U}_{\text{HSE}}$, the HLLC Riemann solver (§A8) is exactly the left/right flux:

$$\mathbf{F}_{\text{HLLC}}(\mathbf{U}_L, \mathbf{U}_R)\Big|_{\mathbf{U}_L = \mathbf{U}_R} = \mathbf{F}_y(\mathbf{U}_L) = (0, 0, \bar{p}(y_{\text{face}}), 0)^{\mathsf{T}}.$$

This follows algebraically from §A8's HLLC strong-form identities: on an identical left-right pair, any wave-branch gives the same flux, and that flux is $\mathbf{F}_y(\mathbf{U}_{\text{HSE}})$. No sympy re-verification is needed — the §A8 identities carry over.

18.5 Balance with the gravity source

The kernel's y-sweep update combines the HLLC flux divergence with the gravity source term $S_{m_y} = -\rho g$, $S_E = -m_y g$ (§C1). The flux divergence at cell j is

$$-\frac{1}{\Delta y}\big[F_y(U_{j+1/2}) - F_y(U_{j-1/2})\big]\Big|_{\text{HSE}} \;=\; -\frac{1}{\Delta y}\big[(0,0,\bar{p}_{j+1/2},0) - (0,0,\bar{p}_{j-1/2},0)\big] \;=\; -(0,0,\frac{d\bar{p}}{dy}\big|_j,0) \;=\; (0,0,$$

The gravity source contributes $(0, 0, -\bar{\rho}_j g, -0 \cdot g) = (0, 0, -\bar{\rho}_j g, 0)$. Their sum is exactly zero, closing the HSE balance to the pointwise discretisation accuracy of §B2's finite-difference ODE residual (which is 2nd-order as $\Delta y \to 0$, per §E5). This is the **only** way the kernel can preserve HSE to round-off; the alternative (cell-centred background) fails below.

18.6 Cell-centred reconstruction: the counter-example

If the kernel had instead reconstructed the face from cell-centre backgrounds,

$$\rho_L^{\text{wrong}} \;=\; \bar{\rho}(y_j) \;+\; \delta\rho_j, \qquad \rho_R^{\text{wrong}} \;=\; \bar{\rho}(y_{j+1}) \;+\; \delta\rho_{j+1},$$

then on pure HSE the difference is

$$\rho_L^{\text{wrong}} - \rho_R^{\text{wrong}} \;=\; -\Delta y\,\frac{d\bar{\rho}}{dy} \;+\; O(\Delta y^3), \quad \text{(B3-wrong)}$$

which is non-zero (the HSE density changes between cells by $\Delta y\, d\bar{\rho}/dy$). The HLLC solver would see this as a spurious density jump at every face and produce a non-zero flux of order Δy per face per step, driving a drift of order $|d\bar{\rho}/dy|$ that accumulates linearly in time. WB would be broken at the leading 1st order.

18.7 Implementation check: face-centred evaluation

The kernel correctness relies on the lines

```
// strang_solver.cu, k_muscl_hancock_y, line 343-345
double y_bot = y_lo + j_phys * dy;          // face y-coordinate
double rho_bar_bot = d_hse_rho(y_bot, rho0_gm1, hse_coeff, inv_gm1);
double p_bar_bot   = d_hse_p(rho_bar_bot, K_poly, gamma);
```

(for the top face, line 360-362 uses `y_top = y_lo + (j_phys + 1) * dy`). Both sides of the face evaluate $\bar{\rho}, \bar{p}$ at the **same** y_{face} because the face index is a single integer — there is no "L-side face y" and "R-side face y". This is the structural guarantee of WB.

18.8 Verification checkpoints

1. **Pure HSE face-state equality.** Start the solver with the HSE background, zero perturbations. At every internal face, assert $\mathbf{U}_L = \mathbf{U}_R$ to ULP precision (all four components). Test: `test_strang_muscl.cu` §B3-hse-face-equal.
2. **Pure HSE flux jump.** Compute the HLLC flux jump $\mathbf{F}_R - \mathbf{F}_L$ at every internal face after the MUSCL-Hancock predictor; required $\le 10\varepsilon_{\text{mach}}\bar{p}(y_{\text{face}})$ (only the P component has a non-zero scale). Test: `test_strang_muscl.cu` §B3-hse-flux.

3. **Long-time HSE preservation.** After 10^4 Strang steps on pure HSE IC, the max-norm of the perturbation state is bounded by $\varepsilon_{\text{mach}} \cdot N \cdot \kappa(\bar{p}, \bar{\rho})$, where κ is the condition number of the face-centred HSE evaluation (§E5). Test: `test_strang_step.cu` §B3-hse-longtime.

Failure of (1) or (2) is a structural bug in `k_muscl_hancock_y` — most likely a cell-centred background reuse (copy-paste of the HSE background from cell j instead of evaluating at the face). Failure of (3) is either (1)/(2) failing silently, or a more subtle bug in the HLLC flux assembly (see §C1 for the gravity- source balance that must be correct for (3) to hold).

19 B4. Periodic-x boundary condition

sympy script: `scripts/b04_periodic_x_bc.py` **generated LaTeX:** `output/b04_periodic_x_bc.latex.tex` **verified:** - 2 index-offset identities (left and right ghost offsets = n_x) - 1 physical- distance identity ($x_{\text{src}} - x_{\text{ghost}} = L_x$) - 1 periodic-manufactured-solution identity (sin wave with $kL_x = 2\pi m$) - 4 flux-commutativity identities ($F_x(U_{\text{ghost}})[i] = F_x(U_{\text{phys}})[i]$, $i = 0..3$)

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_ghost_x` (line 33, cell-data copy: `d_*[k_dst] = d_*[k_src]` where `ig_ghost = g` or `nx + g`) - `src/gpu/explicit/strang_solver.cu :: k_ghost_face_x` (line 564, face-state copy: `d_wL[ig=ng-1]` from `d_wL[ig=ng+nx-1]` etc.)

Periodic-x is the simpler of the two BCs in the Strang kernel: the solution is required to satisfy $\mathbf{U}(x + L_x, y, t) = \mathbf{U}(x, y, t)$, and the ghost cells are filled by a direct copy from the physical interior across the domain. No physics transformation is applied (contrast with §B5's reflective BC).

19.1 BC identity

$$\mathbf{U}(x + L_x, y, t) \;=\; \mathbf{U}(x, y, t) \quad \forall\,(x, y, t). \quad \text{(B4-periodic)}$$

19.2 Ghost-cell copy formulas

Using the kernel's physical index convention where $i_{\text{phys}} \in \{0, 1, \ldots, n_x - 1\}$ is the interior index (index-0 is the leftmost physical cell) and ghost layers extend beyond:

$$\begin{aligned} \mathbf{U}_{\text{ghost}}(i_{\text{phys}} = -1 - g) \;&=\; \mathbf{U}(i_{\text{phys}} = n_x - 1 - g), \\ \mathbf{U}_{\text{ghost}}(i_{\text{phys}} = n_x + g) \;&=\; \mathbf{U}(i_{\text{phys}} = g), \end{aligned} \qquad g \in \{0, \ldots, n_g - 1\}. \quad \text{(B4-ghost)}$$

Equivalently, $i_{\text{src}} - i_{\text{ghost}} = n_x$ on the left side and $i_{\text{ghost}} - i_{\text{src}} = n_x$ on the right side. Both directions implement the wrap-around $i \mapsto i + n_x$ or $i - n_x$.

19.3 Consistency with the PDE

Taking a periodic-in-x smooth solution $\mathbf{U}(x, y, t) = \mathbf{U}(x + L_x, y, t)$ and substituting $x_{\text{ghost}} = i_{\text{ghost}}\,\Delta x$, $x_{\text{src}} = (i_{\text{ghost}} + n_x)\,\Delta x$, the physical distance is $x_{\text{src}} - x_{\text{ghost}} = L_x$, so $\mathbf{U}(x_{\text{ghost}}) = \mathbf{U}(x_{\text{src}})$ automatically. sympy verifies this for a sine-wave manufactured solution with $kL_x = 2\pi m$ (m integer); the more general case (any periodic $\mathbf{U}$) follows from the fundamental periodic hypothesis.

19.4 Ghost-cell width

MUSCL-Hancock (§A12) reads a 3-point stencil $(i-1, i, i+1)$ in each sweep to compute the MC slope (§A10). The predictor therefore requires one layer of ghost. **Additionally**, the face-state refill `k_ghost_face_x` (line 564) operates on the ghost face **after** the predictor has written face states into `d_wL, d_wR`, and the ghost faces are indexed at $i_g = n_g - 1$ and $i_g = n_g + n_x$ — i.e., the first and last **ghost** cell positions. These positions hold cell data that feeds the next sweep's boundary Riemann problem. The full round-trip requires $n_g \geq 2$:

- Layer 1: feeds the MUSCL stencil at the boundary cell.
- Layer 2: holds face-state values at the boundary face for the HLLC Riemann problem.

The kernel uses $n_g = 2$ in `StrangSolver::init()` line 639.

19.5 Flux commutativity

Since the Euler flux $\mathbf{F}_x$ is a pointwise function of $\mathbf{U}$, the copy identity transfers automatically:

$$\mathbf{F}_x(\mathbf{U}_{\text{ghost}}) \;=\; \mathbf{F}_x(\mathbf{U}_{\text{phys}}), \quad \text{(B4-flux-commute)}$$

component-wise. sympy verifies this as a trivial identity (the ghost copy preserves the argument of $\mathbf{F}_x$, so the output is identical). This closes the BC: no additional flux-side correction is needed.

19.6 Compatibility with perturbation storage

The perturbation storage $(\delta\rho, m_x, m_y, \delta E)$ is periodic in x whenever the full state $(\rho, m_x, m_y, E_{\text{tot}})$ is periodic, because the HSE background $\bar{\rho}(y), \bar{p}(y)$ depends only on y: adding a y-only function to a x-periodic function preserves x-periodicity. Therefore the copy on stored variables is automatically the correct BC on the full state — no re-encoding at the boundary is needed.

19.7 Face-state ghost fill

After MUSCL-Hancock writes $\mathbf{w}_L, \mathbf{w}_R$ at each internal cell, the face-state arrays have undefined values at the ghost cells $i_g = n_g - 1$ and $i_g = n_g + n_x$. The kernel `k_ghost_face_x` at line 564 fills them using the same periodic copy pattern:

- $\mathbf{w}_L$ at ghost $i_g = n_g - 1$ (left boundary face) is copied from $\mathbf{w}_L$ at $i_g = n_g + n_x - 1$ (the last physical cell's left face, which by periodicity corresponds to the left-boundary face on the opposite side).
- $\mathbf{w}_R$ at ghost $i_g = n_g + n_x$ (right boundary face) is copied from $\mathbf{w}_R$ at $i_g = n_g$.

This preserves the MUSCL reconstructions computed on the first ghost layer by `k_muscl_hancock_x` (which reads cell data from the outer ghost layer, which is already periodic).

19.8 Verification checkpoints

1. **Periodic-copy exactness.** After `k_ghost_x` has run, every ghost-cell value is bit-identical to its source. Test: `test_strang_unit.cu` §B4-ghost-copy.
2. **Smooth IC round-trip.** A periodic sine-wave IC (§D1 entropy wave) evolved for one period $T = L_x/u_0$ should return to the initial state to truncation-error precision (limited by §E1's $O(\Delta x^2)$ bound, not by the BC itself). Test: `test_strang_convergence.cu`.

3. **Ghost-face consistency.** After the y-sweep has touched the face-state ghost fill on the **x**-periodic axis, the HLLC flux at $i = n_g$ (left boundary) should equal the HLLC flux at $i = n_g + n_x$ (right boundary) to machine precision. Test: `test_strang_unit.cu` §B4-face-flux-equal.

Failure of (1) is a straight copy bug in `k_ghost_x`. Failure of (2) is either (1) failing silently or a deeper bug in the HLLC / update sequence that breaks periodic closure. Failure of (3) is a `k_ghost_face_x` bug (index miscomputation or wrong source-destination pairing).

20 B5. Reflective-y bottom boundary condition

sympy script: `scripts/b05_reflective_y_bc.py` **generated LaTeX:** `output/b05_reflective_y_bc.latex.tex` **verified:** - 1 involution identity ($\mathcal{R}_{\text{ref}}^2 = \mathbf{I}$) - 4 flux-reversal identities ($\mathbf{F}_y(\mathcal{R}\mathbf{U}) = \mathcal{R}'\mathbf{F}_y(\mathbf{U})$, 4 components) - 4 wall-face flux identities ($\mathbf{F}_y(\rho, u, 0, P) = (0, 0, P, 0)$) - 4 HSE-perturbation-zero identities

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_ghost_y` (line 71, bottom branch: `jg_ghost = ng-1-g`, `jg_src = ng+g`, `d_my[k_dst] = -d_my[k_src]`) - `src/gpu/explicit/strang_solver.cu :: k_ghost_face_y` (line 593, bottom branch: `d_wR[kd*4+2] = -d_wL[ks*4+2]` (v-component negated))

The bottom of the Strang domain is a solid wall. The ghost cells are the mirror image of the physical cells across the wall plane, with the normal component of velocity (here v) negated. In conservative-variable form this is the reflection

$$\mathcal{R}_{\text{ref}} \;=\; \text{diag}(+1,\, +1,\, -1,\, +1). \quad \text{(B5-R-ref)}$$

The $+1$s keep $\rho, m_x, E_{\text{tot}}$ unchanged; the -1 negates m_y. This is the correct choice: at the wall, a fluid parcel moving towards the wall is bounced back with its normal velocity reversed, while its tangential velocity and thermodynamic state are preserved.

20.1 Ghost-cell copy formula

With kernel index convention (cell $j_{\text{phys}} \in \{0, 1, \ldots, n_y - 1\}$ with ghost layers at $j_g \in \{0, \ldots, n_g - 1\}$ below and $\{n_g + n_y, \ldots, n_g + n_y + n_g - 1\}$ above):

$$\mathbf{U}_{\text{ghost}}(j_g = n_g - 1 - g) \;=\; \mathcal{R}_{\text{ref}}\,\mathbf{U}(j_g = n_g + g), \quad g \in \{0, \ldots, n_g - 1\}. \quad \text{(B5-ghost)}$$

For $n_g = 2$: $g = 0$ mirrors the first physical cell ($j_g = n_g = 2 \to j_g = n_g - 1 = 1$), and $g = 1$ mirrors the second ($j_g = 3 \to j_g = 0$). The physical wall plane is at $j_g = n_g - 1/2 = 1.5$.

20.2 Flux-reversal identity (strong form)

Reflection and flux assembly commute up to a component-wise sign-flip pattern:

$$\mathbf{F}_y(\mathcal{R}_{\text{ref}}\,\mathbf{U}) \;=\; \text{diag}(-1, -1, +1, -1)\,\mathbf{F}_y(\mathbf{U}), \quad \text{(B5-flux-reversal)}$$

where the "flux-reflection matrix" differs from $\mathcal{R}_{\text{ref}}$: mass, x-momentum, and energy fluxes **do** flip sign (they are linear in v), but the y-momentum flux does **not** (it depends on ρv^2 and P, both even in v). sympy verifies all four components independently.

The immediate physical consequence: at the wall face (where $\mathbf{U}_L = \mathcal{R}_{\text{ref}}\mathbf{U}_R$), the flux-difference contributions from mass, x-momentum, and energy vanish after HLLC averaging by the sign-flip symmetry of the L/R pair, while the y-momentum contribution remains non-zero but isotropic (purely pressure).

20.3 Wall-face flux (v = 0 at wall)

At the wall the normal velocity vanishes (this is the physical content of the no-penetration boundary). Substituting $v = 0$:

$$\mathbf{F}_y(\rho, u, 0, P) \;=\; (0,\; 0,\; P,\; 0)^{\mathsf{T}}. \quad \text{(B5-wall-flux)}$$

No mass or kinetic energy flux crosses the wall. The y-momentum flux is the pressure at the wall — this is Newton's third law (force on the wall = force on the fluid, transmitted by pressure).

Consequence for HLLC at the wall. The Riemann problem with $\mathbf{U}_L = \mathcal{R}_{\text{ref}}\mathbf{U}_R$ has $S_\star = 0$ (by the L/R symmetry in §A8): the intermediate contact wave sits exactly at the wall. The HLLC flux is the contact-wave flux, which by §A8 evaluates to $(0, 0, P^\star, 0)$ with $P^\star$ equal to the physical-side pressure P_R (by §A8's $p_L^\star = p_R^\star$ strong-form identity). No mass or kinetic energy is transported across the wall to round-off.

20.4 Involution

$$\mathcal{R}_{\text{ref}}^2 \;=\; \mathbf{I}, \quad \text{(B5-involution)}$$

which makes reflection a $\mathbb{Z}_2$ symmetry operation. Chaining two reflective BCs returns to the original state; this is useful for two-wall-bounded domains (not used in the Strang kernel's single-wall bottom, but structurally important for §D7).

20.5 HSE preservation

On pure HSE, the perturbation state is identically zero at every cell. Reflection maps zero to zero component-wise:

$$\mathcal{R}_{\text{ref}}\,(0, 0, 0, 0)^{\mathsf{T}} \;=\; (0, 0, 0, 0)^{\mathsf{T}}.$$

So the ghost perturbation is also zero, and the reconstructed face state at the wall is exactly $(\bar\rho, 0, 0, \bar p/(\gamma - 1))^{\mathsf{T}}$. By §B3, $L = R$ on pure HSE, and the wall-face flux is $(0, 0, \bar p, 0)$ — balanced by the cell-interior gravity source $(0, 0, -\bar\rho g, 0)$. The wall does not break well-balancing.

Subtle point. The ghost cell is at a y-coordinate $y_{\text{ghost}} = -y_{\text{phys}}$ (mirrored across the wall $y = 0$). The HSE background $\bar\rho, \bar p$ is **not** symmetric in y (it decreases monotonically with height), so naively one might worry that reflection breaks HSE. The kernel's design avoids this by storing only the **perturbation**, which is zero on pure HSE and stays zero under reflection. The HSE background is evaluated only at **physical** or **face** y-coordinates in the MUSCL predictor (§B3), never at ghost y-coordinates. This decoupling is essential for the reflective BC to be HSE-preserving.

20.6 Face-state reflection

The face-state ghost fill `k_ghost_face_y` at line 593 applies the same reflection on $\mathbf{w}_L, \mathbf{w}_R$, with the special handling that the ghost cell's top face $\mathbf{w}_R$ is the **reflected** version of the physical cell's bottom face $\mathbf{w}_L$:

```
// strang_solver.cu, k_ghost_face_y, line 601-609
d_wR[kd*4+0] =  d_wL[ks*4+0];   // rho unchanged
d_wR[kd*4+1] =  d_wL[ks*4+1];   // u unchanged
d_wR[kd*4+2] = -d_wL[ks*4+2];   // -v (reflect normal)
d_wR[kd*4+3] =  d_wL[ks*4+3];   // P unchanged
```

This implements the reflection matrix on the face-state primitive vector (ρ, u, v, P) (the -1 applies only to the normal component v). The mapping is from the physical cell's $\mathbf{w}_L$ (bottom face) to the ghost's $\mathbf{w}_R$ (top face, which lies on the wall).

20.7 Verification checkpoints

1. **Pure-HSE preservation.** After arbitrary ghost-fill passes on HSE IC, the perturbation state remains bitwise zero at all ghost and physical cells. Test: `test_strang_unit.cu` §B5-hse-pres.
2. **Wall-flux zero-mass.** Initialise a non-HSE y-flux at the wall cell (e.g., a symmetric-bouncing IC), run one Strang step, and confirm that the total mass integrated over the physical domain is conserved to ULP precision (no leakage through the wall). Test: `test_strang_step.cu` §B5-wall-mass.
3. **Flux-reversal identity check.** Given random admissible $\mathbf{U}$, compute $\mathbf{F}_y(\mathcal{R}\mathbf{U})$ and $\mathcal{R}'\mathbf{F}_y(\mathbf{U})$ on the host; agreement to ULP precision for all four components. Test: `test_strang_unit.cu` §B5-flux-reversal.
4. **Face-state reflection.** After `k_ghost_face_y`, assert that $\mathbf{w}_R[\text{ghost}] = \mathcal{R}\,\mathbf{w}_L[\text{phys}]$ for all four components. Test: `test_strang_unit.cu` §B5-face-reflection.

Failure of (1) is a sign error in `k_ghost_y` or a BG-related bug (the kernel is subtracting or adding the wrong background at the wall). Failure of (2) is a deeper issue — the wall is leaking mass, which means the Riemann-problem symmetry is broken (either $\mathcal{R}$ is wrong, or HLLC's L/R treatment is not symmetric). Failure of (3) is a direct math bug and should be fixed by reading §A1's flux formula and restoring the correct signs.

21 B6. Outflow-y top boundary condition

sympy script: `scripts/b06_outflow_y_bc.py` **generated LaTeX:** `output/b06_outflow_y_bc.latex.tex` **verified:** - 4 ghost-uniform identities ($\mathbf{U}_{\text{ghost}}(g_1)[k] = \mathbf{U}_{\text{ghost}}(g_2)[k]$, $k = 0..3$) - plus 2 documentation identities (Neumann continuum interpretation; Riemann-invariant extrapolation error leading order)

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_ghost_y` (line 71, top branch: `jg_ghost = ng+ny+g`, `jg_src = ng+ny-1` — all ghosts copy from the same last physical cell) - `src/gpu/explicit/strang_solver.cu :: k_ghost_face_y` (line 618, top branch: outflow face-state copy)

The top of the Strang domain is an **outflow** boundary implemented by zero-gradient copy: every ghost cell is a copy of the last physical cell. The BC is non-reflecting for supersonic outflow and only approximately non-reflecting for subsonic outflow (one characteristic is incoming and its amplitude is set by zeroth- order extrapolation, which admits an $O(\Delta y)$ reflection error). This is a standard choice for 2D compressible hydro with gravity — the alternatives

(exact characteristic BC, sponge layers) are more complex and the kernel opts for the simple, robust zero-gradient form.

21.1 Ghost-cell copy formula

$$\mathbf{U}_{\text{ghost}}(j_g = n_g + n_y + g) \;=\; \mathbf{U}(j_g = n_g + n_y - 1), \quad g \in \{0, \dots, n_g - 1\}. \quad \text{(B6-zero-gradient)}$$

All ghost cells receive the **same** copy (the last physical cell's value). This is a zeroth-order extrapolation — it is flat, not linearly extrapolated.

21.2 Continuum interpretation

In the limit $\Delta y \to 0$, the zero-gradient copy is equivalent to the Neumann BC

$$\left.\frac{\partial \mathbf{U}}{\partial y}\right|_{y=y_{\text{top}}} \;=\; \mathbf{0}, \quad \text{(B6-neumann)}$$

imposed componentwise on all four conservative variables. sympy verifies that all ghosts are uniform copies (pairwise identical), which is the discrete statement of zero normal derivative.

21.3 Characteristic structure of the y-flux Jacobian

From §A3 (rotationally covariant: swap $x \leftrightarrow y$), the eigenvalues of $\mathcal{A}_y(\mathbf{U})$ are

$$\text{spec}(\mathcal{A}_y) \;=\; \{v - c,\;\; v,\;\; v,\;\; v + c\}, \quad \text{(B6-eigenvalues)}$$

with corresponding eigenvectors (see §A3 rotated to y).

21.4 Subsonic outflow

If $0 < v < c$ at the top, three characteristics $(v, v, v + c)$ propagate **out of** the domain and one ($v - c < 0$) propagates **into** the domain. The incoming characteristic carries information from outside the domain that the BC must supply.

The 1D Riemann-invariant carried by the incoming acoustic wave is

$$R_{\_} \;=\; u - \frac{2c}{\gamma - 1}, \quad \text{(B6-R-minus)}$$

where u is the velocity and c the sound speed (in 1D along the y-axis $u \leftrightarrow v$; we keep the 1D notation here). The **correct** non-reflecting BC would set $R_{\_}^{\text{ghost}} = R_{\_}^{\text{ext}}$ where $R_{\_}^{\text{ext}}$ is the value implied by whatever condition holds outside (for stellar-atmosphere outflow this is typically zero perturbation in $R_{\_}$).

Zero-gradient copy instead sets $R_{\_}^{\text{ghost}} \;=\; R_{\_}^{\text{phys,last}}$, the interior value at the last cell. The leading-order error is

$$R_{\_}^{\text{extrap}} \;-\; R_{\_}^{\text{true}} \;=\; -\,\tfrac{\Delta y}{2}\,\frac{dR_{\_}}{dy} \;+\; O(\Delta y^2), \quad \text{(B6-error)}$$

i.e., an $O(\Delta y)$ reflection at the boundary. For smooth steady-state outflow where $dR_-/dy \approx 0$ this is acceptable. For strong transients (an acoustic pulse travelling up) the zero-gradient BC will partially reflect the pulse, generating an $O(\Delta y)$ returning wave. If the test requires clean outflow, a sponge layer or a proper characteristic BC is needed; the Strang kernel does not provide one.

21.5 Supersonic outflow

For $v > c$ at the top, all four eigenvalues are positive (all outgoing). No information flows in from outside the domain. Zero- gradient copy is then **exact** in the sense that the interior solution's evolution cannot depend on the ghost values — the BC is irrelevant provided it does not cause instability. Zero-gradient is stable (no additional reflection).

21.6 HSE interaction

On **pure HSE**, the perturbation state is zero at every physical cell. Zero-gradient copies zero to the ghost. The reconstructed face state at the top boundary uses the face y-coordinate for the HSE background (§B3), so the face-(ρ, u, v, P) pair is $(\bar{\rho}(y_{\text{top}}), 0, 0, \bar{p}(y_{\text{top}}))$ on both sides. §B3's WB identity holds, and HSE is preserved.

For **non-HSE interior states**, zero-gradient does **not** preserve HSE in the ghost — the interior perturbation values (some non-zero combination of $\delta\rho, m_x, m_y, \delta E$) are copied, and the reconstructed face state may differ from the pure-HSE face state. This is intrinsic to the outflow BC: material is allowed to leave the domain, and the ghost state represents what has "just left". The gravity source still operates correctly because it reads the physical-cell values, not the ghost values.

21.7 Face-state ghost fill

The face-state ghost fill `k_ghost_face_y` (line 618) mirrors the cell-data ghost fill: the top-boundary ghost faces receive copies of the last physical cell's face states. For outflow, the top ghost's $\mathbf{w}_L$ (bottom face) is set to the physical cell's $\mathbf{w}_R$ (top face) and the ghost's $\mathbf{w}_R$ is set to the same:

```
// strang_solver.cu, k_ghost_face_y, line 620-625 (outflow top branch)
for (int c = 0; c < 4; ++c) {
    d_wL[kd*4+c] = d_wR[ks*4+c];   // ghost bottom = last cell top
    d_wR[kd*4+c] = d_wR[ks*4+c];   // ghost top = same (outflow)
}
```

This closes the top-boundary face-state buffers for the next sweep's HLLC Riemann problem.

21.8 Trade-off and alternatives

BC	accuracy	incoming char	implementation	stability
zero-gradient (current)	$O(\Delta y)$	linearly extrapolated	1 copy	stable
linear extrapolation	$O(\Delta y^2)$	linearly extrapolated	2-point extrap	stable
characteristic BC	exact (smooth)	solved separately	complex; requires 1D Riemann at boundary	stable

BC	accuracy	incoming char	implementation	stability
sponge-layer damping	varies	attenuated	many layers, tunable	stable

The Strang kernel uses the simplest (zero-gradient) because (a) it is the standard Godunov outflow choice, (b) stellar-atmosphere applications typically have steady-state outflow where $dR_-/dy \approx 0$, and (c) tests in §D-series and §E-series do not probe the outflow reflection sensitivity. Future work needing quiet outflow should add a characteristic BC or sponge layer.

21.9 Verification checkpoints

1. **Zero-gradient copy exactness.** All ghost cells are bit- identical to the last physical cell. Test: `test_strang_unit.cu` §B6-ghost-copy.
2. **Pure-HSE preservation.** On HSE IC, the top ghost cells retain zero perturbation state; the face reconstruction at the top boundary agrees with §B3's WB identity. Test: `test_strang_unit.cu` §B6-hse-pres.
3. **Supersonic outflow exit.** Initialise a supersonic y-flow at the top (e.g., Gaussian pulse with $v > c$) and measure the return flux at the top boundary; required: returned flux amplitude $< 10^{-6}$ of the outgoing pulse. Test: `test_strang_step.cu` §B6-supersonic-outflow.
4. **Subsonic outflow reflection.** Initialise a subsonic acoustic pulse travelling upwards and measure the reflection amplitude; expected: $O(\Delta y)$ reflection (which at $n_y = 256$ is $\sim 10^{-2}$ of the outgoing pulse). Test: `test_strang_step.cu` §B6-subsonic-reflection.

Failure of (1) is a copy bug in `k_ghost_y` (wrong source index). Failure of (2) is usually a §B1 perturbation storage bug (ghost is being filled with full state instead of perturbation). Failures (3) and (4) indicate a deeper flux or reconstruction bug.

22 C1. Gravity source consistency

sympy script: `scripts/c01_gravity_source_consistency.py` **generated LaTeX:** `output/c01_gravity_source_consistency.latex.tex` **verified:** - 1 kinetic+potential energy conservation law ($\partial_t(E+\rho g y) + \nabla\cdot[(E+P+\rho g y)\mathbf{v}] = 0$) - 1 HSE balance reduction identity - 1 work-form identity ($S_E = -m_y g = \rho v \cdot (-g)$)

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_hllc_update_y` (line 513-516: `S_my = -rho_total * g_grav; S_E = -d_my[k0] * g_grav;` line 522-523: separate `+= dt * S_my` and `+= dt * S_E` applied after the HLLC flux divergence)

The Strang solver's y-sweep applies the gravity source term $\mathbf{S}(\mathbf{U}; g) = (0, 0, -\rho g, -m_y g)^{\mathsf{T}}$ inside `k_hllc_update_y`. This section proves that

1. This is the **unique** source consistent with the Euler PDE under a uniform downward gravitational acceleration $g\mathbf{e}_y$;
2. The form closes a strong-form conservation law for **total** (kinetic + internal + gravitational-potential) energy;
3. On pure HSE the source exactly cancels the background pressure gradient, preserving well-balancing;

4. The Strang kernel inserts gravity **only once** inside the y-sweep update — no double-counting via flux-side source injection (pre-empting a known family of lowmach-solver bugs).

22.1 PDE with gravity

$$\partial_t \mathbf{U} \;+\; \partial_x \mathbf{F}_x \;+\; \partial_y \mathbf{F}_y \;=\; \mathbf{S}(\mathbf{U}; g), \qquad \mathbf{S}(\mathbf{U}; g) \;=\; \big(0,\; 0,\; -\rho g,\; -m_y g\big)^{\mathsf{T}}. \quad \text{(C1-euler-gravity)}$$

The non-zero components are:

- **y-momentum.** $-\rho g$ is the gravitational force per unit volume: mass ρ times acceleration g in the $-\mathbf{e}_y$ direction.
- **energy.** $-m_y g = -\rho v g$ is the work done by gravity per unit time per unit volume: force $(-\rho g)$ times velocity v. It is negative when $v > 0$ (fluid rising, losing kinetic energy) and positive when $v < 0$ (fluid falling, gaining kinetic energy).

22.2 Kinetic + potential energy conservation

Define the gravitational potential energy density $\rho g y$. The total mechanical + internal energy $E + \rho g y$ satisfies a strong-form conservation law:

$$\frac{\partial}{\partial t}(E + \rho g y) \;+\; \nabla\cdot\big[(E + P + \rho g y)\,\mathbf{v}\big] \;=\; 0. \quad \text{(C1-KE-PE-conservation)}$$

Strong-form verification. sympy substitutes the mass equation $\partial_t \rho = -\nabla\cdot(\rho\mathbf{v})$ and the energy equation with source $-m_y g$, computes $\partial_t(E + \rho g y)$, and confirms it equals the divergence $-\nabla\cdot[(E + P + \rho g y)\mathbf{v}]$ exactly. The algebra turns on the identity

$$g y\, \nabla\cdot(\rho\mathbf{v}) \;+\; \rho v g \;=\; \nabla\cdot(g y \rho \mathbf{v}),$$

which follows from $\partial_y(g y \rho v) = g\rho v + g y \partial_y(\rho v)$, i.e., the $\rho v g$ work term is exactly absorbed into the divergence of $g y \rho \mathbf{v}$. This is **not** a coincidence: it is the content of the virial identity for a stratified gas in uniform gravity.

The conservation law is the mathematical statement that gravity is a conservative force: no energy is lost or gained in any closed volume beyond what flows across the boundary.

22.3 HSE balance reduction

On pure HSE ($\delta\rho = \delta P = u = v = 0$, stored $m_y = 0$), the y-momentum equation reduces to

$$\partial_t m_y \;+\; \partial_y(\rho v^2 + P) \;+\; \rho g \;=\; 0 \;+\; \partial_y \bar{p} \;+\; \bar{\rho} g. \quad \text{(C1-HSE-balance)}$$

By §B2's HSE ODE $\partial_y \bar{p} = -\bar{\rho} g$, the right-hand side is identically zero. Stored m_y stays at zero — HSE is preserved strong-form by the gravity source plus the flux gradient, and not by either alone.

Discrete consequence. The kernel's §B3-compliant face flux reconstruction produces a flux-divergence term $-(\bar{p}_{j+1/2} - \bar{p}_{j-1/2})/\Delta y$ at cell j, which discretely approximates $-\partial_y \bar{p} = \bar{\rho} g$. The kernel's source term $-\bar{\rho}_j g$ then exactly cancels this to the cell-centred background-density value. The residual is the difference between the cell-centred $\bar{\rho}_j$ and the cell- averaged

$\int \bar{\rho}\, dy/\Delta y$, which is $O(\Delta y^2)$ — a structural truncation error bounded by §E5's HSE drift estimate.

22.4 Work-form of the energy source

$$S_E \;=\; -m_y g \;=\; \rho v \cdot (-g). \quad \text{(C1-work)}$$

sympy confirms the tautology (same expression written two ways). The physical content is that S_E is the work rate of the gravitational body force: force density × velocity $= (-\rho g) v = -m_y g$.

22.5 Kernel applies gravity ONCE

The only place the gravity source is added in the kernel is inside `k_hllc_update_y` at line 522-523:

```
double rho_total = fmax(d_rho[k0] + rho_bg, 1e-20);
double S_my = -rho_total * g_grav;
double S_E  = -d_my[k0] * g_grav;
// ... flux divergence via HLLC ...
d_my[k0] = d_my[k0] - dtdy * (GT_mn - GB_mn) + dt * S_my;
d_E [k0] = d_E [k0] - dtdy * (GT3   - GB3)   + dt * S_E;
```

There is no second gravity term inside `k_muscl_hancock_y` (the Hancock predictor does not include gravity — see §A12: gravity is a simple source that does not contribute to the half-step flux divergence at leading order), and there is no gravity term inside the x-sweep or inside the flux evaluation `d_euler_flux_y`.

Why this matters. A known anti-pattern in related solvers (see CLAUDE.md's "P32 lowmach-family bug") is to add a gravity term **both** inside the source and inside the flux, counting the energy work twice. The symptom is an unphysical energy gain proportional to $\rho v g \Delta t$ per step, which accumulates into a secular drift. The Strang kernel avoids this by keeping gravity as a pure source and never embedding it in the flux.

22.6 Operator-split consistency

The gravity source is absorbed into the $\mathcal{Y}$-sweep (the y-direction flux operator), not added as a separate $\mathcal{Z}$-operator. Under §A14's self-adjointness condition, this choice is compatible with 2nd-order Strang splitting: the full operator chain is still $\mathcal{X}(\Delta t/2)\,\mathcal{Y}(\Delta t/2)\,\mathcal{Y}(\Delta t/2)\,\mathcal{X}(\Delta t/2)$, symmetric about $\Delta t/2$.

If the gravity source were split off as $\mathcal{Z}(\Delta t)$ acting after the Strang chain (a "sequential" addition), the composite would become $\mathcal{X}(\Delta t/2)\,\mathcal{Y}(\Delta t)\,\mathcal{X}(\Delta t/2)\,\mathcal{Z}(\Delta t)$, which is **not** symmetric under time reversal; the splitting would degrade to 1st-order. §E4 verifies that the kernel's absorption of gravity into $\mathcal{Y}$ preserves 2nd-order Strang order.

22.7 Verification checkpoints

1. **Bit-identical source-application.** On a known state with known ρ_{tot}, m_y, compute the kernel's S_{m_y}, S_E contributions via `cudaMemcpy` of the state buffer before and after `k_hllc_update_y`, subtract the flux-divergence contribution, and verify the residual equals $dt \cdot (-\rho_{\text{tot}} g, -m_y g)$ to ULP precision. Test: `test_strang_hllc.cu` §C1-gravity-source.

2. **Pure-HSE long-time preservation.** After 10^4 Strang steps on HSE IC, the stored $m_y, \delta E$ stay at $O(\varepsilon_{\text{mach}} N)$ — no secular drift. Test: `test_strang_step.cu` §C1-hse-long.
3. **KE+PE conservation on closed domain.** Initialise a symmetric perturbation (§D7 reflection-symmetric IC) on a closed-wall domain; after any number of Strang steps, $\int_V (E + \rho g y)\, dV$ is conserved to $O(\varepsilon_{\text{mach}} N)$. Test: `test_strang_step.cu` §C1-kepe-conservation.
4. **No double-counting.** Code-inspection check: search for any occurrence of `g_grav` in `k_muscl_hancock_y`, `k_hllc_update_x`, or `d_euler_flux_{x,y}`. Required: zero occurrences. Test: `test_strang_unit.cu` §C1-code-inspection.

Failure of (1) is an arithmetic bug in the kernel's source application (sign error or wrong variable — e.g., using $\delta\rho$ instead of ρ_{tot}). Failure of (2) means HSE is being broken by gravity application, often due to the cell-centred background vs. face-centred reconstruction mismatch (§B3). Failure of (3) is a deeper energy-balance violation. Failure of (4) flags a likely double-count bug that (1)-(3) might not catch if the extra term is small enough on the test IC.

23 C2. CFL bound

sympy script: `scripts/c02_cfl_bound.py` **generated LaTeX:** `output/c02_cfl_bound.latex.tex` **verified:** - 1 Lax-Friedrichs amplification identity ($|g|^2 = 1 + (\nu^2 - 1)\sin^2(k\Delta x)$, strong-form sympy) - 1 max-wave-speed identity $\max(|u-c|, |u+c|) = |u| + c$ — sympy cannot fold absolute-value expressions for symbolic sign, so this is verified at 100 random samples with max residual 0

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: k_strang_cfl` (line 529-556: computes $(|u|+c)/\Delta x + (|v|+c)/\Delta y$ per cell; host reduction gives global max; $\Delta t = \sigma / \max\{\cdot\}$)

The kernel's time-step selection implements a **conservative** CFL condition based on a 1D von-Neumann analysis of the linear scalar advection equation, combined into a single 2D-style estimator. This section proves the underlying 1D stability bound and documents how the Strang kernel's choice relates to the split-vs-unsplit stability regions.

23.1 1D linear advection: Lax-Friedrichs von-Neumann analysis

For $u_t + a u_x = 0$ the Lax-Friedrichs (LF) update in amplification form gives

$$|g(k)|^2 \;=\; 1 \;+\; (\nu^2 - 1)\,\sin^2(k\,\Delta x), \qquad \nu \;=\; \frac{a\,\Delta t}{\Delta x}. \quad \text{(C2-1D-LF)}$$

Stability. Von-Neumann requires $|g(k)| \le 1$ for all k. Since $\sin^2(k\Delta x) \in [0, 1]$, the worst case is $\sin^2 = 1$, giving $|g|^2_{\max} = 1 + (\nu^2 - 1) = \nu^2$, and $|g|^2 \le 1$ iff $|\nu| \le 1$.

sympy verifies the amplification identity via direct expansion $\mathrm{Re}(g)^2 + \mathrm{Im}(g)^2$ with $g = \cos\theta - i\nu\sin\theta$; the analytic simplification reduces to $\cos^2\theta + \nu^2\sin^2\theta = 1 + (\nu^2 - 1)\sin^2\theta$. The stability condition $|\nu| \le 1$ then follows from elementary calculus (supremum over θ).

23.2 Fastest Euler wave speed

From §A3, the flux Jacobian $\mathcal{A}_x$ has eigenvalues $\{u - c, u, u, u + c\}$. The fastest signal speed — the absolute value of the largest eigenvalue — is

$$\lambda_{\max}(\mathcal{A}_x) \;=\; \max(|u-c|,\,|u|,\,|u+c|) \;=\; |u|+c. \quad \text{(C2-euler-wave)}$$

Proof. The identity $\max(|u-c|, |u+c|) = |u|+c$ is the standard triangle inequality: for any reals u, c (with $c > 0$), $|u+c|+|u-c| \geq |u+c|-|u-c| = 2u$ if $u \geq 0$ (giving $|u+c| = u+c$), or $2c$ if $u \leq 0$ (giving $|u-c| = c-u = |u|+c$). In either case $\max = |u|+c$.

sympy cannot fold nested absolute values for symbolic u (sign unknown), so this is verified at 100 random (u, c) samples; the residual is exactly 0 in all samples.

23.3 Strang-split 2D CFL

Each 1D sweep in the Strang kernel has its own stability condition:

$$\Delta t \cdot \max\left\{\frac{|u|+c}{\Delta x}\right\} \leq \sigma_{1D}, \qquad \Delta t \cdot \max\left\{\frac{|v|+c}{\Delta y}\right\} \leq \sigma_{1D}. \quad \text{(C2-Strang-cfl)}$$

For the linear LF scheme $\sigma_{1D} = 1$. For MUSCL-Hancock the 1D stability limit is slightly tighter ($\sigma_{1D} \approx 2/3$ or 0.5 depending on the limiter family) because the higher-order reconstruction adds dispersive error that can go unstable at $\nu = 1$.

Strang splitting means each sweep updates an intermediate state, and the stability region is the **rectangular** product $\{(\nu_x, \nu_y) : \nu_x \leq \sigma_{1D}, \nu_y \leq \sigma_{1D}\}$. This allows $\nu_x = \nu_y = 1$ simultaneously (linear LF limit) — a full factor of 2 more than the unsplit case below.

23.4 Combined 2D estimator (used in kernel)

The kernel's CFL buffer computes

$$\text{buf}[i,j] \;=\; \frac{|u_{ij}|+c_{ij}}{\Delta x} \;+\; \frac{|v_{ij}|+c_{ij}}{\Delta y}. \quad \text{(C2-combined)}$$

The global Δt is $\sigma / \max\{\text{buf}\}$. This enforces

$$\nu_x + \nu_y \;\leq\; \sigma, \qquad \nu_x = (|u|+c)\Delta t/\Delta x, \;\; \nu_y = (|v|+c)\Delta t/\Delta y.$$

This is the **unsplit 2D MUSCL** form of the CFL condition, which is more conservative than the Strang-split rectangular region. It over-restricts the time step for a Strang-split scheme — at the cost of a factor of 2 in Δt it buys robustness across transients where the Strang splitting's "independent sweeps" assumption breaks down (e.g., during large shocks where the reconstruction error grows faster than the linear LF analysis predicts).

23.5 Stability margin comparison

Scheme / bound	$\sigma_{\max}$
split 1D linear LF	1.0
split 1D MUSCL	~ 0.67
unsplit 2D LF	0.5 (diamond: $\nu_x + \nu_y \leq 1$, worst corner at $\nu_x = \nu_y = 0.5$)
unsplit 2D MUSCL	~ 0.4

Scheme / bound	$\sigma_{\max}$
kernel default	**0.4**

The kernel's default $\sigma = 0.4$ is at the unsplit 2D MUSCL limit; it provides $\sim 60\%$ margin below the Strang-split 1D LF bound. The choice trades per-step efficiency for robustness across realistic hydrodynamic tests (shocks, contact discontinuities, gravity-driven convection) where the linear analysis is optimistic. Users can relax to $\sigma = 0.8$ or 1.0 for smooth flows (e.g., §D1 entropy wave) if per-step cost matters.

23.6 Acoustic vs. advective CFL

In the stratified-atmosphere setup (HSE background $\bar{\rho}, \bar{p}$ with $c \approx \sqrt{\gamma \bar{p}/\bar{\rho}}$), the sound speed c dominates $|u|, |v|$ throughout — the flow is low-Mach. The CFL condition is therefore **acoustic**:

$$\Delta t \;\sim\; \frac{\sigma \min(\Delta x, \Delta y)}{c_{\max}}.$$

This is the restriction that motivates the LM-HLLC blending of §C3: the pressure-dissipation of standard HLLC at this Δt smears acoustic waves too aggressively, so a Mach-dependent blend reduces the dissipation without violating CFL.

23.7 Verification checkpoints

1. **Kernel CFL reduction.** After `k_strang_cfl` populates the buffer, the host reduction computes $\max\{\text{buf}\}$. The Δt selected is $\sigma / \max\{\text{buf}\}$; verify that $\nu_x + \nu_y$ at the globally-tightest cell equals σ to ULP precision. Test: `test_strang_step.cu` §C2-cfl.
2. **Linear stability.** On an IC with known advection velocity (e.g., uniform $u = 0.5$, $v = 0$, $c = 1$), run with $\sigma = 0.99$; the scheme remains stable for > 100 steps. Run with $\sigma = 1.01$ (just above the split 1D bound) and check that the scheme becomes unstable (entropy wave amplitude grows). Test: `test_strang_step.cu` §C2-near-limit.
3. **Low-Mach-dominant CFL.** On the HSE IC, measure Δt and confirm it is acoustic-limited ($\Delta t \approx \sigma \Delta y / c_{\max}$), not advective. Test: `test_strang_step.cu` §C2-acoustic.

Failure of (1) is a kernel arithmetic bug. Failure of (2) — specifically stability at $\sigma > 1$ — indicates the linear analysis is wrong, but in practice the kernel becomes unstable slightly below $\sigma = 1$ due to non-linear terms. Failure of (3) would mean the CFL formula is not computing the expected acoustic speed — likely a missing factor of c or wrong density reference.

24 C3. LM-HLLC blending

sympy script: `scripts/c03_lm_hllc_blending.py` **generated LaTeX:** `output/c03_lm_hllc_blending.latex.tex` **verified:** - 1 transonic ($M = 1$) reduction to standard HLLC - 1 linearity identity ($\partial S_\star / \partial f_M$) - 3 reflective-BC identities ($p_R - p_L = 0$, f_M invariance, wall $S_\star = 0$) - 1 dispersion-ratio identity ($\sim 1/M$ suppression) - 1 Mach-cutoff clamp identity

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (lines 107-122: `fM = fmin(1.0, fmax(M_local, M_cutoff))`; line 127-128: `S_star = (fM * (PR - PL) + ...)/denom`)

Standard HLLC injects a pressure-jump term $p_R - p_L$ into the contact-wave speed $S_\star$ (§A8). For low-Mach convective flows this pressure jump is dominated by the **hydrostatic** component of pressure ($\delta P = O(\rho g H)$, which is $O(\rho c^2)$), not by the physical convective signal. The resulting numerical dissipation scales as $\rho c^3 M$, but the physical convective flux scales as $\rho c^3 M^3$, so the numerical dissipation overwhelms the signal by a factor M^{-2}.

The LM-HLLC fix multiplies the pressure jump by a Mach-based blend factor f_M, reducing the dissipation by a factor $f_M \sim M$, which matches the physical scaling. At transonic and supersonic speeds ($M \geq 1$), $f_M = 1$ recovers standard HLLC. At arbitrary low Mach, a floor $M_{\text{cut}} = 10^{-3}$ retains enough dissipation for stability.

24.1 LM-HLLC contact wave formula

$$S_\star \;=\; \frac{f_M\,(P_R - P_L) \;+\; \rho_L u_L (S_L - u_L) \;-\; \rho_R u_R (S_R - u_R)}{\rho_L (S_L - u_L) \;-\; \rho_R (S_R - u_R)}. \quad \text{(C3-S-star-LM)}$$

The **only** modification relative to §A8's standard HLLC is $P_R - P_L \to f_M (P_R - P_L)$ in the numerator. The denominator is unchanged, and the Davis wave-speed estimates S_L, S_R (§A9) are unchanged.

24.2 Mach-based blend factor

$$f_M \;=\; \text{clamp}(M_{\text{local}},\, M_{\text{cut}},\, 1), \qquad M_{\text{local}} \;=\; \frac{|u_L| + |u_R|}{c_L + c_R}, \qquad M_{\text{cut}} \;=\; 10^{-3}. \quad \text{(C3-fM)}$$

- M_{local} is an average Mach at the face, symmetric in L, R (important for well-balancing — an asymmetric choice would break the symmetry of the numerical flux).
- $M_{\text{cut}} = 10^{-3}$ is the lower floor. It sets a minimum pressure dissipation so that even at vanishing Mach the scheme is stable (against purely internal-energy sources that would otherwise destabilise via negative effective viscosity).
- $f_M \leq 1$ always, so LM-HLLC is strictly a **reduction** of standard HLLC dissipation, never an enhancement.

24.3 Reduction at $M = 1$

$$f_M\big|_{M=1} \;=\; 1 \;\Longrightarrow\; S_\star^{\text{LM}} \;=\; S_\star^{\text{HLLC}}. \quad \text{(C3-M1)}$$

sympy verifies this as a direct substitution. At $M \geq 1$ (transonic and supersonic), LM-HLLC is exactly standard HLLC: the blend adds zero error in the physically-important shock regime.

24.4 Reflective-BC invariance

Under the reflective BC substitution (§B5): $\rho_L = \rho_R$, $u_L = -u_R$, $P_L = P_R$. The pressure jump vanishes identically:

$$P_R - P_L\big|_{\text{reflective}} \;=\; 0, \quad \text{(C3-reflective-pressure-jump)}$$

so the f_M factor multiplying it is irrelevant. LM-HLLC and standard HLLC give identical $S_\star$ on reflective L/R pairs — LM-HLLC preserves §B5's wall-symmetry exactly:

$$S_\star^{\text{LM}}\big|_{\text{reflective}} \;=\; S_\star^{\text{HLLC}}\big|_{\text{reflective}} \;=\; 0. \quad \text{(C3-wall-symmetry)}$$

The third identity (sympy verified) uses the additional fact that Davis wave speeds on the reflective pair satisfy $S_L = -S_R$ by $\mathbf{u} \mapsto -\mathbf{u}$ symmetry, which makes the full $S_\star$ numerator vanish.

24.5 Low-Mach dispersion suppression

Scaling argument (§E3 quantifies this as effective viscosity):

Regime	standard HLLC dissipation	LM-HLLC dissipation	ratio
$M \sim 1$	$\sim \rho c^3$	$\sim \rho c^3$	1
$M \ll 1$	$\sim \rho c^3 \cdot M$	$\sim \rho c^3 \cdot M^2$	M^{-1}
$M \le M_{\text{cut}}$	$\sim \rho c^3 \cdot M$	$\sim \rho c^3 \cdot M_{\text{cut}}$	M/M_{cut}

At the atmospheric Mach $M \sim 10^{-2}$, LM-HLLC suppresses pressure dissipation by $M/M_{\text{cut}} \sim 10$ without going to zero (which would be numerically unstable for some operators). sympy verifies the $1/M$ ratio factor between the two dispersions (dimensional analysis on Mach-linear dispersion from §E3).

24.6 Implications for acoustic convergence tests

An **acoustic wave** has velocity perturbation $\delta u = O(Mc)$ and pressure perturbation $\delta P = O(M\rho c^2)$ — **both** are of order M. The true physical decay rate under HLLC is proportional to the numerical pressure dissipation $\sim c\delta P = O(M\rho c^3)$. With LM-HLLC enabled ($f_M \to M$), the dissipation drops to $O(M^2\rho c^3)$: the acoustic wave is artificially amplified (relative to standard HLLC, the expected $\sim M^2$ decay becomes $\sim M^3$).

For **§D2's acoustic linwave test** (convergence order of HLLC on a known acoustic mode), `use_lm_fix` must be **disabled** so the measured convergence rate reflects the standard HLLC theory (2nd order in Δx, with $\nu_{\text{eff}} = c\Delta x/2$). With `use_lm_fix = true` the measured convergence would be artificially super-linear because the LM-HLLC dissipation is suppressed below the leading-order truncation of the spatial reconstruction. This is what the kernel's `use_lm_fix = false` branch (line 120-121) enables for testing:

```
if (use_lm_fix) {
    // ... compute M_local and fM = clamp(M_local, M_cut, 1)
} else {
    fM = 1.0;
}
```

24.7 Implications for convective tests

For **§D5's bubble test** (low-Mach convective flow over HSE), `use_lm_fix = true` is physically mandatory: without it, the convective signal is swamped by the pressure-dissipation artefact and the bubble rises too slowly or fragments prematurely. This is the opposite of the acoustic case above — for each test class, the correct flag value follows from the pressure-dissipation structure of the expected solution.

24.8 Robustness against strong shocks

At a strong shock, $M_{\text{local}} \approx (|u_L| + |u_R|)/(c_L + c_R) \sim 1$ (since the post-shock velocity is of order c), so $f_M \to 1$ and LM-HLLC recovers standard HLLC. The modification is inactive

where full HLLC dissipation is needed. This is a design feature: LM-HLLC is a **low-Mach correction**, not a shock-capturing modification.

24.9 Verification checkpoints

1. $f_M = 1$ **regime.** On a strong-shock Sod IC (§D3), verify LM-HLLC produces bit-identical output to standard HLLC (`use_lm_fix = true` vs `false` comparison). Test: `test_strang_hllc.cu` §C3-shock-equiv.
2. **Reflective BC symmetry.** On a symmetric IC (§D7) with `use_lm_fix = true`, the solution preserves reflection symmetry to ULP precision (the f_M factor does not break it). Test: `test_strang_reflection_symmetry.cu` §C3-sym.
3. **Low-Mach dispersion scaling.** On a low-Mach acoustic wave ($M = 10^{-2}$), measure the effective numerical viscosity with `use_lm_fix = false` vs `true`; the ratio should be close to $M/M_{\text{cut}} = 10$ (§E3 quantifies). Test: `test_strang_linwave_convergence.cu` §C3-nu-ratio.
4. **HSE preservation with LM fix.** On pure HSE with `use_lm_fix = true`, the state stays at $O(\varepsilon_{\text{mach}} N)$ drift — the LM fix does not destroy well-balancing. Test: `test_strang_step.cu` §C3-hse-lm.

Failure of (1) is an arithmetic bug in the LM branch. Failure of (2) means f_M is asymmetric between L and R. Failure of (3) would indicate an incorrect M_{local} formula. Failure of (4) is rare — it would mean the f_M multiplication breaks the §B3 WB guarantee, which should be impossible because $P_R - P_L = 0$ on pure HSE (symmetric MUSCL reconstruction) makes the f_M factor irrelevant.

25 C4. Smooth-flow entropy invariant

> **sympy script:** `scripts/c04_entropy_invariant.py` **generated LaTeX:** `output/c04_entropy_invariant.latex.tex` **verified:** - 1 chain-rule identity ($D_t s = (1/P) D_t P - (\gamma/\rho) D_t \rho$) - 1 entropy invariant ($D_t s = 0$ on smooth Euler) - 1 entropy-function chain identity - 1 entropy-function invariant ($D_t(P\rho^{-\gamma}) = 0$) - 1 entropy-function identity ($\log K = s$, via `expand_log(force=True)`)
>
> **code checkpoints:** - `src/gpu/explicit/strang_solver.cu :: write_vtk` (entropy computed as a post-processing diagnostic; the kernel does not enforce $D_t s = 0$, it emerges from §E1's smooth- convergence measurement)

On smooth solutions of the Euler equations (no shocks, no discontinuities), the specific entropy

$$s \;=\; \log(P\rho^{-\gamma}) \;=\; \log P \;-\; \gamma \log \rho \quad \text{(C4-entropy)}$$

is a **Lagrangian invariant**:

$$\left(\partial_t + \mathbf{v}\cdot\nabla\right) s \;=\; D_t s \;=\; 0. \quad \text{(C4-invariant)}$$

Each fluid parcel maintains its initial entropy forever, provided the flow stays smooth. This is the material-derivative form of the adiabatic first law — no heat exchange, no dissipation, so entropy is constant per parcel.

At **shocks** the entropy strictly **increases** (§A5), so this invariant fails by design — shock-capturing schemes must introduce numerical dissipation equivalent to entropy production per the Lax condition. The identity in this section is the smooth-flow limit.

25.1 Derivation

From mass conservation $D_t\rho = -\rho\,\nabla\cdot\mathbf{v}$ and the adiabatic pressure equation $D_tP = -\gamma P\,\nabla\cdot\mathbf{v}$ (which follows from energy + mass via substitution), the chain rule gives

$$D_t s \;=\; \frac{D_t P}{P} \;-\; \gamma\,\frac{D_t\rho}{\rho} \;=\; -\gamma\,\nabla\cdot\mathbf{v} \;+\; \gamma\,\nabla\cdot\mathbf{v} \;=\; 0. \quad \text{(C4-deriv)}$$

Strong-form verification. sympy substitutes the PDE-derived forms of $\partial_t P, \partial_t\rho$ into the chain-rule expansion of $D_t s$ and simplifies to zero. The equivalent form

$$D_t(P\rho^{-\gamma}) \;=\; 0 \quad \text{(C4-K)}$$

is also verified independently via chain-rule + PDE substitution. The two forms are equivalent via $s = \log(P\rho^{-\gamma})$; the second is the **entropy function** used in the HSE background definition of §B2 (where $K = P/\rho^\gamma$ is an explicit constant).

25.2 Gravity is irrelevant

The material derivative $D_t s$ depends only on the local thermodynamic state, not on the gravity source. Gravity enters the momentum and energy equations as non-zero RHS, but in the entropy derivation:

- D_tP is derived from mass + (energy equation minus momentum $\cdot\mathbf{v}$ equation), and the gravity terms (which appear in both energy and momentum with consistent signs, §C1) **cancel** in the difference.
- $D_t\rho$ is derived from mass, which has no gravity source.

So $D_t s = 0$ holds **even with gravity** on smooth flows. This is the statement that gravity is a conservative body force that does not dissipate energy into entropy — Potential energy exchanges with kinetic energy, with internal energy (and entropy) unchanged.

25.3 HSE consistency

On pure HSE ($\mathbf{v} = \mathbf{0}$), the material derivative $D_t s = (\partial_t + \mathbf{v}\cdot\nabla)s = \partial_t s = 0$ because $s = \log K$ is constant in y (isentropic stratification, §B2). The entropy invariant is satisfied trivially — there is no advection to advect anything, and the Eulerian $\partial_t s$ is zero because s is constant.

25.4 Shock contrast with §A5

The §A5 entropy inequality states

$$\partial_t\eta + \partial_i(\eta u_i) \le 0, \qquad \eta = -\rho s,$$

with strict equality only on smooth states. At shocks $\partial_t\eta + \partial_i(\eta u_i) < 0$, which by standard manipulation implies $D_t s > 0$ (entropy of a parcel crossing a shock increases). The smooth-flow identity $D_t s = 0$ is the equality case of the §A5 inequality — they are consistent, not competing, statements.

25.5 Numerical consequences

On the Strang kernel:

- **Smooth-flow tests** (§D1 entropy wave, §D5 bubble) should preserve s to truncation-error precision. The kernel's numerical dissipation (from HLLC and MUSCL) introduces an $O(\Delta x^2 \nabla^2 s)$ entropy production on smooth flow, which converges to zero as $\Delta x \to 0$ (measured in §E1).
- **Shock tests** (§D3 Sod, §D4 Woodward-Colella blast) should show s strictly increasing across shocks. The rate is set by HLLC's numerical entropy production (which matches the physical rate to leading order under sufficient resolution, per §A5 Lax condition).
- **HSE tests** (§D6 hse-zero-lock) preserve $s = \log K$ to machine precision, since the perturbation storage keeps $\delta\rho = \delta P = 0$ and the full state matches §B2's K-constant background.

25.6 Diagnostic in the kernel

The kernel does **not** evolve s explicitly — it is a derived quantity. The VTK diagnostic output includes s as a computed field (line 946-947 of strang_solver.cu after cons2prim). Tests in §D and §E series read this field and check the invariance property at the numerical level.

25.7 Verification checkpoints

1. **Entropy preservation on entropy wave.** At the end of one entropy-wave period, the entropy distribution $s(x, y)$ must return to its initial value to $O(\Delta x^2)$. Test: `test_strang_convergence.cu` §C4-entropy-wave.
2. **HSE entropy uniformity.** On pure HSE, $s(x, y)$ is identically $\log K$ to machine precision at every cell. Test: `test_strang_init.cu` §C4-hse-entropy.
3. **Shock entropy jump sign.** Across a shock (§D3 Sod), the measured post-shock entropy is strictly greater than the pre-shock entropy (agreeing with Rankine-Hugoniot prediction from §A5). Test: `test_strang_sod.cu` §C4-shock-entropy.
4. **Smooth bubble entropy tracking.** The bubble IC (§D5) has an initial entropy anomaly; the Lagrangian entropy (following fluid parcels) stays within $O(\Delta x^2)$ of the initial value throughout the bubble's rise. Test: `test_strang_step.cu` §C4-bubble-entropy.

Failure of (1) is an HLLC-dissipation issue (possibly with `use_lm_fix` interference). Failure of (2) is an HSE-preservation bug (usually §B3 or §C1). Failure of (3) would indicate a broken Lax condition — the HLLC flux is producing negative entropy at shocks, a sign of scheme instability or wrong wave-speed estimates. Failure of (4) is an entropy dissipation issue — the kernel is leaking entropy out of the bubble faster than truncation error predicts.

26 D1. Entropy-wave canonical IC

sympy script: `scripts/d01_entropy_wave.py` **generated LaTeX:** `output/d01_entropy_wave.latex.tex` **generated goldens:** `output/d01_entropy_wave.goldens.json` (per Rule 5, not committed; regenerated by `run_all.sh`) **verified:** - 4 Euler PDE component-wise (mass, x-mom, y-mom, energy) - 1 periodicity at $T = L_x/u_0$ - 1 HLLC contact-speed reduction ($S_\star = u_0$) - 4 upwind-flux form identities - 1 entropy-advection $D_t s = 0$

code checkpoints: - `src/gpu/explicit/strang_solver.cu` — new IC builder `init_entropy_wave()` to be added per §D1 - `tests/test_strang_convergence.cu` consumes golden JSON (§E1 quantifies the L1 convergence slope measurement)

The entropy wave is the simplest closed-form exact solution of the compressible Euler equations: a density perturbation advected by a uniform flow at constant pressure. It is the left eigenvector of $\mathcal{A}_x$ with eigenvalue u_0 (§A3): the advected mode carries no acoustic information. This makes it the canonical test for a shock-capturing scheme's **entropy-wave decoupling** — the numerical flux should transport density without generating spurious pressure or velocity perturbations.

26.1 IC ansatz

$$\begin{aligned} \rho(x,t) &= \rho_0 + A\,\sin(k(x-u_0\,t)), \\ u(x,t) &= u_0, \\ v(x,t) &= 0, \\ P(x,t) &= P_0, \end{aligned} \qquad \text{(D1-IC)}$$

with $\rho_0 > 0$, $P_0 > 0$, $|A| < \rho_0$ for positivity, and $kL_x = 2\pi m$ for m a positive integer (so the ansatz is periodic on $[0, L_x]$).

26.2 Strong-form PDE satisfaction

sympy verifies all four component equations of 2D Euler on this ansatz:

$$\begin{aligned} \partial_t\rho + \partial_x(\rho u) + \partial_y(\rho v) &= 0, \\ \partial_t m_x + \partial_x(\rho u^2 + P) + \partial_y(\rho uv) &= 0, \\ \partial_t m_y + \partial_x(\rho uv) + \partial_y(\rho v^2 + P) &= 0, \\ \partial_t E + \partial_x((E+P)u) + \partial_y((E+P)v) &= 0. \end{aligned}$$

Each reduces: since $u = u_0$ and $v = 0$ and $P = P_0$ are all constant in space-time, the nontrivial content is the mass equation $\partial_t\rho + u_0\partial_x\rho = 0$, which is exactly the wave equation $\rho_t + u_0\rho_x = 0$ satisfied by $\sin(k(x-u_0t))$.

26.3 Periodicity

At $t = T = L_x/u_0$, $k(x-u_0T) = kx - kL_x = kx - 2\pi m$ so $\rho(x,T) = \rho(x,0)$ exactly. The IC returns to itself after one period.

26.4 HLLC reduction (decoupling proof)

At every face the left and right states differ only in ρ; u, v, P are identical. The HLLC contact-wave speed §A8 reduces:

$$S_\star = \frac{0 + \rho_L u_0(S_L - u_0) - \rho_R u_0(S_R - u_0)}{\rho_L(S_L - u_0) - \rho_R(S_R - u_0)} = u_0. \quad \text{(D1-S-star)}$$

The pressure-jump numerator vanishes ($P_R - P_L = 0$); the remaining terms factor as $u_0 \times [\rho_L(S_L - u_0) - \rho_R(S_R - u_0)]/[\rho_L(S_L - u_0) - \rho_R(S_R - u_0)] = u_0$. **The contact speed is exactly the advection speed** — HLLC degenerates to pure upwind on the entropy wave, which is the desired behaviour.

Moreover, the upwind flux (for $u_0 > 0$) is $\mathbf{F}^\star = \mathbf{F}(\mathbf{U}_L)$, component-wise verified in sympy.

26.5 Entropy advection

The specific entropy $s = \log P - \gamma \log \rho$ has $\log P_0$ is constant, so $s(x,t) = \log P_0 - \gamma \log \rho(x,t)$. Since ρ is advected by u_0, s is advected by u_0 as well. $D_t s = 0$ (sympy verified directly).

26.6 Reference (golden) profile

The script dumps the canonical IC parameters and a reference profile at $N_{\text{ref}} = 4096$ grid points on $x \in [0, L_x]$:

parameter	value	role
ρ_0	1.0	background density
P_0	$1/\gamma$	s.t. $c_0 = 1$
u_0	1.0	advection velocity
A	0.05	wave amplitude
L_x	1.0	domain length
γ	1.4	ratio of specific heats
k	$2\pi/L_x$	wavenumber (one full wave)
N_{ref}	4096	golden grid density
T	$L_x/u_0 = 1.0$	one period
`rho_initial`	list of 4096 values	$\rho(x,0)$ samples
`rho_final_at_T`	list of 4096 values	$\rho(x,T) = \rho(x,0)$
`L1_expected_error_at_T`	0.0	analytic zero

At these parameters $M_{\text{loc}} = |u_0|/c_0 = 1.0$, so the LM-HLLC blend factor $f_M = 1$ (standard HLLC regime). This is the **standard convergence test IC** — the solver's L^1 error between its computed $\rho(x,T)$ and the golden `rho_final_at_T` is the convergence-rate measurement of §E1.

26.7 Measurement protocol

For each grid resolution $n_x \in \{64, 128, 256, 512\}$:

1. Initialise kernel with IC per §D1.
2. Evolve for time $T = L_x/u_0$.
3. Download $\rho(x, y, T)$ from device, take $\rho(x, y_{\text{mid}}, T)$ (arbitrary row, since $v = 0$ implies no y-variation).
4. Compute $L^1 = \sum_i |\rho_i(T) - \rho_i(0)| \cdot \Delta x$.
5. Fit $\log L^1$ vs $\log n_x$; slope should be ≈ -2 (2nd-order convergence).

Use `use_lm_fix = true` here: at $M = u_0/c_0 = 1$, the LM blend is inactive ($f_M = 1$), so the kernel behaves as standard HLLC. This is the convergence test's intent.

26.8 Verification checkpoints

1. **IC match.** After `init_entropy_wave()`, the solver's stored $(\rho, m_x, m_y, \delta E)$ after reconstruction to (ρ, u, v, P) matches `rho_initial, u_0, 0, P_0` at every cell to ULP precision. Test: `test_strang_init.cu` §D1-IC.
2. **Periodicity at T.** After $T/\Delta t$ steps (rounded), the $\rho(x, y, T)$ matches `rho_final_at_T` to truncation error. Test: `test_strang_convergence.cu` §D1-periodic.

3. **L1 convergence slope.** Slope measured at 4 resolutions is in $[1.8, 2.2]$ (§E1 predicts $p = 2.0$). Test: `test_strang_convergence.cu` §E1-slope.
4. **Pressure invariance.** $P(x, y, T) \equiv P_0$ to round-off accumulation (entropy wave should not excite pressure perturbation). Test: `test_strang_convergence.cu` §D1-P-inv.

Failure of (4) is a specific diagnostic: it means the HLLC solver is coupling the entropy mode to the acoustic modes — a bug in the upwinding of the contact wave, or incorrect treatment of the degenerate $\Delta P = 0$ case.

27 D2. Acoustic linear-wave canonical IC

sympy script: `scripts/d02_acoustic_linwave.py` **generated LaTeX:** `output/d02_acoustic_linwave.latex.tex` **generated goldens:** `output/d02_acoustic_linwave.goldens.json` **verified:** - 1 adiabatic relation ($\delta P = c_0^2 \delta\rho$) - 3 linearised PDE components (mass, x-momentum, pressure equation) - 1 $O(\epsilon)$ non-linear residual vanishing - 1 periodicity at $T = L_x/(u_0 + c_0)$ - 1 phase-speed documentation - 4 right-eigenvector projection identities

code checkpoints: - `src/gpu/explicit/strang_solver.cu` — new `init_linwave()` IC builder - `tests/test_strang_linwave_convergence.cu` must set `use_lm_fix = false` (golden JSON includes the flag)

A right-going acoustic wave from §A3's $(u + c)$ eigenvector, linearised around a stationary uniform background. This is the canonical test for a shock-capturing scheme's **acoustic convergence** — the scheme's pressure-diffusion truncation drives an amplitude decay that, at a known rate, sets the convergence slope for smooth acoustic flows.

Critically, this test requires `use_lm_fix = false`. At the amplitude $\epsilon = 10^{-6}$ chosen below, the local Mach $M = \epsilon$ is far below $M_{\text{cut}} = 10^{-3}$, so LM-HLLC (§C3) would reduce pressure dissipation by a factor $M/M_{\text{cut}} = 10^{-3}$, artificially super-converging the linwave and hiding the true HLLC behaviour. §E2 quantifies this.

27.1 IC ansatz

$$\begin{aligned} \delta\rho/\rho_0 &= \epsilon \sin\big(k(x - (u_0 + c_0)t)\big), \\ \delta u/c_0 &= \epsilon \sin\big(k(x - (u_0 + c_0)t)\big), \\ \delta v &= 0, \\ \delta P/(\gamma P_0) &= \epsilon \sin\big(k(x - (u_0 + c_0)t)\big), \end{aligned} \qquad \text{(D2-IC)}$$

with $c_0 = \sqrt{\gamma P_0/\rho_0}$ and phase $kL_x = 2\pi$.

27.2 Eigenvector projection

The amplitudes $(\rho_0, c_0, 0, \gamma P_0)^{\mathsf{T}}$ (in primitive (ρ, u, v, P) order) are **parallel** to the right eigenvector of $\mathcal{A}_x$ at eigenvalue $u_0 + c_0$ (§A3). The IC is a pure acoustic mode with no projection onto the entropy ($\lambda = u_0$), shear ($\lambda = u_0$), or left-acoustic ($\lambda = u_0 - c_0$) components.

27.3 Adiabatic relation

$$\delta P = c_0^2 \, \delta\rho, \quad \text{(D2-adiabatic)}$$

which is the standard acoustic wave relation: pressure and density perturbations are in phase, with proportionality c_0^2. sympy verifies this directly from the IC amplitudes.

27.4 Linearised PDE satisfaction

The linearised Euler system around $(\rho_0, u_0, 0, P_0)$:

$$\begin{aligned} \partial_t \delta\rho + \rho_0 \partial_x(\delta u) + u_0 \partial_x(\delta\rho) &= 0, \\ \rho_0 \partial_t(\delta u) + \rho_0 u_0 \partial_x(\delta u) + \partial_x(\delta P) &= 0, \\ \partial_t(\delta P) + u_0 \partial_x(\delta P) + \gamma P_0 \partial_x(\delta u) &= 0, \end{aligned}$$

each verified by sympy against the ansatz. These are satisfied exactly at all ϵ for the linear system.

27.5 Non-linear residual

Substituting the ansatz into the **full non-linear** Euler PDE and expanding in ϵ, the $O(\epsilon)$ coefficient of the mass residual vanishes (sympy verified). The $O(\epsilon^2)$ residual is non-zero — it represents acoustic self-steepening (a small-amplitude wave eventually steepens into a shock). For short evolution $t \lesssim 1/(k\epsilon c_0)$ (the non-linear breaking time) and small ϵ, this residual is negligible, so the ansatz is an excellent approximation to a true Euler solution.

At $\epsilon = 10^{-6}$, the breaking time $\sim 10^6/(kc_0)$ is astronomically long relative to the one-period evolution $T \sim 1/c_0$, so non-linear effects are invisible in the test.

27.6 Periodicity

At $t = T = L_x/(u_0 + c_0)$ the phase has advanced by $kL_x = 2\pi$, so all four perturbation fields return to their initial values. sympy verifies this directly.

27.7 HLLC response at this IC

Unlike the §D1 entropy wave, here the face states have non-zero pressure jump $\delta P_R - \delta P_L = O(\epsilon)$ at every interface. The HLLC pressure-dissipation term contributes a truncation error proportional to $\Delta x \cdot c \cdot \delta P$, giving the leading $O(\Delta x^2)$ amplitude decay and hence 2nd-order L^1 convergence. §E2 derives the exact expression for the numerical viscosity ν_{eff} and the resulting convergence rate.

27.8 LM-HLLC interaction (critical!)

With `use_lm_fix = true`, the blend factor $f_M = \max(M_{\text{local}}, M_{\text{cut}}, 1)$ becomes $f_M = M_{\text{cut}} = 10^{-3}$ at $M \leq M_{\text{cut}}$. This multiplies the pressure-jump dissipation by 10^{-3}, artificially amplifying the wave (relative to standard HLLC). The measured convergence rate would be super-2nd-order (the kernel's error becomes machine-order for most resolutions), which is **not** the standard HLLC convergence theory. See §E2 for the derivation.

For this test, `use_lm_fix = false` is mandatory. The golden JSON dumps `use_lm_fix: false` as a flag the test reads to configure the solver.

27.9 Golden values dump

parameter	value
ρ_0	1.0
P_0	$1/\gamma$
u_0	0.0 (stationary background)
c_0	1.0
ϵ	10^{-6}
L_x	1.0
γ	1.4
k	$2\pi/L_x$
N_{ref}	4096
T	$1/c_0 = 1.0$
`delta_rho_initial[4096]`	linear ansatz $\rho_0\epsilon\sin(kx)$
`delta_u_initial[4096]`	$c_0\epsilon\sin(kx)$
`delta_P_initial[4096]`	$\gamma P_0\epsilon\sin(kx)$
`delta_rho_final_at_T[4096]`	identical to initial
`L1_expected_error_at_T`	0.0
`use_lm_fix`	**false**

27.10 Measurement protocol

For $n_x \in \{64, 128, 256, 512\}$:

1. `init_linwave()` with the canonical parameters; `use_lm_fix = false`.
2. Evolve for $T = L_x/(u_0 + c_0) = 1.0$.
3. Download $\delta\rho(x, y, T)$.
4. Compute $L^1 = \sum_i |\delta\rho_i(T) - \delta\rho_i(0)|$.
5. Fit slope; expected $p \approx 2.0$ (2nd-order).

27.11 Verification checkpoints

1. **IC consistency.** After `init_linwave()`, the face-state reconstruction (§A11) yields $(\rho_0 + \delta\rho, u_0 + \delta u, 0, P_0 + \delta P)$ with the linear ansatz profile, to ULP precision. Test: `test_strang_linwave_convergence.cu` §D2-IC.
2. **LM-flag flag consistency.** The test configures the kernel with `use_lm_fix = false` as specified in golden JSON. Test: `test_strang_linwave_convergence.cu` reads flag -> sets solver.
3. **L1 convergence slope.** Slope over 4 resolutions in $[1.8, 2.2]$. Test: §E2 slope check.
4. **No growth.** The wave amplitude at T must not exceed the IC amplitude by more than 10^{-12} (numerical dispersion tolerance). Growth would indicate an unstable scheme. Test: `test_strang_linwave_convergence.cu` §D2-no-growth.

Failure of (2) is a test-configuration bug — confirm the test actually sets `use_lm_fix = false`. Failure of (3) with `use_lm_fix = true` would show super-2nd-order convergence (the wrong answer for this test — see §E2). Failure of (4) is a deep-solver bug; usually it means the scheme is producing negative dissipation on low-Mach acoustic perturbations, which is non-physical.

28 D3. Sod shock tube canonical IC

sympy script: `scripts/d03_sod_shock_tube.py` **generated LaTeX:** `output/d03_sod_shock_tube.latex.tex` **generated goldens:** `out-`

put/d03_sod_shock_tube.goldens.json **verified:** - 2 strong-form sympy identities (rarefaction and shock f-functions vanishing at $p = P_K$) - plus closed-form Newton solution for $p^\star$ at 15-digit precision

code checkpoints: - new `init_sod()` IC builder in `src/gpu/explicit/strang_solver.cu` - `tests/test_strang_sod.cu` (new test — wire into CMakeLists)

Sod's shock-tube (Sod 1978) is the canonical Riemann problem: the diaphragm breakup generates a left-going rarefaction, a contact discontinuity, and a right-going shock. The solution is closed-form (modulo a transcendental Newton iteration for $p^\star$), making it an excellent test for shock-capturing schemes' resolution of all three wave types simultaneously.

28.1 Initial condition

At $t = 0$, discontinuity at $x = 0$ on $x \in [-0.5, 0.5]$:

$$\begin{aligned} \text{Left:}\ & \rho_L = 1.0, \quad u_L = 0, \quad P_L = 1.0, \\ \text{Right:}\ & \rho_R = 0.125, \quad u_R = 0, \quad P_R = 0.1, \quad \text{(D3-IC)} \\ & \gamma = 1.4. \end{aligned}$$

28.2 Riemann fan structure

Starting from the $t > 0$ similarity solution (functions of $\xi = x/t$):

$$\text{L state} \to \text{rarefaction} \to \text{left star} \to \text{contact} \to \text{right star} \to \text{shock} \to \text{R state}. \quad \text{(D3-fan)}$$

Five intervals on ξ, separated by four wave speeds ($S_{HL} < S_{TL} < S_C < S_R$). The "star" region is the intermediate state between the contact and the adjacent wave (left rarefaction tail or right shock), sharing pressure $p^\star_L = p^\star_R = p^\star$ and velocity $u^\star_L = u^\star_R = u^\star$ per §A8.

28.3 Star-region equation

The pressure $p^\star$ satisfies the transcendental

$$u_L - u_R \;=\; f_L(p^\star; \rho_L, P_L) \;+\; f_R(p^\star; \rho_R, P_R), \quad \text{(D3-f)}$$

where each f_K selects rarefaction or shock depending on the sign of $p^\star - P_K$:

$$f_K(p) \;=\; \begin{cases} (p - P_K)\sqrt{\dfrac{A_K}{p + B_K}}, & p > P_K \text{ (shock)} \\ \dfrac{2c_K}{\gamma - 1}\left[(p/P_K)^{(\gamma-1)/(2\gamma)} - 1\right], & p \le P_K \text{ (rarefaction)} \end{cases}$$

with $A_K = 2/(\rho_K(\gamma + 1))$, $B_K = \frac{\gamma-1}{\gamma+1} P_K$.

For Sod, $p^\star < P_L$ (rarefaction left) and $p^\star > P_R$ (shock right).

28.4 Numerical solution

Newton's method on the residual $f_L(p^\star) + f_R(p^\star) - (u_L - u_R)$ (with $u_L = u_R = 0$) converges in ~ 10 iterations from the arithmetic-mean initial guess. The script computes all derived quantities to full double-precision:

quantity	value
$p^\star$	0.303 130 178 050 647
$u^\star$	0.927 452 620 048 950
$\rho_L^\star$	0.426 319 428 178 495
$\rho_R^\star$	0.265 573 711 705 307
S_{HL} (rarefaction head)	$-1.183\,215\,956\,619\,923$
S_{TL} (rarefaction tail)	$-0.070\,272\,812\,561\,183$
S_C (contact)	0.927 452 620 048 950
S_R (shock)	1.752 155 732 030 178

28.5 Reference profile

At $t = T = 0.2$, sampled at $N = 200$ uniform x-points on $[-0.5, 0.5]$. The sampling function handles each region:

- Inside the rarefaction fan: Riemann invariants give $c(\xi) = \frac{2}{\gamma+1}[c_L + \frac{\gamma-1}{2}(u_L - \xi)]$, $u(\xi) = \frac{2}{\gamma+1}[c_L + \frac{\gamma-1}{2}u_L + \xi]$, with $\rho = \rho_L(c/c_L)^{2/(\gamma-1)}$, $P = P_L(c/c_L)^{2\gamma/(\gamma-1)}$.
- Star regions: constant.
- Outside the fan / shock: IC states.

28.6 Verification identities (sympy)

Two closed-form identities are symbolically verified:

1. **f-function rarefaction zero:** $f_{\text{rar}}(P_K; c_K, P_K) = 0$. Trivial; sympy simplifies the $(P_K/P_K)^\alpha - 1 = 0$ term directly.
2. **f-function shock zero:** $f_{\text{shock}}(P_K; \rho_K, P_K) = 0$. By the $(p - P_K)$ factor.

These anchor the Newton iteration: at $p^\star = P_L$ or $p^\star = P_R$, one branch vanishes. For a **constant-pressure** Riemann problem ($P_L = P_R$, $u_L = u_R$ — the trivial case), the solution is $p^\star = P_L = P_R$ and both branches contribute zero, consistent with the trivial Riemann solution being the IC itself.

The **contact invariant** $p_L^\star = p_R^\star = p^\star$ is a §A8 strong-form identity, propagated without re-derivation.

28.7 Measurement protocol

1. Initialise kernel with the Sod IC at $n_x = 200$.
2. Evolve for $T = 0.2$.
3. Download $\rho, m_x, m_y, \delta E$; reconstruct (ρ, u, P).
4. Compare against golden `rho_profile, u_profile, P_profile` at x = cell centres.
5. Compute L^1 norm of the error; required: $L^1 < 10^{-2}$ at $n_x = 200$ (the scheme's typical Sod accuracy).

For **convergence test** mode, run at $n_x \in \{100, 200, 400, 800\}$ and fit slope; expected $p \approx 1.0$ (1st order through shocks and contacts — Godunov limit) with better rate on the rarefaction interior.

28.8 Verification checkpoints

1. **IC consistency.** After `init_sod()`, cells with $x_c < 0$ have $(\rho, u, v, P) = (1.0, 0, 0, 1.0)$ and cells with $x_c > 0$ have $(0.125, 0, 0, 0.1)$. Test: `test_strang_sod.cu` §D3-IC.
2. **Star-region numerical match.** Measure post-step $p^\star$ across the contact (middle of the star region); required $|p^\star_{\text{measured}} - p^\star_{\text{golden}}|/p^\star_{\text{golden}} < 0.05$ (5% tolerance for finite-n_x resolution). Test: `test_strang_sod.cu` §D3-star-match.
3. **Wave-speed tracking.** Measure the positions of the shock, contact, and rarefaction tail at $T = 0.2$; each must lie within 1.5 cells of the analytic position $S_\cdot \cdot T$. Test: `test_strang_sod.cu` §D3-wave-positions.
4. **Entropy monotonicity at shock.** Across the right shock the entropy $s = \log(P/\rho^\gamma)$ must strictly increase post-shock vs pre-shock; required $\Delta s > 0$ (§A5 Lax condition). Test: `test_strang_sod.cu` §D3-shock-entropy.

Failure of (1) is a direct IC bug. Failure of (2) usually means the HLLC middle-state formula (§A8) has a bug or the Davis wave speeds (§A9) are incorrect. Failure of (3) is a typical convergence-level issue — can be expected at $n_x = 100$ and lower, but a systematic shock-lag signal indicates a flux inconsistency. Failure of (4) is a serious Lax-condition violation; the scheme is producing negative entropy and is unstable.

29 D4. Woodward-Colella two-blast-wave interaction

sympy script: `scripts/d04_woodward_colella_blast.py` **generated LaTeX:** `output/d04_woodward_colella_blast.latex.tex` **generated goldens:** `output/d04_woodward_colella_blast.goldens.json` **verified:** - 2 closed-form Riemann-problem solutions (at $x = 0.1$ and $x = 0.9$, early-time window) via §D3's Newton routine - late-time profile is **[WEAK]** per Rule 4 (no closed form after shock-shock collision)

code checkpoints: - new `init_woodward_colella()` IC builder in `src/gpu/explicit/strang_solver.cu` - `tests/test_strang_wc_blast.cu` wired to golden JSON

The Woodward & Colella (1984) "two blast waves" test has three high-pressure / low-pressure / high-pressure regions separated by two diaphragms. Upon release, two shocks propagate inward into the low-pressure middle region, collide, reflect off the walls, and interact with each other in a highly nonlinear fashion. The test is a standard benchmark for:

1. **Shock-capturing accuracy at high pressure ratios** ($P_L/P_M = 10^5$).
2. **Robustness of Riemann solvers under wave collision** (two strong shocks meeting head-on).
3. **Boundary-reflection handling** (both outer boundaries reflect the trailing rarefaction and returning shocks).

29.1 Initial condition

On $x \in [0, 1]$ with reflective walls at both ends:

$$\begin{aligned} 0 \le x < 0.1 &: \ \rho = 1,\ u = 0,\ P = 1000 \\ 0.1 \le x < 0.9 &: \ \rho = 1,\ u = 0,\ P = 0.01 \\ 0.9 \le x \le 1.0 &: \ \rho = 1,\ u = 0,\ P = 100 \\ \gamma &= 1.4. \end{aligned} \qquad \text{(D4-IC)}$$

29.2 Early-time window: two independent Riemann problems

For $t < t_{\text{collision}} \approx 0.026$, the inward-moving shocks have not yet met and the two Riemann problems (at $x = 0.1$ and $x = 0.9$) evolve independently. Using §D3's Newton routine, the script computes both star-region states closed-form:

quantity	Left (x = 0.1, L-blast vs. M)	Right (x = 0.9, M vs. R-blast)
$p^\star$	460.89	46.10
$u^\star$	$+19.60$	-6.20
$\rho^\star_L$	0.575	5.992
$\rho^\star_R$	5.999	0.575
outward shock speed	$S_R = 23.52$ (rightward)	$S_L = -7.44$ (leftward)

Both are strong shocks (post-shock density compression $\sim 6\times$, close to the strong-shock limit of $(\gamma+1)/(\gamma-1) = 6$ for $\gamma = 1.4$). The post-shock velocities are high-Mach.

29.3 Collision time

Inward-moving shock speeds from above: $+23.52$ from the left blast, -7.44 from the right blast. They collide when their positions equalise:

$$0.1 + 23.52\,t \;=\; 0.9 + (-7.44)\,t \quad\Longrightarrow\quad t_{\text{collision}} \;\approx\; \frac{0.8}{30.96} \;\approx\; 0.026. \quad \text{(D4-t-coll)}$$

The test's standard final time is $T = 0.038$, which is **after** the collision — the test specifically probes the post-collision regime.

29.4 Late-time window: [WEAK]

After shock-shock collision, no closed-form analytic solution exists. The test validates by:

1. **High-resolution reference run.** Run the same IC on $n_x = 3200$ cells (far beyond the converged resolution) and treat this as "truth".
2. **L1-integrated comparison.** For lower-resolution runs ($n_x = 100, 200, 400, 800$), compute the L^1 norm of the density error against the $n_x = 3200$ reference. Expect convergence rate $p \sim 1$ (shocks dominate; 1st-order under Godunov-limit analysis).
3. **Feature preservation.** Visual inspection of the density profile at $T = 0.038$ should show:
   - A persistent high-density peak near $x \approx 0.65$ (post- collision contact discontinuity).
   - Multiple weaker shock fronts from the wave interactions.
   - The characteristic two-peak structure of the Woodward-Colella solution.

Per Rule 4, the late-time reference is **[WEAK]**: it is numerical, not symbolic. The goldens JSON marks this explicitly:

```
"WEAK_caveat": "Late-time t = 0.038 profile has NO closed-form
solution; reference comparison is against a high-resolution
(N >= 3200) run, L1 diff measured against lower-resolution
runs."
```

The early-window verification (closed-form Riemann solutions at $t \lesssim 0.025$) is strong-form and provides the pointwise test anchor.

29.5 Measurement protocol

Early window test (strong-form oracle, $t = 0.02$): - Run $n_x = 1000$. - Compare at $t = 0.02$ (before collision). Expected: two independent Riemann fans; star-region values match the closed- form predictions within 5%.

Late window test (weak-form, $t = 0.038$): - Run the reference at $n_x = 3200$. - Run test resolutions at $n_x \in \{100, 200, 400, 800\}$. - Compute L^1 norm of ρ against the reference. - Fit slope; expected $p \sim 0.8—1.1$ (somewhere between 1st- order for shocks and a sub-linear rate from wave-interaction complexity).

29.6 Verification checkpoints

1. **Early-window star-region match.** At $t = 0.02$, measured $p^\star$ at $x = 0.3$ (within the left blast's post-shock region) agrees with closed-form $p^\star \approx 461$ within 3%. Test: `test_strang_wc_blast.cu` §D4-early.
2. **Shock position tracking.** At $t = 0.02$, the left-blast's outgoing shock has reached $x \approx 0.1 + 23.52 \cdot 0.02 = 0.57$; tolerance ± 2 cells. Test: `test_strang_wc_blast.cu` §D4-shock-position.
3. **Late-window L1 convergence.** L^1 slope over four resolutions in $[0.7, 1.2]$. Test: `test_strang_wc_blast.cu` §D4-late-convergence.
4. **Entropy floor preservation.** No cell has $P \leq 0$ at any time during the simulation. Test: `test_strang_wc_blast.cu` §D4-positivity.
5. **Reflective-wall energy conservation.** $\int_V (E + \rho g y) dV$ (with $g = 0$ for this test) stays within $10\varepsilon_{\text{mach}} N$ of the initial value. Test: `test_strang_wc_blast.cu` §D4-energy-conservation.

Failure of (1) is a Riemann-solver bug (probably §A8 formula error). Failure of (2) is a scheme-speed bug (wrong wave-speed estimation in §A9). Failure of (3) with a slope far below 0.7 is a deeper convergence issue. Failure of (4) indicates positivity preservation is broken — the HLLC or floor logic has a bug triggered at high pressure ratios. Failure of (5) points to a wall-BC bug (§B5 reflective); the blast-wave geometry stresses this more than simpler tests.

30 D5. Bubble (entropy-boost) canonical IC

sympy script: `scripts/d05_bubble_init.py` **generated LaTeX:** `output/d05_bubble_init.latex.tex` **generated goldens:** `output/d05_bubble_init.goldens.json` **verified:** - 1 isentropic-to-ρ map ($\rho' = \bar{\rho} \exp(-\delta s/\gamma)$ at constant P) - 1 linearised density perturbation - 1 zero δE on static bubble IC

code checkpoints: - `src/gpu/explicit/strang_solver.cu :: StrangSolver::init_bubble` (line 765; kernel `k_strang_init_bubble` at line 708)

The bubble IC is the canonical convective test for the Strang kernel: a warm (positive entropy anomaly) "bubble" embedded in the HSE atmosphere, optionally modulated by an azimuthal mode. The bubble is initially at pressure equilibrium with the background, so the only initial perturbation is to density. Under gravity it becomes buoyant and rises, eventually developing Rayleigh-Taylor instability if the rise exceeds the atmosphere's local scale height.

30.1 IC ansatz

$$s(\mathbf{x}) \;=\; s_{\text{bg}} \;+\; \delta s \, \exp\!\left(-r^2/R_0^2\right) \left(1 + \epsilon \, \cos(k_\theta \theta)\right), \quad \text{(D5-IC)}$$

with $r = \sqrt{(x-x_0)^2 + (y-y_0)^2}$, $\theta = \text{atan2}(y-y_0, x-x_0)$, and the IC enforces **constant pressure** $P = \bar{p}(y)$ (equilibrium with HSE). The azimuthal mode with wavenumber k_θ and amplitude ϵ is optional; with $\epsilon = 0$ the bubble is purely radial.

30.2 Isentropic closure -> density perturbation

Given $s = \log(P\rho^{-\gamma})$ and $P = \bar{p}$ constant, ρ must adjust to the new entropy:

$$\rho(\mathbf{x}) \;=\; \left(\bar{p}/\exp(s)\right)^{1/\gamma} \;=\; \bar{\rho}(y)\,\exp\!\left(-\delta s_{\text{local}}/\gamma\right), \quad \text{(D5-rho)}$$

where $\delta s_{\text{local}} = s(\mathbf{x}) - s_{\text{bg}}$ is the local entropy excess. For $\delta s > 0$ (hot bubble), $\rho < \bar{\rho}$ (density deficit) — the hot gas is less dense at the same pressure, producing positive buoyancy.

30.3 Linear-order density perturbation

For small δs:

$$\frac{\delta\rho}{\bar{\rho}} \;\approx\; -\frac{\delta s_{\text{local}}}{\gamma} \;+\; O(\delta s^2). \quad \text{(D5-linear)}$$

At $\delta s = 0.5$, $\gamma = 1.4$, the linear prediction is $\delta\rho/\bar{\rho} \approx -0.357$; the full nonlinear value is $\exp(-0.5/1.4) - 1 \approx -0.300$, a 16% correction at this amplitude. Tests using strong bubbles ($\delta s \gtrsim 0.3$) must use the full nonlinear formula.

30.4 Zero energy perturbation

The stored δE decomposes as (§B1):

$$\delta E \;=\; (P - \bar{p})/(\gamma - 1) \;+\; \tfrac{1}{2}\rho(u^2 + v^2).$$

At IC, $P = \bar{p}$ (const-pressure bubble) and $u = v = 0$ (static), so both terms vanish:

$$\delta E\big|_{t=0} \;=\; 0. \quad \text{(D5-dE-zero)}$$

This is useful for testing: after `init_bubble()` the stored δE buffer must be bitwise-zero at every cell. Any non-zero value indicates an arithmetic error in the IC builder.

30.5 Kernel implementation

`k_strang_init_bubble` (line 708-752 of strang_solver.cu) computes:

```
// Simplified view:
double r2 = dx*dx + dy*dy;
double theta = atan2(dy, dx);
double local_ds = delta_s * exp(-r2 / (R0*R0)) *
                  (1 + epsilon * cos(k_theta * theta));
double rho_new = rho_bg * exp(-local_ds / gamma);
// Store perturbation:
d_rho[k] = rho_new - rho_bg;   // delta rho
d_mx[k]  = 0.0;
```

```
d_my[k]  = 0.0;
// Pressure-perturbation check:
// P = p_bg, so delta_E = 0 + 0 = 0.
d_E[k]   = 0.0;
```

The stored $\delta\rho$ can be negative (hot bubble has less density than background). The stored $\delta E = 0$ exactly.

30.6 Canonical parameters

param	value	role
x_0	0.5	bubble centre x
y_0	0.3	bubble centre y
R_0	0.1	bubble radius
δs	0.5	entropy boost
k_θ	3	azimuthal mode
ϵ	0.1	azimuthal amplitude

Background parameters (shared with HSE build): $\rho_0 = 1.0$, $K = 1.0$, $g = 1.0$, $L_y = 1.0$, $\gamma = 1.4$.

The atmosphere cut-off is at $y^\star = \gamma K \rho_0^{\gamma-1}/((\gamma-1)g) = 1.4/0.4 = 3.5$, so the bubble is well within the valid atmosphere.

30.7 Golden values

Golden JSON dumps:

field	purpose
scalar params	feed IC builder
`delta_rho_rel_ref_grid_64x64`	reference 2D array for test comparison
`delta_rho_rel_at_center`	closed-form $\exp(-\delta s/\gamma) - 1$ for sanity check

30.8 Verification checkpoints

1. **IC match.** After `init_bubble()` with canonical params on $n_x = n_y = 64$, the stored $\delta\rho/\bar{\rho}$ 2D array matches `delta_rho_rel_ref_grid_64x64` to ULP precision at every cell. Test: `test_strang_init.cu` §D5-profile.
2. $\delta E = 0$**.** All cells have stored $\delta E = 0$ to bitwise precision. Test: `test_strang_init.cu` §D5-dE-zero.
3. **Centre value.** At the cell containing the bubble centre, $\delta\rho/\bar{\rho} \approx \exp(-\delta s/\gamma) - 1 \approx -0.300$ to ULP precision. Test: `test_strang_init.cu` §D5-center.
4. **Bubble rise rate (downstream).** After $t = 0.2$ of evolution, the bubble centre has risen by approximately $\Delta y \approx \frac{1}{2} g t^2 |\delta\rho/\rho| \approx 0.2 \cdot 0.5 \cdot 1.0 \cdot 0.3 \approx 0.03$ (buoyancy- driven acceleration). Exact check depends on the scheme; this is a convergence-sanity rather than a strong-form check. Test: `test_strang_step.cu` §D5-rise.

Failure of (1) or (3) indicates an arithmetic bug in `k_strang_init_bubble`. Failure of (2) means the kernel is writing non-zero δE — usually a cons2prim / delta-E confusion in the IC code.

Failure of (4) is a coarser downstream check; typical failure modes include too-aggressive HSE dissipation (bubble dissolves before it rises) or over-strong entrainment (bubble fragments prematurely).

31 D6. HSE zero-perturbation lock

> **sympy script:** `scripts/d06_hse_zero_perturbation_lock.py` **generated LaTeX:** `output/d06_hse_zero_perturbation_lock.latex.tex` **generated goldens:** `output/d06_hse_zero_perturbation_lock.goldens.json` **verified:** - flux-source cancellation on HSE (y-momentum) - energy update zero on HSE - density update zero on HSE - x-momentum update zero on HSE
>
> **code checkpoints:** - whole kernel — this test verifies the **composite** of §B2, §B3, §B4, §B5, §B6, §C1. Any one of these failing breaks the lock. Specifically: init-time HSE build (`StrangSolver::init`), face HSE reconstruction (`k_muscl_hancock_y`), ghost-fill kernels (`k_ghost_x, k_ghost_y`), gravity source application (`k_hllc_update_y`).

The most stringent well-balancing test in the book. With stored state $(\delta\rho, m_x, m_y, \delta E) = (0,0,0,0)$ at every cell (pure HSE), the kernel must preserve this state to machine precision under arbitrary numbers of Strang steps. Any non-zero drift within $O(\varepsilon_{\text{mach}} N)$ indicates a composition failure among the HSE-related sections.

31.1 IC

$$(\delta\rho, m_x, m_y, \delta E)(\mathbf{x}, t=0) \;=\; (0,0,0,0) \quad \forall\, \mathbf{x} \in \text{domain.} \quad \text{(D6-IC)}$$

After `StrangSolver::init` builds the HSE background and before any perturbation is added, the storage buffer **is** the zero state. `init_bubble()` is **not** called for this test.

31.2 Strong-form balance

The stored $\mathbf{U}_{\text{store}} = \mathbf{0}$ is preserved because each component's update is identically zero on pure HSE:

component	flux divergence	source	sum
$\delta\rho$	$-\partial_x(\rho v) - \partial_y(\rho v) = 0$ (since $v = 0$)	0	**0**
m_x	$-\partial_x(\rho u^2 + P)$: $P = \bar{p}$ indep of x -> 0; $-\partial_y(\rho u v) = 0$	0	**0**
m_y	$-\partial_y(\rho v^2 + P) = -\partial_y \bar{p} = +\bar{\rho} g$	$-\bar{\rho} g$	**0**
δE	$-\partial_x((E+P)u) = 0$; $-\partial_y((E+P)v) = 0$	$-m_y g = 0$	**0**

The non-trivial cancellation is in the y-momentum: the background pressure gradient contributes $+\bar{\rho} g$ (via the HSE ODE $d\bar{p}/dy = -\bar{\rho} g$ from §B2), and the gravity source contributes $-\bar{\rho} g$. The two cancel exactly in the continuum limit, and to $O(\Delta y^2)$ under discretisation (§B3 + §C1 composite).

sympy verifies this cancellation as a symbolic identity:

$$\underbrace{-\frac{\Delta t}{\Delta y}\left[\bar{p}(y_{j+1/2}) - \bar{p}(y_{j-1/2})\right]}_{\text{flux divergence}} + \underbrace{-\Delta t\, \bar{\rho}_j\, g}_{\text{gravity source}} = 0. \quad \text{(D6-balance)}$$

31.3 Round-off drift

Under IEEE-754 double precision with standard (non-Kahan) accumulation, the stored state accumulates round-off linearly in the number of steps:

$$\|\mathbf{U}_{\text{store}}\|_\infty(N_{\text{step}}) \lesssim \varepsilon_{\text{mach}}\, N_{\text{step}}\, \kappa(\text{HSE}), \quad \text{(D6-drift)}$$

where $\kappa(\text{HSE}) \sim O(1)$ is a problem-dependent condition number (bounded by the ratio $\bar{p}_{\max}/\bar{p}_{\min}$ across the domain in the worst case; for the canonical HSE with $y^\star \gg L_y$ this is $O(1)$).

At $N = 1000$ steps and $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$, the expected drift bound is $\approx 2.2 \times 10^{-13}$. The kernel's `d_rho, d_mx, d_my, d_E` should remain within this bound.

If the kernel used **Kahan summation** at every accumulation site, the drift would be $\varepsilon_{\text{mach}}\sqrt{N}$, roughly 7×10^{-15} at $N = 1000$. The Strang kernel uses standard (non-Kahan) accumulators, so the linear bound applies.

31.4 Test tolerance

The golden JSON provides `comparison_tolerance = 1e-10`, much looser than both theoretical bounds. This should always pass; violations indicate structural issues (not round-off).

31.5 Composite dependencies

This test is the **acceptance criterion** for all of Part B + §C1:

If failed	Likely root cause
drift spikes at step 1	§B3 face HSE is wrong (cell-centred bg), or §C1 source sign wrong
drift grows super-linearly	§B4/B5/B6 ghost fill doesn't preserve zero state (non-zero ghost drives flux)
drift is $O(\bar{\rho}g)$ per step	§C1 source sign flipped (gravity adds, not cancels, flux div)
energy component drifts, y-mom stays at 0	§C1 energy source formula wrong ($-m_y g$ not applied)
drift is $O(\Delta y^2 \cdot \bar{\rho}g/c)$ per step	§B3 face HSE reconstruction is at cell-centred values (wrong face)

31.6 Golden values dump

Reference HSE profile at $N = 8192$ y-points for canonical parameters ($\rho_0 = 1, K = 1, g = 1, L_y = 1, \gamma = 1.4$). The test consumer reads `rho_bar_profile, p_bar_profile` and compares against the kernel's `d_rho_bar, d_p_bar` to ULP precision before running any Strang steps (separate §B2-level check).

31.7 Measurement protocol

1. Initialise kernel with canonical HSE parameters.
2. Confirm `d_rho, d_mx, d_my, d_E` buffers are all-zero.
3. Confirm `d_rho_bar, d_p_bar` match the reference profile to ULP precision.
4. Take $N_{\text{step}} = 1000$ Strang steps at default CFL ($\sigma = 0.4$).
5. Download `d_rho, d_mx, d_my, d_E`; compute the infinity norm.
6. Required: max-norm drift $\leq 10^{-10}$.

31.8 Verification checkpoints

1. **Initial zero state.** After `init()`, the stored state is bitwise-zero. Test: `test_strang_init.cu` §D6-init-zero.
2. **Single-step drift.** After one Strang step on HSE IC, the max-norm drift is $\leq 10\varepsilon_{\text{mach}}\bar{\rho}_{\max}$. Test: `test_strang_step.cu` §D6-one-step.
3. **Long-time drift.** After $N = 1000$ Strang steps, drift $\leq 10^{-10}$. Test: `test_strang_step.cu` §D6-long-time.
4. **HSE background self-consistency.** The reference-profile `rho_bar_profile, p_bar_profile` matches the kernel's HSE background to ULP precision. Test: `test_strang_init.cu` §D6-hse-match.

Failure of (2) is a composition bug among §B2-§C1. Use the failure diagnostics above to triage. Failure of (3) with linear-in-N drift is expected (round-off); only super-linear or constant-offset drift indicates a bug. Failure of (4) means the reference JSON is stale (regenerate via `bash run_all.sh`) or the kernel's HSE builder parameters don't match the canonical.

32 D7. Reflection-symmetric IC (bit-reproducibility test)

sympy script: `scripts/d07_reflection_symmetric_ic.py` **generated LaTeX:** `output/d07_reflection_symmetric_ic.latex.tex` **generated goldens:** `output/d07_reflection_symmetric_ic.goldens.json` **verified:** - 1 involution ($\mathcal{R}_x^2 = \mathbf{I}$) - 4 $\mathbf{F}_x$ reflection identities - 4 $\mathbf{F}_y$ reflection identities - 4 gravity-source invariance identities (all under x-reflection)

code checkpoints: - new `init_rt_symmetric()` IC builder in `src/gpu/explicit/strang_solver.cu` (book-anchored: the book says this IC must exist) - new test `tests/test_strang_reflection_symmetry.cu`

An **x-reflection-symmetric** IC must evolve into an x-reflection- symmetric state at all times. The test: start with $\mathbf{U}(x, y) = \mathcal{R}_x\mathbf{U}(-x, y)$, evolve, check that $\mathbf{U}(x, y, t) - \mathcal{R}_x\mathbf{U}(-x, y, t)$ stays at $O(\varepsilon_{\text{mach}}N)$ for any N. If the kernel's operator chain is **asymmetric** in any way (e.g., Riemann solver uses L/R asymmetrically, HSE build has x-dependent round-off, ghost-cell fill order introduces drift), this test will reveal the asymmetry as a non-zero symmetry residual.

This is the cleanest **bit-reproducibility** test for a 2D shock-capturing kernel: any deviation is a structural bug, not a physical phenomenon.

32.1 Choice of reflection axis

Gravity in the Strang kernel points in the $-y$ direction. A y-reflection would change the sign of gravity, breaking the symmetry by the gravity source term. But an x**-reflection** keeps gravity unchanged (it acts only on y), so the x-reflection is compatible with the gravity-driven HSE atmosphere. This is the axis chosen for §D7.

32.2 x-reflection matrix

Under $x \to -x$: $u \to -u$, $m_x \to -m_x$, all other components unchanged. On the conservative-state vector:

$$\mathcal{R}_x \;=\; \mathrm{diag}(+1,\,-1,\,+1,\,+1). \quad \text{(D7-R-x)}$$

$\mathcal{R}_x^2 = \mathbf{I}$ (involution, sympy verified).

32.3 Flux reflection identities

$$\mathbf{F}_x(\mathcal{R}_x\mathbf{U}) \;=\; \mathrm{diag}(-1,+1,-1,-1)\,\mathbf{F}_x(\mathbf{U}), \quad \text{(D7-flux-x)}$$

$$\mathbf{F}_y(\mathcal{R}_x\mathbf{U}) \;=\; \mathcal{R}_x\,\mathbf{F}_y(\mathbf{U}). \quad \text{(D7-flux-y)}$$

The $\mathbf{F}_x$ identity has sign pattern $(-1,+1,-1,-1)$ because three of the four $\mathbf{F}_x$ components contain u linearly (mass, x-mom-diagonal absent, xy-mom, energy), and the middle component $\rho u^2 + P$ is even in u.

The $\mathbf{F}_y$ identity is trivial: $\mathbf{F}_y$ has a u factor only in the x-momentum component (ρuv), so under $u \to -u$ only that component flips — exactly matching $\mathcal{R}_x$.

Both identities are sympy-verified component-wise.

32.4 Gravity source invariance

$$\mathcal{R}_x\mathbf{S} \;=\; \mathbf{S}. \quad \text{(D7-source)}$$

$\mathbf{S} = (0, 0, -\rho g, -m_y g)$ has zero in the m_x-component (the only place $\mathcal{R}_x$ has a sign flip), and the m_y component $-m_y g$ is unchanged under $m_y \mapsto m_y$ (which is the $+1$ entry of $\mathcal{R}_x$). The gravity source is x-reflection invariant.

32.5 Symmetry preservation

Combining:

1. **Initial symmetry.** $\mathbf{U}(x, y, 0) = \mathcal{R}_x\mathbf{U}(-x, y, 0)$ by IC construction.
2. **x-sweep preserves x-symmetry.** The x-sweep's Riemann problems at x and $-x$ are related by $(\mathbf{U}_L(x), \mathbf{U}_R(x)) = (\mathcal{R}_x\mathbf{U}_R(-x), \mathcal{R}_x\mathbf{U}_L(-x))$ (the L-R roles are swapped by the x-flip), and the HLLC flux respects this by §A8 symmetry.
3. **y-sweep preserves x-symmetry.** At each x, the y-sweep operates row-wise and does not mix x-positions. x-symmetry is preserved independently at each x.
4. **Gravity preserves x-symmetry.** By the source invariance above.

Therefore

$$\mathbf{U}(x, y, 0) \;=\; \mathcal{R}_x\mathbf{U}(-x, y, 0) \;\Longrightarrow\; \mathbf{U}(x, y, t) \;=\; \mathcal{R}_x\mathbf{U}(-x, y, t) \quad \forall\, t. \quad \text{(D7-preserve)}$$

This should hold to $O(\varepsilon_{\text{mach}} N)$ for the kernel (where N is the step count), since the only sources of symmetry-breaking are floating-point round-off errors, which accumulate linearly.

32.6 Canonical IC

Two symmetric bubbles at positions $(0.3, 0.3)$ and $(0.7, 0.3)$, mirror-image across $x = 0.5$:

- Bubble 1: $(x_0, y_0, R_0, \delta s) = (0.3, 0.3, 0.1, 0.5)$.
- Bubble 2: $(x_0, y_0, R_0, \delta s) = (0.7, 0.3, 0.1, 0.5)$.
- HSE background, gravity, etc. as in §D5 canonical.

At $t = 0$: $\rho(0.3, 0.3)$ and $\rho(0.7, 0.3)$ are bitwise equal (same bubble, mirrored). At the reflection axis $x = 0.5$, $u(0.5, y, 0) = 0$ by symmetry.

32.7 Test procedure

1. Run `init_rt_symmetric()` (NEW IC builder — book-anchored per user rule) with the canonical parameters.
2. Evolve for $T = 0.1$ (roughly 50 steps at CFL 0.4).
3. Download state; compare $\mathbf{U}(x, y, T)$ with $\mathcal{R}_x\mathbf{U}(1 - x, y, T)$ cell-by-cell.
4. Required: max residual $\leq 10^{-12}$ (much stricter than the $\varepsilon_{\text{mach}} N \approx 10^{-14}$ upper bound; slack accounts for cumulative round-off through HLLC and MUSCL).

32.8 Solver changes needed

Per the user rule "book is the anchor, solver follows": the kernel currently does not have `init_rt_symmetric()` wired up. This IC builder must be added to `strang_solver.cu` to satisfy §D7's test coverage. The book drives this addition — not a solver-first design decision.

Signature:

```
void StrangSolver::init_rt_symmetric(
    double x0_L, double x0_R,   // bubble centers (mirrored)
    double y0,                  // common y
    double R0,                  // common radius
    double delta_s              // common entropy boost
);
```

Both bubbles use the same $y_0, R_0, \delta s$; only the x_0 differs. The asserted symmetry is $x_0^L + x_0^R = L_x$.

32.9 Verification checkpoints

1. **IC symmetry.** After `init_rt_symmetric(0.3, 0.7, 0.3, 0.1, 0.5)`, $\delta\rho(x, y) - \delta\rho(1 - x, y)$ is zero to ULP precision at every cell pair. Test: `test_strang_reflection_symmetry.cu` §D7-IC-sym.
2. **x-flip of m_x on IC.** The initial m_x is zero on symmetric IC (static bubbles, no flow), so $m_x(x, y) + m_x(1 - x, y) = 0$ trivially. After one step, m_x is small but $m_x(x) + m_x(1 - x)$ stays at $O(\varepsilon_{\text{mach}})$. Test: §D7-x-mom-antisymmetry.
3. **Long-time symmetry.** After 50 steps, max $(\mathbf{U}(x, y, t) - \mathcal{R}_x\mathbf{U}(1 - x, y, t))$ stays $\leq 10^{-12}$. Test: §D7-long-time.
4. **Azimuthal mode preservation.** If the IC includes a single bubble with azimuthal perturbation $\cos(k\theta)$ for even k (e.g., $k = 2$), the x-reflection of this perturbation is itself, so the symmetry is exactly preserved. Test: §D7-azim-mode (extension).

Failure of (1) is an IC builder bug. Failure of (2) with non- zero drift in m_x on the IC would indicate the solver is producing asymmetric momenta from symmetric IC — HLLC L/R bias.

Failure of (3) with steady linear drift in the symmetry residual means the kernel has a structural asymmetry (most likely in the ghost-cell fill order or Riemann-solver L/R handling). Failure of (4) is a special test for the azimuthal IC; if (1)-(3) pass but (4) fails, there is a specific angular-mode bug.

33 E1. Entropy-wave convergence order prediction

sympy script: `scripts/e01_entropy_wave_order.py` **generated LaTeX:** `output/e01_entropy_wave_order.latex.tex` **verified:** - 1 at-least-1st-order (per-step $O(h)$ coefficient = 0) - 1 at-least-2nd-order ($O(h^2)$ coefficient = 0) - 1 leading $O(h^3)$ coefficient identity ($\nu(2\nu^2-3\nu+1)/12 \cdot u_{xxx}$) - 3 "magic CFL" identities (error vanishes at $\nu = 0, 1/2, 1$)

code checkpoints: - entire MUSCL-Hancock + MC + HLLC x-sweep pipeline - measured by `tests/test_strang_convergence.cu` against §D1 goldens

Modified-equation analysis of the MUSCL-Hancock + MC + HLLC scheme applied to the smooth entropy-wave IC of §D1. Strong-form Taylor expansion of the discrete update reveals the **per-step** leading truncation at $O(\Delta x^3)$, giving a **global** L^1 error scaling of $O(\Delta x^2)$ after $N \sim 1/\Delta x$ steps. The predicted convergence slope is $p = 2.0$.

33.1 Discrete update

On the linear advection $u_t + u_0 u_x = 0$ (which the entropy wave obeys):

$$u_j^{n+1} \;=\; u_j^n \;-\; \nu\,(u^L_{j+1/2} - u^L_{j-1/2}), \qquad \nu \;=\; \frac{u_0\,\Delta t}{\Delta x}. \quad \text{(E1-update)}$$

The face states $u^L_{j\pm1/2}$ come from MUSCL reconstruction (§A11) plus Hancock half-step (§A12):

$$u^L_{j+1/2} \;=\; u_j \;+\; \tfrac{1}{2}(\Delta x - u_0\,\Delta t)\,\sigma_j, \qquad \sigma_j \;=\; \frac{u_{j+1} - u_{j-1}}{2\,\Delta x}, \quad \text{(E1-face)}$$

where the MC limiter (§A10) reduces to the central difference on smooth data. HLLC on the entropy wave gives pure upwind flux (§D1), so the discrete update is exactly the formula above.

33.2 Taylor expansion

Expand $u(x_{j\pm1}, t_n)$ around (x_j, t_n) to $O(\Delta x^5)$, and $u(x_j, t^{n+1})$ via the exact advection $u(x, t+\Delta t) = u(x - u_0\Delta t, t)$. The residual $u_j^{n+1} - u_{\text{exact}}(x_j, t^{n+1})$, written with $\Delta t = \nu\Delta x/u_0$ so that all terms are in Δx:

- $O(\Delta x^1)$ coefficient: **0** (sympy verified)
- $O(\Delta x^2)$ coefficient: **0** (sympy verified)
- $O(\Delta x^3)$ coefficient: $\dfrac{\nu(2\nu^2-3\nu+1)}{12}\,u_{xxx}$

33.3 Leading truncation (strong form)

$$u_j^{n+1} \;-\; u_{\text{exact}}(x_j, t^{n+1}) \;=\; \frac{\nu(2\nu^2 - 3\nu + 1)}{12}\,\Delta x^3\, u_{xxx}(x_j, t_n) \;+\; O(\Delta x^4). \quad \text{(E1-trunc)}$$

sympy verifies this coefficient exactly. The factor $\nu(2\nu^2 - 3\nu + 1) = \nu(2\nu - 1)(\nu - 1)$ has three roots at $\nu = 0, 1/2, 1$, so the leading truncation **vanishes** at these CFL numbers:

- $\nu = 0$: trivial (no time advance, no error).
- $\nu = 1/2$: "magic" CFL where the MUSCL-Hancock face state lands exactly at the half-step upwind interpolant.
- $\nu = 1$: courant-number-1 exact advection (the scheme is exact for linear advection at $\nu = 1$, which is a classical Warming-Beam / Lax-Wendroff property).

For the kernel's $\sigma = 0.4$, $\nu = 0.4$, the coefficient is $0.4 \cdot (0.32 - 1.2 + 1)/12 = 0.4 \cdot 0.12/12 = 0.004$ — small but non-zero.

33.4 Global convergence

Per-step error is $O(\Delta x^3)$. Number of steps to reach a fixed time T is $N = T/\Delta t = Tu_0/(\nu \Delta x)$, which is $O(1/\Delta x)$. The global error is

$$\|u_{\text{num}} - u_{\text{exact}}\|_{L^1} \;\sim\; N \cdot \Delta x^3 \;=\; O(\Delta x^2). \quad \text{(E1-slope)}$$

So the predicted convergence slope is

$$p \;=\; 2.0 \pm 0.1. \quad \text{(E1-pred)}$$

33.5 Strang-split compatibility

The §D1 entropy wave has $v = 0$ uniformly. The y-sweep's HLLC Riemann problems see $v_L = v_R = 0$, which gives identical L/R states (after accounting for §B3 HSE background, which doesn't enter a no-gravity, constant-P test). The y-sweep contributes exactly zero update. Therefore Strang-split entropy-wave convergence = 1D entropy-wave convergence, giving the same $p = 2.0$.

33.6 Measurement protocol

For $n_x \in \{64, 128, 256, 512\}$:

1. IC from §D1 at `N_ref = 4096` (interpolated / averaged down to n_x).
2. Run to $T = L_x/u_0 = 1$ with `use_lm_fix = true` ($M_{\text{loc}} = 1$ so $f_M = 1$ anyway).
3. Download $\rho(x, y, T)$, project to 1D along a row.
4. Compute $L^1 = \sum_i |\rho_i(T) - \rho_i(0)|\Delta x$.
5. Fit $\log L^1$ vs $\log n_x$; slope expected in $[1.8, 2.2]$.

33.7 Verification checkpoints

1. **Slope match.** $p \in [1.8, 2.2]$ at the four resolutions. Test: `test_strang_convergence.cu` §E1-slope.

2. **Magic CFL.** Running at $\nu = 0.5$ (setting `cfl_number` so that the tightest cell has $\nu = 0.5$) should give L^1 error lower than at $\nu = 0.4$ by a factor of $\sim (0.4 \cdot 0.12)/(0.5 \cdot 0) —$ i.e., orders of magnitude lower (ideally machine precision). Test: `test_strang_convergence.cu` §E1-magic.
3. **No y-variation.** At the measurement time T, the kernel's $\rho(x, y, T)$ should be independent of y (to round-off precision). Failure indicates the y-sweep is contaminating the 1D entropy-wave test. Test: §E1-1D-check.

Failure of (1) with slope below 1.8 means the scheme is effectively 1st-order (e.g., the limiter is clamping in smooth regions, which would be an MC-limiter bug). Failure of (2) is an exotic feature test; it confirms the modified-equation analysis is correct. Failure of (3) indicates a structural y-sweep bug active even at $v = 0$.

34 E2. Linwave convergence under LM-HLLC vs standard HLLC

sympy script: `scripts/e02_linwave_lm_hllc_order.py` **generated LaTeX:** `output/e02_linwave_lm_hllc_order.latex.tex` **verified:** - log-decay-ratio $\log(\mathcal{A}_{\text{std}}/\mathcal{A}_{\text{LM}}) = -(1 - M_{\text{cut}})ck^2\Delta xT/2$

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (f_M branch versus `f_M = 1` branch); §D2 linwave test; `tests/test_strang_linwave_convergence.cu`

Modified-equation dispersion analysis of the MUSCL-HLLC scheme on the §D2 acoustic linwave. Predicts the numerical viscosity ν_{eff} and the resulting amplitude decay rate under two configurations: (a) `use_lm_fix = false` (standard HLLC), and (b) `use_lm_fix = true` (LM-HLLC at Mach below M_{cut}).

Key conclusion: for a correct acoustic convergence measurement, `use_lm_fix = false` is mandatory. LM-HLLC at the linwave's low $M \approx \epsilon \ll M_{\text{cut}} = 10^{-3}$ gives $f_M = M_{\text{cut}}$, suppressing the pressure dissipation by $10^3\times$ and artificially super-converging the test. The standard HLLC behaviour (2nd-order in Δx) requires $f_M = 1$, which `use_lm_fix = false` enforces.

34.1 Dispersion relation

The linearised MUSCL-HLLC update on a right-going acoustic mode gives the discrete dispersion

$$\omega(k) \;=\; c\,k \;-\; i\,\nu_{\text{eff}}\,k^2 \;+\; O(k^3), \quad \text{(E2-dispersion)}$$

where c is the sound speed and the imaginary part gives the amplitude decay rate. The effective viscosity is

$$\nu_{\text{eff}} \;=\; f_M \cdot \frac{c\,\Delta x}{2}, \quad \text{(E2-nu-eff)}$$

proportional to the HLLC pressure-dissipation coefficient. The factor $c\Delta x/2$ is the standard Godunov scheme's numerical viscosity. The factor f_M (the LM-HLLC blend, §C3) scales the pressure-jump contribution to the HLLC flux.

34.2 Per-period amplitude decay

Over one wave period $T = L_x/(u_0 + c)$ (which is $T = L_x/c$ for the canonical stationary background $u_0 = 0$), the amplitude decays as

$$\frac{\mathcal{A}(T)}{\mathcal{A}(0)} \;=\; \exp(-\nu_{\text{eff}}\, k^2\, T) \;=\; \exp(-f_M\, c\, k^2\, \Delta x\, T/2). \quad \text{(E2-decay)}$$

At the canonical $k = 2\pi/L_x$, $L_x = 1$, $c = 1$, $T = 1$, $\Delta x = 1/64$:

- **Standard HLLC** ($f_M = 1$): $\log(\mathcal{A}/\mathcal{A}_0) = -\pi^2/64 \approx -0.154$; retention factor ≈ 0.857 (15% loss).
- **LM-HLLC** ($f_M = M_{\text{cut}} = 10^{-3}$): $\log(\mathcal{A}/\mathcal{A}_0) = -10^{-3}\pi^2/64 \approx -1.5 \times 10^{-4}$; retention factor ≈ 0.99985 (0.015% loss).

The LM-HLLC case retains the amplitude almost perfectly — far better than the kernel's truncation-error floor allows at $\Delta x = 1/64$. The L^1 error is dominated by **higher-order** dispersive terms, not the ν_{eff} pressure dissipation.

34.3 Convergence rate

L^1 error $\sim 1 - \mathcal{A}(T)/\mathcal{A}_0$:

- Standard HLLC: $\sim \nu_{\text{eff}} k^2 T = f_M c k^2 \Delta x T/2$. Linear in Δx.
- After modified-equation expansion to $O(\Delta x^3)$ dispersion (the scheme's actual leading per-step error), the global L^1 is $O(\Delta x^2)$.

Standard HLLC predicted slope: $p = 2.0$**.**

Under LM-HLLC at low Mach:

- The pressure-dissipation term is suppressed by $f_M = M_{\text{cut}}/1$ (when $M \le M_{\text{cut}}$), making the ν_{eff} contribution to L^1 negligibly small.
- The L^1 error is dominated by **other** truncation sources (higher-order dispersion, entropy coupling, etc.) which at low amplitude become machine-precision-bounded.
- The measured slope can be **super-linear** (the L^1 error reaches the floor quickly and then plateaus at $\varepsilon_{\text{mach}}$).

LM-HLLC predicted slope: $p > 2$ **(not matching standard theory).**

34.4 Decay ratio

$$\log\left(\frac{\mathcal{A}_{\text{std}}(T)}{\mathcal{A}_{\text{LM}}(T)}\right) \;=\; -(1 - M_{\text{cut}})\,\frac{c\, k^2\, \Delta x\, T}{2}, \quad \text{(E2-ratio)}$$

sympy-verified. At the canonical parameters above, the ratio is $\exp(-0.308) \approx 0.735$: LM-HLLC retains amplitude $\sim 35\%$ more than standard HLLC over one period. This is a large, easily- measurable effect that distinguishes the two modes.

34.5 Implication for §D2 linwave test

The golden JSON (§D2) dumps `use_lm_fix: false`. The test must read this flag and configure the solver accordingly. If the test accidentally runs with `use_lm_fix = true`:

- The measured L^1 error at $n_x = 64, 128, 256, 512$ will show **super-linear** slope (e.g., $p = 2.5$ or higher) — which looks "better" than theory but is a diagnostic of the wrong configuration.
- The measured amplitude retention at T will be ~ 0.9999, much higher than standard HLLC's 0.857.

Both signals indicate a configuration error, not a solver bug. The regression test should both (a) measure the slope and check it's in $[1.8, 2.2]$ and (b) measure the amplitude retention and compare to §E2's prediction.

34.6 Verification checkpoints

1. **Standard HLLC slope.** With `use_lm_fix = false`, $p \in [1.8, 2.2]$ over four resolutions. Test: `test_strang_linwave_convergence.cu` §E2-std-slope.
2. **LM-HLLC super-convergence.** With `use_lm_fix = true`, $p > 2.2$ (tests confirm the expected super-linear behaviour). Test: §E2-LM-slope (diagnostic; not a pass/fail on p, but assert $p > 2.2$ or the amplitude retention > 0.99).
3. **Amplitude retention match.** With `use_lm_fix = false` at $n_x = 64$, $T = 1$, the measured amplitude retention is 0.857 ± 0.05 (matching the linear theory). Test: §E2-std-amplitude.
4. **Decay-ratio match.** Computing the ratio of amplitude losses between the two configurations gives ≈ 0.735 (matching the $\exp(-0.308)$ prediction). Test: §E2-ratio.

Failure of (1) with $p < 1.8$ would indicate a scheme-order-degradation bug (the convergence is weaker than 2nd-order, suggesting limiter clamping or flux asymmetry). Failure of (2) with $p \in [1.8, 2.2]$ means the LM fix is not activating — the kernel might be using `fM = 1` regardless of `use_lm_fix = true`. Failures (3, 4) are numeric cross-checks of the theoretical prediction.

35 E3. LM-HLLC effective numerical viscosity

sympy script: `scripts/e03_lm_hllc_nu_eff.py` **generated LaTeX:** `output/e03_lm_hllc_nu_eff.latex.tex` **verified:** - 1 ν_{eff} ratio ($\nu_{\text{LM}}/\nu_{\text{std}} = M$) - 1 Re_{eff} under LM ($= 2NM_{\text{conv}}/M_{\text{loc}}$) - 1 under standard HLLC ($= 2NM_{\text{conv}}$) - 1 Re-ratio ($\text{Re}_{\text{LM}}/\text{Re}_{\text{std}} = 1/M_{\text{loc}}$) - 1 clamped-regime identity

code checkpoints: - `src/gpu/explicit/strang_device.cuh :: d_lmhllc` (f_M clamp logic, lines 115-122); measurement via a dedicated scheme- characterisation test at Andrassy-style low-Mach convection parameters

Quantifies the practical advantage of LM-HLLC over standard HLLC for low-Mach convective flows — specifically, the **effective Reynolds number** the scheme supports at a fixed grid resolution. Standard HLLC's pressure-jump dissipation dominates at low Mach, restricting the effective viscosity to $\sim c\Delta x$; LM-HLLC reduces this to $\sim M_{\text{loc}} c\Delta x$, improving Re_{eff} by a factor $1/M_{\text{loc}}$.

35.1 Effective numerical viscosity

From §E2's dispersion analysis:

$$\nu_{\text{eff}}^{\text{LM}} \;=\; M_{\text{loc}}\,\frac{c\,\Delta x}{2}, \qquad \nu_{\text{eff}}^{\text{std}} \;=\; \frac{c\,\Delta x}{2}. \quad \text{(E3-nu-eff)}$$

The ratio $\nu_{\mathrm{LM}}/\nu_{\mathrm{std}} = M_{\mathrm{loc}}$ is the fundamental LM-HLLC advantage: the pressure-dissipation strength is reduced to the physical Mach level, matching what the continuum flow actually contains.

35.2 Effective Reynolds number

For a convective flow at speed u_{conv} across domain L:

$$\mathrm{Re}_{\mathrm{eff}} \;=\; \frac{L\,u_{\mathrm{conv}}}{\nu_{\mathrm{eff}}}, \qquad N \;=\; L/\Delta x, \;\; M_{\mathrm{conv}} \;=\; u_{\mathrm{conv}}/c.$$

Standard HLLC:

$$\mathrm{Re}_{\mathrm{eff}}^{\mathrm{std}} \;=\; 2\,N\,M_{\mathrm{conv}}.$$

LM-HLLC (active regime, $M > M_{\mathrm{cut}}$):

$$\mathrm{Re}_{\mathrm{eff}}^{\mathrm{LM}} \;=\; 2\,N\,\frac{M_{\mathrm{conv}}}{M_{\mathrm{loc}}}. \quad (\text{E3-Re-eff})$$

The LM advantage:

$$\frac{\mathrm{Re}_{\mathrm{eff}}^{\mathrm{LM}}}{\mathrm{Re}_{\mathrm{eff}}^{\mathrm{std}}} \;=\; \frac{1}{M_{\mathrm{loc}}}. \quad (\text{E3-advantage})$$

At the canonical Andrassy stratified-convection ambient $M_{\mathrm{loc}} \sim 10^{-3}$, this gives a $1000\times$ Reynolds-number boost — the LM fix is the **reason** one can simulate meaningful turbulence in the low-Mach regime at affordable resolutions.

35.3 Clamped regime ($M < M_{\mathrm{cut}}$)

When the local Mach drops below $M_{\mathrm{cut}} = 10^{-3}$, the blend factor f_M clamps at M_{cut}:

$$\nu_{\mathrm{eff}}^{\mathrm{LM,\,clamp}} \;=\; M_{\mathrm{cut}}\,\frac{c\,\Delta x}{2}, \qquad \mathrm{Re}_{\mathrm{eff}}^{\mathrm{LM,\,clamp}} \;=\; 2\,N\,\frac{M_{\mathrm{conv}}}{M_{\mathrm{cut}}}. \quad (\text{E3-clamp})$$

At $M_{\mathrm{loc}} = M_{\mathrm{cut}} = 10^{-3}$, the clamp is not yet active and the regime is a smooth transition. Below M_{cut}, the LM reduction saturates — Reynolds number grows no further as M_{loc} decreases. This is by design: M_{cut} provides a stability floor to prevent the scheme from going negative-viscosity at vanishing Mach.

35.4 Numerical example

At $n_x = 512$, $L = 1$, $c = 1$, $u_{\mathrm{conv}} = M_{\mathrm{conv}}c$ with $M_{\mathrm{conv}} = M_{\mathrm{loc}} = 10^{-2}$ (typical for stellar convection at the top of the convection zone):

scheme	ν_{eff}	$\mathrm{Re}_{\mathrm{eff}}$	improvement
standard HLLC	5×10^{-4}	10.24	1
LM-HLLC	5×10^{-6}	1024.00	$100\times$

The LM-HLLC scheme supports $100\times$ higher effective Reynolds at the same grid. For turbulent flows where Reynolds drives self-similar cascades, this is a dramatic expansion of the accessible regime.

35.5 Application to Strang kernel

The stellar2d Strang kernel uses LM-HLLC by default (`use_lm_fix = true` at all production sites; `use_lm_fix = false` is a test flag only). The kernel's intended use case is stratified atmospheres with convective motions at $M \sim 10^{-2}$ to 10^{-3}; LM-HLLC is the appropriate choice for these applications.

For tests that probe the **scheme's convergence theory** (§D2 linwave), `use_lm_fix = false` is used to recover the standard HLLC behaviour; this is a configuration choice, not a kernel defect.

35.6 Verification checkpoints

1. ν_{eff} **measurement under standard HLLC.** On a smoothly-driven low-Mach flow ($u_{\text{conv}} = 10^{-2}c$) at $n_x = 512$ with `use_lm_fix = false`, measure the decay rate of a tagged perturbation; extract ν_{eff} from the decay coefficient. Expected $\approx c\Delta x/2 = 5\times10^{-4}$ (well above the physical viscosity $\nu = 0$). Test: scheme-char probe.
2. ν_{eff} **measurement under LM-HLLC.** Same setup with `use_lm_fix = true`; expected $\nu_{\text{eff}} \approx M_{\text{loc}} c\Delta x/2 = 5 \times 10^{-6}$ — $100\times$ lower. Test: scheme-char probe.
3. **Ratio check.** The measured ratio $\nu_{\text{std}}/\nu_{\text{LM}} \approx 100$ matches $1/M_{\text{loc}} = 100$. Test: scheme-char probe.
4. **Reynolds crossover at M_{cut}.** Running at progressively lower M and measuring ν_{eff}, observe the clamp activate around $M_{\text{loc}} = 10^{-3}$. Below, ν_{eff} stays constant at the clamped value. Test: scheme-char probe sweep over M_{loc}.

Failure of (1) or (2) far from the predicted ν_{eff} indicates a dispersion-analysis error; revisit §E2. Failure of (3) is a direct diagnostic of LM branch breakage.

36 E4. Strang-split gravity source commutator

> **sympy script:** `scripts/e04_strang_split_source_commutator.py` **generated LaTeX:** `output/e04_strang_split_source_commutator.latex.tex` **verified:** - 4 non-commutative polynomial residual coefficients at Δt^2 showing the wrong (separate-$\mathcal{Z}$) operator-chain fails at Δt^2 with the commutator structure $\pm\frac{\Delta t^2}{2}[\mathcal{Z}, \mathcal{X}]$ and $\pm\frac{\Delta t^2}{2}[\mathcal{Z}, \mathcal{Y}_{\text{hyd}}]$
>
> **code checkpoints:** - `src/gpu/explicit/strang_solver.cu :: k_hllc_update_y` (gravity source `S_my, S_E` applied INSIDE the y-sweep kernel, line 513-523). The operator chain is X(Dt/2) * Y_total(Dt) * X(Dt/2), not a separate Z-operator.

The Strang kernel absorbs the gravity source **into** the y-direction operator: the update `k_hllc_update_y` computes the full HLLC flux divergence **and** adds the gravity source in the same kernel call, treating them as a single $\mathcal{Y}_{\text{total}} = \mathcal{Y}_{\text{hyd}} + \mathcal{Y}_{\text{grav}}$ operator. This is structurally critical for the 2nd-order accuracy claim (§A13).

If one instead split gravity as a **separate** $\mathcal{Z}$ operator applied **after** the Strang chain, the composite would be:

$$L_{\text{wrong}}(\Delta t) \;=\; e^{\Delta t\,\mathcal{Z}}\,\underbrace{e^{(\Delta t/2)\mathcal{X}}\,e^{\Delta t\,\mathcal{Y}_{\text{hyd}}}\,e^{(\Delta t/2)\mathcal{X}}}_{\text{Strang chain}}, \quad \text{(E4-wrong)}$$

which introduces a Lie-type step for $\mathcal{Z}$ after the Strang chain. Per §A14, Lie splitting is 1st-order — the composite becomes 1st-order for the $\mathcal{Z}$ piece even though the inner Strang chain is 2nd-order.

36.1 Correct (kernel's) choice

$$L_{\text{correct}}(\Delta t) \;=\; e^{(\Delta t/2)\mathcal{X}}\,e^{\Delta t\,(\mathcal{Y}_{\text{hyd}}+\mathcal{Y}_{\text{grav}})}\,e^{(\Delta t/2)\mathcal{X}}. \quad \text{(E4-correct)}$$

Treating $\mathcal{Y}_{\text{total}} = \mathcal{Y}_{\text{hyd}} + \mathcal{Y}_{\text{grav}}$ as a single operator, §A13's Strang proof applies directly: the residual against $e^{\Delta t(\mathcal{X}+\mathcal{Y}_{\text{tot}})}$ is $O(\Delta t^3)$, 2nd-order globally.

36.2 Wrong-chain leading error (sympy verified)

For the wrong composite L_{wrong} vs. the exact $e^{\Delta t(\mathcal{X}+\mathcal{Y}_{\text{hyd}}+\mathcal{Z})}$, sympy computes the Δt^2 residual monomial coefficients:

monomial	coefficient
$\mathcal{Z}\mathcal{X}$	$+\Delta t^2/2$
$\mathcal{X}\mathcal{Z}$	$-\Delta t^2/2$
$\mathcal{Z}\mathcal{Y}_{\text{hyd}}$	$+\Delta t^2/2$
$\mathcal{Y}_{\text{hyd}}\mathcal{Z}$	$-\Delta t^2/2$
$\mathcal{X}\mathcal{Y}_{\text{hyd}}, \mathcal{Y}_{\text{hyd}}\mathcal{X}$	0 (Strang chain already cancels)

Summed, the residual is

$$L_{\text{wrong}}(\Delta t) \;-\; e^{\Delta t\,(\mathcal{X}+\mathcal{Y}_{\text{hyd}}+\mathcal{Z})} \;=\; \frac{\Delta t^2}{2}\,[\mathcal{Z},\;\mathcal{X}+\mathcal{Y}_{\text{hyd}}] \;+\; O(\Delta t^3). \quad \text{(E4-residual)}$$

The non-zero Δt^2 term indicates 1st-order global error (via time-reversal analysis of §A14): the Δt^1 error in the forward-only update comes from the Δt^2 residual in the composition $L_{\text{wrong}}L_{\text{wrong}}^{-1}$.

36.3 Why gravity can go inside $\mathcal{Y}$

The gravity source contributes only to the y-momentum and energy components ($\mathcal{S} = (0, 0, -\rho g, -m_y g)$, §C1). These are both components whose physical y-sweep update is non-trivial (the y-momentum flux contains $\rho v^2 + P$, and the energy flux contains the pressure term $\bar{p}$). Gravity balances these flux components pointwise by §B3 + §C1, so absorbing it into the y-operator is natural — the operator is "what happens to the cell during the y-direction update".

If the physics had gravity that coupled to x-momentum (e.g., a rotating frame or a non-aligned gravity vector), it would need a separate treatment or a more complex splitting.

36.4 Kernel implementation verification

The gravity application in `k_hllc_update_y` (strang_solver.cu line 513-523) is called **inside** the y-sweep kernel, after the HLLC flux computation and before the state write-back:

```
// Compute HLLC fluxes GT, GB at top and bottom faces.
// ...
// Apply gravity source INSIDE y-sweep (not outside):
double rho_total = fmax(d_rho[k0] + rho_bg, 1e-20);
double S_my = -rho_total * g_grav;
double S_E  = -d_my[k0] * g_grav;
// Combined update: flux divergence + gravity source.
d_my[k0] = d_my[k0] - dtdy * (GT_mn - GB_mn) + dt * S_my;
d_E [k0] = d_E [k0] - dtdy * (GT3   - GB3)   + dt * S_E;
```

This is the correct `L_correct` structure. No separate gravity step is called before or after the Strang chain.

36.5 Verification checkpoints

1. **Absence of separate Z operator.** Code inspection: `grep -n "g_grav" strang_solver.cu` — all occurrences must be inside `k_hllc_update_y` or in init code (for HSE background); none in a standalone `apply_gravity` function. Test: `test_strang_unit.cu` §E4-no-Z-operator.
2. **Convergence rate on non-trivial gravity problem.** At §D5 bubble IC (which has gravity, and hence probes the kernel's gravity-inclusive Strang order), slope $p \in [1.8, 2.2]$. Test: `test_strang_convergence.cu` §E4-bubble-order.
3. **Symmetry under source-flip.** Running a canonical test with gravity sign reversed (`g_grav = -g` instead of `+g`) should give results related by a symmetry (the atmosphere "upside down"). This is primarily a consistency check for gravity's pure-source nature. Test: optional scheme-char diagnostic.

Failure of (1) means someone has added a separate gravity operator call somewhere, degrading the scheme to 1st-order. Failure of (2) with slope $\ll 1.8$ on a gravity-inclusive test might indicate (1), or a deeper §C1 gravity-source bug.

37 E5. Long-time HSE drift bound

sympy script: `scripts/e05_hse_drift_bound.py` **generated LaTeX:** `output/e05_hse_drift_bound.latex.tex` **generated goldens:** `output/e05_hse_drift_bound.goldens.json` **verified:** - 1 linear-summation drift bound ($\varepsilon_{\text{mach}} \cdot N_{\text{step}} \cdot \kappa(\text{HSE})$) - 1 Kahan-summation drift bound ($\varepsilon_{\text{mach}}\sqrt{N}\kappa$, not used in kernel)

code checkpoints: - composite of §B2, §B3, §C1, §D6 - measured by `tests/test_strang_step.cu` §D6-long-time

Quantifies the maximum drift of the stored state away from zero on pure HSE IC over long time evolution. The kernel uses standard (non-Kahan) summation for its accumulators, so the drift is **linear in step count**:

$$\max_t \|\delta U\|_\infty(N_{\text{step}}) \;\le\; \varepsilon_{\text{mach}}\, N_{\text{step}}\, \kappa(\text{HSE}), \quad \text{(E5-bound)}$$

where $\kappa(\mathrm{HSE})$ is a problem-dependent condition number that captures how much the floating-point errors in evaluating the HSE flux divergence and gravity source get amplified by the ratio of background scales across the domain.

37.1 Condition number

The worst case is a floating-point subtraction of two nearly-equal but large numbers: $\bar{p}(y_{j+1/2}) - \bar{p}(y_{j-1/2})$ in the flux divergence minus the gravity-source contribution $\bar{\rho}_j g \Delta y$. The relative error in either term is $\sim \varepsilon_{\text{mach}}$; the absolute error is $\varepsilon_{\text{mach}} \bar{p}$ per flux evaluation.

The condition number

$$\kappa(\mathrm{HSE}) \;\sim\; \frac{\max_y \bar{\rho}(y)}{\min_y \bar{\rho}(y)} \;\sim\; \left(\frac{1}{1 - L_y/y^\star}\right)^{1/(\gamma-1)} \qquad \text{(E5-kappa)}$$

captures the ratio of the largest background density to the smallest over the domain. For the canonical HSE setup ($L_y = 1$, $y^\star = 3.5$, $\gamma = 1.4$):

$$\kappa \;=\; \left(\frac{1}{1 - 1/3.5}\right)^{1/0.4} \;=\; (7/5)^{2.5} \;\approx\; 2.32. \quad \text{(E5-canonical)}$$

This is a very gentle condition number — the atmosphere is not deeply stratified within the canonical domain. For a deeper domain ($L_y \to y^\star$), κ grows rapidly; at $L_y = 0.99 y^\star$, $\kappa \sim 10^5$ and the HSE drift becomes a limiting factor.

37.2 Numerical drift prediction

At $\varepsilon_{\text{mach}} = 2.22 \times 10^{-16}$, $\kappa = 2.32$, $N = 1000$:

- **Linear summation (kernel default):** 5.1×10^{-13}.
- **Kahan summation (not implemented):** 1.6×10^{-14}.

The linear bound is well below the typical test tolerance of 10^{-10}, so the kernel should pass §D6's long-time test with ample margin.

37.3 Kahan summation option

If performance could accommodate it, Kahan summation in the flux divergence accumulator would reduce the drift to $\varepsilon_{\text{mach}} \sqrt{N} \kappa$:

$$\max_t \|\delta U\|_\infty(N) \;\lesssim\; \varepsilon_{\text{mach}} \sqrt{N}\, \kappa(\mathrm{HSE}). \quad \text{(E5-kahan)}$$

For $N = 10^6$, this gives $\sim 5 \times 10^{-13}$ vs. linear's $\sim 5 \times 10^{-10}$ — a factor 10^3 improvement. The kernel currently does not use Kahan because the tolerance budget is ample; a future optimisation could revisit this if very long HSE evolution becomes a primary use case.

37.4 Drift diagnostics from §D6

§D6's long-time test runs $N = 1000$ steps and requires $\|\delta U\|_\infty \leq 10^{-10}$. §E5 predicts $\sim 5 \times 10^{-13}$, a factor $200\times$ below tolerance. If the kernel's measured drift is close to 10^{-10} (near tolerance), the κ is much higher than predicted — either the domain is too deep, or the HSE ODE discretisation is weaker than 2nd-order.

37.5 Regression-test robustness

The drift test at $N = 10^4$ would bring the predicted drift to $\sim 5 \times 10^{-12}$ — still well below 10^{-10}. At $N = 10^5$, $\sim 5 \times 10^{-11}$ — approaching tolerance. This is why the §D6 test uses $N = 1000$: a comfortable but realistic time horizon for a regression check. For convective evolution tests (§D5 bubble, run time $T = 0.2$, dt ~ 0.001 so $N \sim 200$), the drift is negligible at $\sim 10^{-13}$.

37.6 Extending to non-HSE initial conditions

The analysis here is specific to **pure HSE** initial conditions where the stored state is identically zero. For general ICs with non-zero perturbations, the drift has two sources:

1. The HSE-preservation drift above ($\varepsilon_{\text{mach}} N\kappa$).
2. The physical perturbation evolution's discretisation error ($O(\Delta x^2)$ from §E1 modified equation).

The latter dominates for finite-amplitude perturbations. The §E5 bound applies only to the floor below which the kernel cannot preserve a quiescent HSE state — a diagnostic of the kernel's round-off accumulation, not of its physical convergence rate.

37.7 Verification checkpoints

1. $N = 1000$ **drift bound.** Measured $\|\delta U\|_\infty$ at $N = 1000$ is in $[\varepsilon_{\text{mach}}, 2 \times \varepsilon_{\text{mach}} N\kappa] = [2.2 \times 10^{-16}, 10^{-12}]$. Test: `test_strang_step.cu` §E5-drift-bound.
2. **Linear-in-N growth.** Running at $N \in \{100, 300, 1000, 3000\}$ and measuring the drift shows linear scaling in N. Test: `test_strang_step.cu` §E5-linear-scaling.
3. κ **scaling with domain depth.** Running the same $N = 1000$ test at three domain depths $L_y \in \{0.3, 0.7, 0.99\}y^\star$ shows drift scaling like $\kappa(L_y)$:
   - $L_y = 0.3y^\star$: $\kappa \sim 1.06$, drift $\sim 2 \times 10^{-13}$.
   - $L_y = 0.7y^\star$: $\kappa \sim 11$, drift $\sim 2 \times 10^{-12}$.
   - $L_y = 0.99y^\star$: $\kappa \sim 10^5$, drift $\sim 2 \times 10^{-8}$. Test: scheme-char probe (not a production regression).

Failure of (1) with drift $> 10^{-11}$ at $N = 1000$ indicates either a bug (likely §C1 or §B3) or a more-stratified test domain where $\kappa > 10$. Failure of (2) with super-linear growth (e.g., N^2) means the kernel is accumulating a **systematic** error, not round-off — a structural bug.